

# ENTREGA FINAL



---

Ingeniería Informática de Gestión y Sistemas de Información  
Análisis y Diseño de Sistemas de Información

---

**Grupo: REGICODERS**

**Integrantes:**

Sofia Granja

Ziyan Jiang

Ibai Munné

Aitana Niño

Mateo Ortiz

Diego Pomares

Adrián Vinagre

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| 1. INTRODUCCIÓN.....                                                                 | 3  |
| 2. DIAGRAMAS DE CASO DE USO Y CASO DE USO EXTENDIDOS.....                            | 4  |
| Diagrama de casos de uso.....                                                        | 4  |
| FUNCIONALIDAD 1: Gestión de usuarios – Ziyang Jiang.....                             | 5  |
| FUNCIONALIDAD 2: Crear Equipos Pokémon - Adrián Vinagre.....                         | 16 |
| FUNCIONALIDAD 3: ChangeLog - Aitana Niño.....                                        | 26 |
| FUNCIONALIDAD 4: Lista Completa de Pokémon y Búsquedas con Filtros – Ibai Munné..... | 28 |
| FUNCIONALIDAD 5: ChatBot.....                                                        | 32 |
| FUNCIONALIDAD 6: Bot de Telegram.....                                                | 37 |
| FUNCIONALIDAD 7: Pokedle -.....                                                      | 40 |
| 3. MODELO DE DOMINIO.....                                                            | 43 |
| Diagrama completo.....                                                               | 43 |
| Entidades.....                                                                       | 43 |
| Relaciones.....                                                                      | 47 |
| 1. Usuario-Usuario.....                                                              | 47 |
| 2. Usuario-Equipo.....                                                               | 47 |
| 3. Usuario-Fecha.....                                                                | 48 |
| 4. Equipo-Pokémon.....                                                               | 48 |
| 5. Pokémon-Especie.....                                                              | 49 |
| 6. Ataque-Tipo.....                                                                  | 49 |
| 7. Tipo -Tipo.....                                                                   | 50 |
| 8. Especie - Especie.....                                                            | 50 |
| 9. Especie - Ataque.....                                                             | 51 |
| 10. Pokémon- Ataque.....                                                             | 52 |
| 11. Especie - Tipo.....                                                              | 52 |
| 4. DIAGRAMA DE CLASES Y ESQUEMA DE BASE DE DATOS.....                                | 54 |
| 1. DIAGRAMA DE BASE DE DATOS.....                                                    | 54 |
| 2. DIAGRAMA DE CLASES.....                                                           | 55 |
| 2.1 Sistema Telegram + rework de controladores.....                                  | 55 |
| 5. DIAGRAMAS DE COMUNICACIÓN.....                                                    | 55 |
| 5.1 GESTIÓN DE USUARIOS.....                                                         | 55 |
| 5.1.1 Registrarse.....                                                               | 55 |
| 5.1.2 Iniciar sesión.....                                                            | 56 |
| 5.1.3 Actualizar datos.....                                                          | 58 |
| 5.1.4 Gestionar amigos.....                                                          | 59 |
| 5.1.5 Gestionar cuentas.....                                                         | 62 |
| 5.2 CREAR EQUIPOS POKÉMON.....                                                       | 64 |
| 5.2.1 Ver un equipo Pokémon.....                                                     | 64 |
| 5.2.1.1 Crear un Pokémon.....                                                        | 66 |
| 5.2.1.2 Crear una especie.....                                                       | 68 |
| 5.2.1.3 Crear un ataque.....                                                         | 69 |
| 5.2.1.4 Crear un tipo.....                                                           | 71 |
| 5.2.2 Eliminar un equipo Pokémon.....                                                | 72 |
| 5.2.3 Crear un nuevo equipo Pokémon.....                                             | 73 |

|                                                                                          |     |
|------------------------------------------------------------------------------------------|-----|
| 5.2.4 Añadir un Pokémon a un equipo.....                                                 | 74  |
| 5.2.5 Eliminar un Pokémon de un equipo.....                                              | 77  |
| 5.3 CHANGELOG.....                                                                       | 79  |
| 5.4 LISTA COMPLETA DE POKÉMON Y BÚSQUEDAS CON FILTRO.....                                | 81  |
| 5.5 CHATBOT.....                                                                         | 86  |
| 5.6 COMPARTIR A TRAVÉS DE TELEGRAM.....                                                  | 89  |
| 5.7 POKEDLE.....                                                                         | 90  |
| 6. DIAGRAMA DE SECUENCIA.....                                                            | 93  |
| FUNCIONALIDAD 1: Gestión de usuarios.....                                                | 93  |
| FUNCIONALIDAD 2: Equipos.....                                                            | 104 |
| 2.1 VER EQUIPO POKÉMON.....                                                              | 104 |
| 2.2 ELIMINAR UN EQUIPO POKÉMON.....                                                      | 106 |
| 2.3 CREAR UN NUEVO EQUIPO POKÉMON.....                                                   | 107 |
| 2.4 AÑADIR UN POKÉMON A UN EQUIPO.....                                                   | 108 |
| 2.5 EXPULSAR UN POKÉMON DE UN EQUIPO.....                                                | 111 |
| FUNCIONALIDAD 3: ChangeLog.....                                                          | 112 |
| FUNCIONALIDAD 5: ChatBot.....                                                            | 114 |
| 5.2 OBTENER MEJOR POKEMON DE UN EQUIPO:.....                                             | 114 |
| 5.3 BUSCAR POKEMON:.....                                                                 | 115 |
| 5.4 VER ESTADÍSTICAS:.....                                                               | 116 |
| 5.5 VER CADENA EVOLUTIVA:.....                                                           | 117 |
| 5.6 VER RIVALES:.....                                                                    | 118 |
| 5.7 CARGAR EQUIPOS:.....                                                                 | 119 |
| FUNCIONALIDAD 6: Bot de Telegram.....                                                    | 120 |
| FUNCIONALIDAD 6: Bot de Telegram.....                                                    | 122 |
| 7. PLANES DE PRUEBA E IMPLEMENTACIÓN.....                                                | 123 |
| FUNCIONALIDAD 1: Gestión de usuarios - Ibai Munné.....                                   | 123 |
| FUNCIONALIDAD 2: Crear equipos Pokémon - Aitana Niño.....                                | 126 |
| FUNCIONALIDAD 3: ChangeLog - Ziyang Jiang.....                                           | 128 |
| FUNCIONALIDAD 4: Lista Completa de Pokémon y Búsquedas con Filtros - Adrián Vinagre..... | 130 |
| FUNCIONALIDAD 5: ChatBot - Diego Pomares.....                                            | 134 |
| FUNCIONALIDAD 6: Compartir equipo a través de Telegram - Sofía Granja.....               | 135 |
| FUNCIONALIDAD 7: Pokedle - Mateo Ortiz.....                                              | 136 |
| 8. DESCRIPCIÓN DE LOS PROBLEMAS DE IMPLEMENTACIÓN.....                                   | 137 |
| 9. CONCLUSIONES.....                                                                     | 138 |

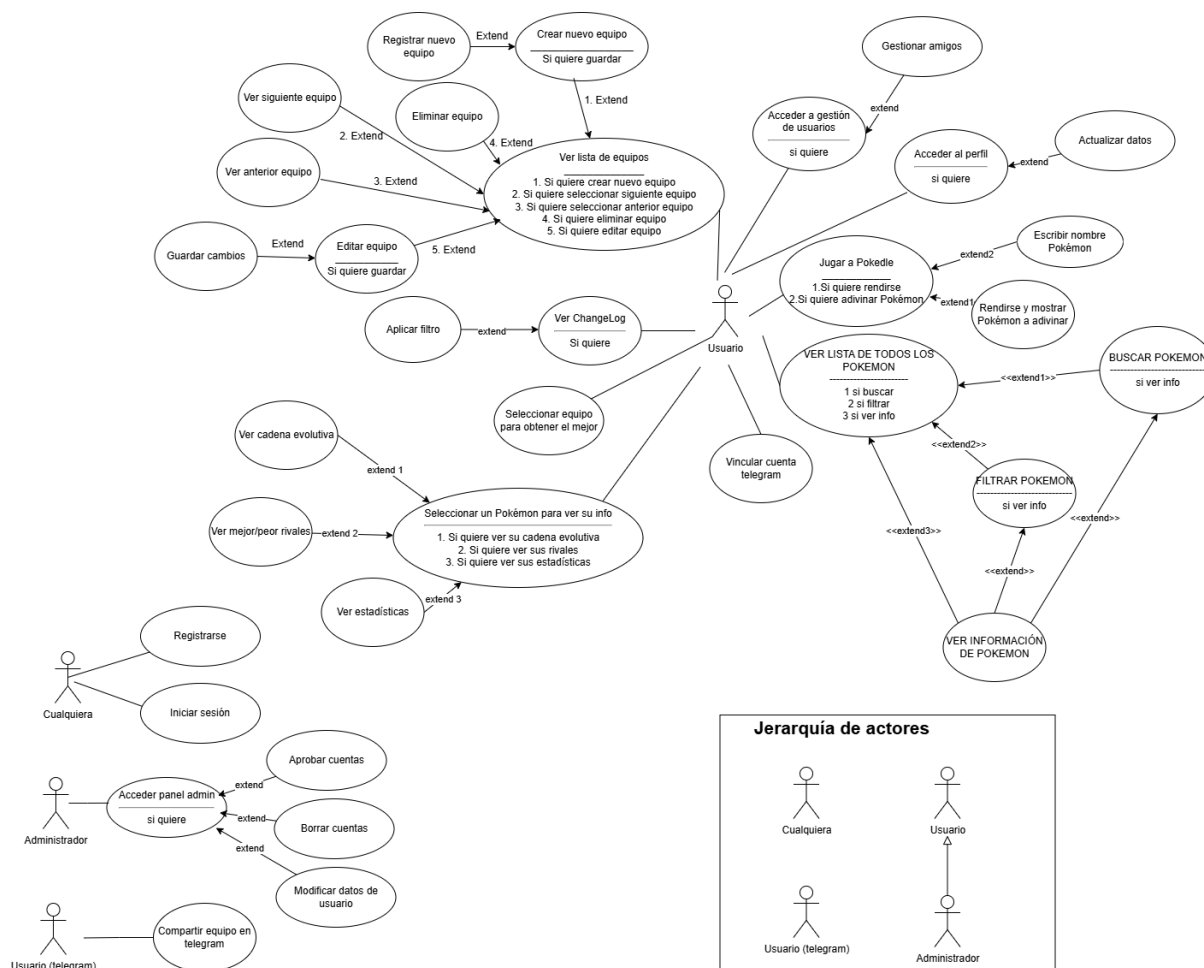
# **1. INTRODUCCIÓN**

El presente documento es la entrega final del proyecto de la parte práctica de la asignatura Análisis y Diseño de Sistemas de Información del curso 2025/2026 del grupo de trabajo bajo el nombre de Regicoders. Es la última entrega incremental.



## 2. DIAGRAMAS DE CASO DE USO Y CASO DE USO EXTENDIDOS

### Diagrama de casos de uso



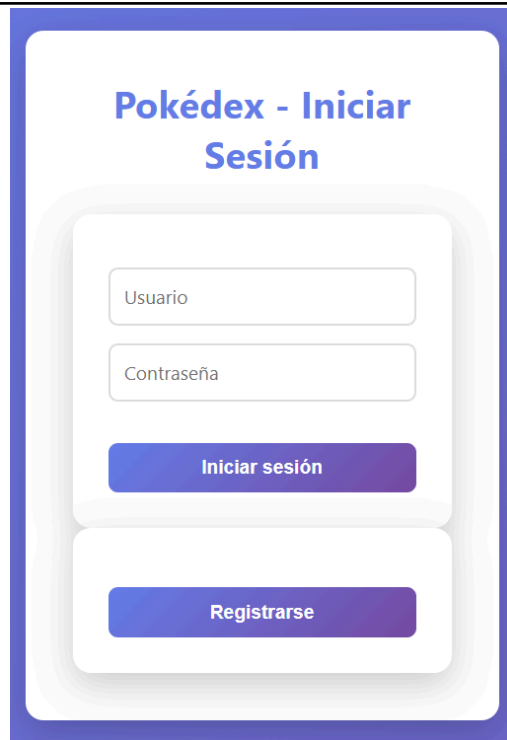
*Ilustración 1 - Diagrama de Casos de Uso*

Explicación de actores:

- Usuario: persona identificada, con una cuenta activa. Puede usar la mayoría de las funcionalidades (crear equipo de Pokémon, acceder al perfil, ChangeLog, ChatBot o Pokedle)
- Cualquiera: actor no identificado en el sistema. Puede registrarse e iniciar sesión.
- Usuario (telegram): usuario del sistema que tiene vinculada su cuenta con Telegram.
- Administrador: actor identificado con privilegios especiales. Puede aprobar, borrar y modificar cuentas de usuario. Además, hereda las mismas funcionalidades que puede ejecutar el usuario.

## FUNCIONALIDAD 1: Gestión de usuarios – Ziyang Jiang

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <pre>graph LR; A((Cualquiera)) --- B([Registrarse]);</pre> <p>A UML Use Case Diagram showing an actor labeled 'Cualquiera' connected by a line to an oval use case labeled 'Registrarse'.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                           |             |
| <b>Nombre:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Registrarse |
| <b>Descripción:</b> Permite a un nuevo usuario crear una cuenta en el sistema de Pokédex.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |             |
| <b>Actores:</b> Cualquiera                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |             |
| <b>Precondiciones:</b> -                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |             |
| <b>Requisitos no funcionales:</b> -                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |             |
| <b>Flujo de Eventos:</b> <ol style="list-style-type: none"><li>1. El usuario selecciona la opción “Registrarse” de la pantalla de inicio.</li><li>2. El sistema muestra el formulario de registro con campos: username, email, contraseña, confirmación de contraseña.</li><li>3. El usuario completa los campos requeridos.</li><li>4. El sistema valida los datos.</li><li>5. El sistema crea la cuenta con estado “Pendiente de aprobación”.</li><li>6. El sistema muestra un mensaje de éxito: “Cuenta creada. Pendiente de aprobación administrativa”.</li><li>7. El sistema redirige al usuario a la pantalla de Login.</li></ol> |             |
| <b>Postcondiciones:</b> Se crea una nueva cuenta de usuario en estado “pendiente”.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |             |
| <b>Interfaz Gráfica:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |             |



**Pokédex - Iniciar Sesión**

Usuario

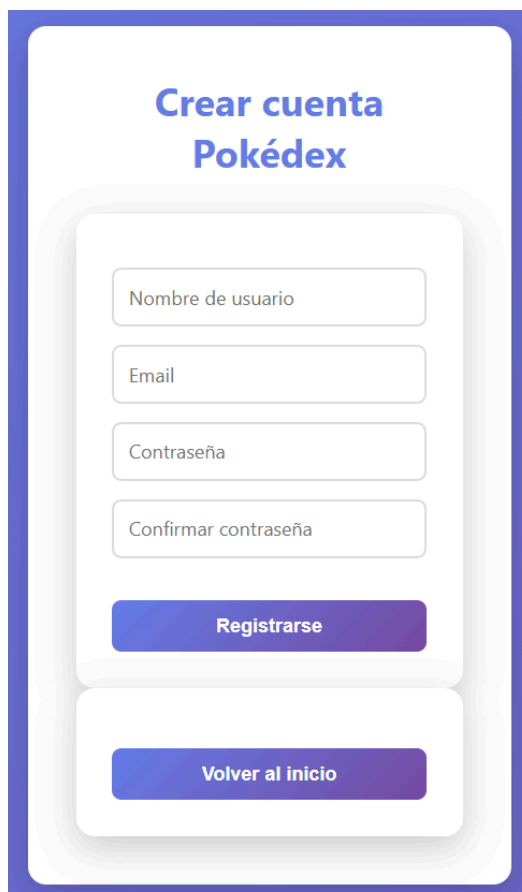
Contraseña

Iniciar sesión

Registrarse

This is a login form for 'Pokédex'. It features a title 'Pokédex - Iniciar Sesión' in blue. Below the title, there are two input fields: 'Usuario' and 'Contraseña'. Under the 'Contraseña' field is a blue button labeled 'Iniciar sesión'. At the bottom of the form is another blue button labeled 'Registrarse'. The entire form is enclosed in a white rounded rectangle with a subtle shadow, set against a blue background.

*Ilustración 2 - Menú de inicio de sesión*



**Crear cuenta Pokédex**

Nombre de usuario

Email

Contraseña

Confirmar contraseña

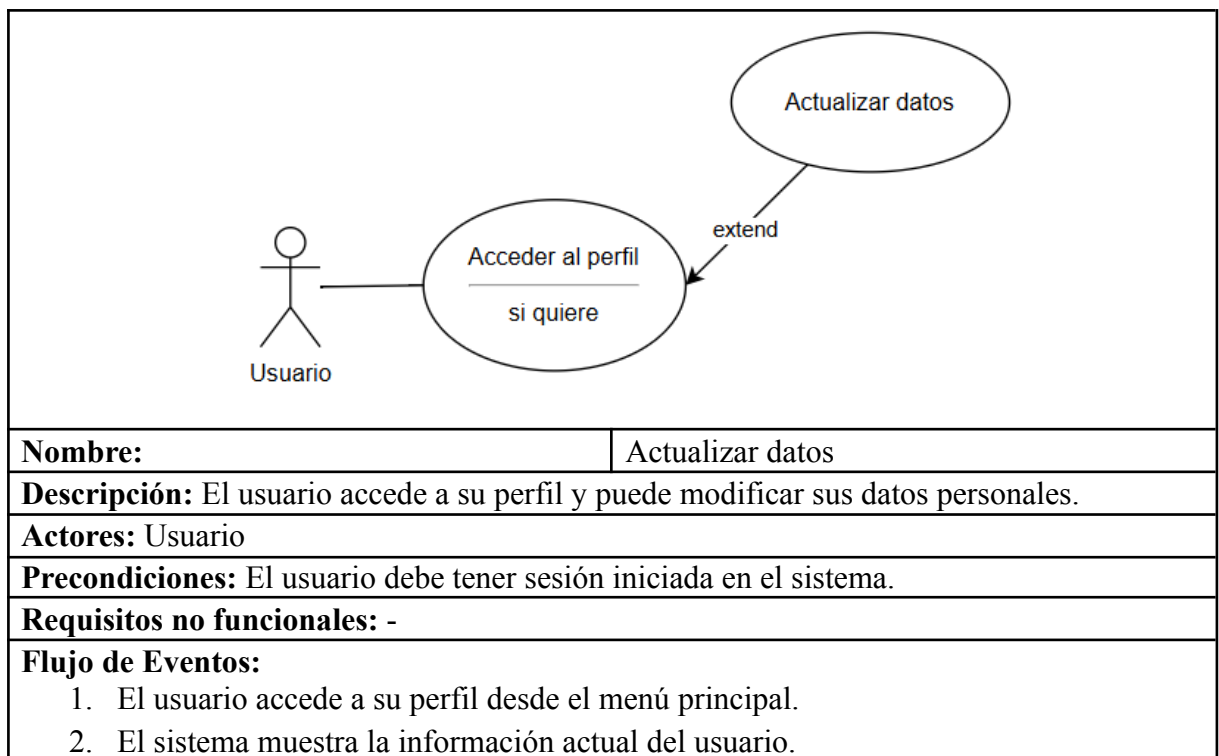
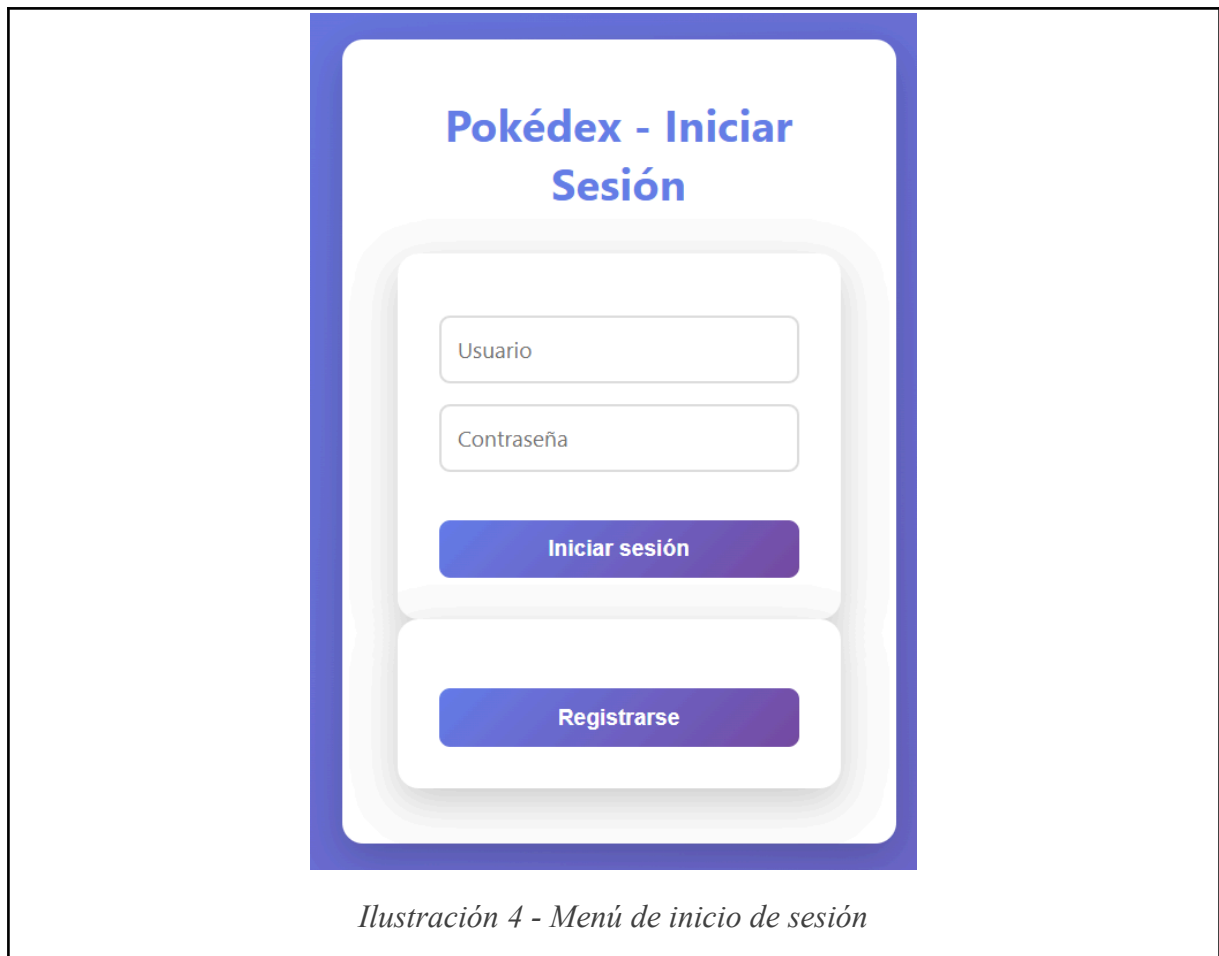
Registrarse

Volver al inicio

This is a registration form for 'Pokédex'. It features a title 'Crear cuenta Pokédex' in blue. Below the title, there are four input fields: 'Nombre de usuario', 'Email', 'Contraseña', and 'Confirmar contraseña'. Under the 'Confirmar contraseña' field is a blue button labeled 'Registrarse'. At the bottom of the form is another blue button labeled 'Volver al inicio'. The entire form is enclosed in a white rounded rectangle with a subtle shadow, set against a blue background.

*Ilustración 3 – Registrar cuenta*

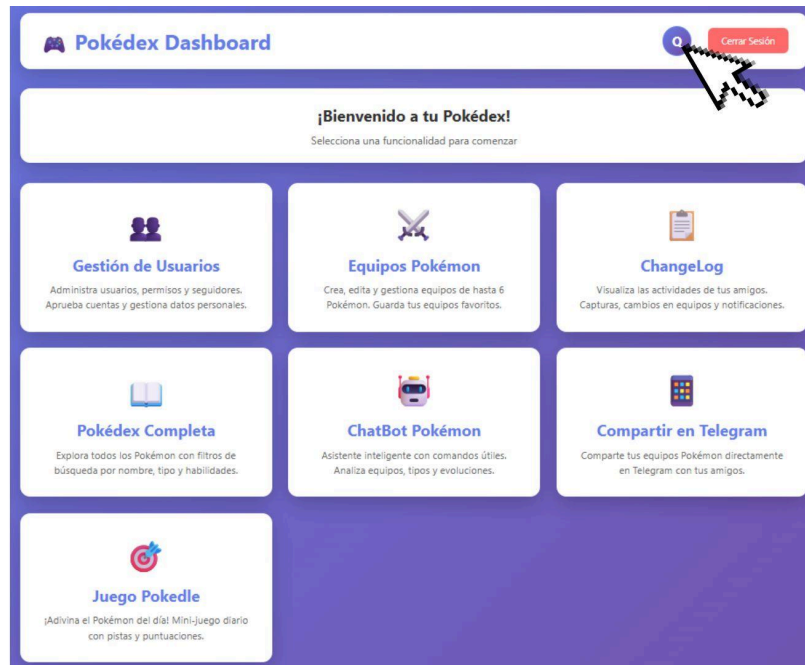
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <pre> graph LR     Actor[Cualquiera] --- UseCase([Iniciar sesión]) </pre> <p>The diagram shows a stick figure actor labeled 'Cualquiera' connected by a line to an oval use case labeled 'Iniciar sesión'.</p>                                                                                                                                                                                                                                                                                                                                       |                |
| <b>Nombre:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Iniciar sesión |
| <b>Descripción:</b> El usuario autentica su identidad mediante credenciales para acceder al sistema.                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                |
| <b>Actores:</b> Cualquiera                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                |
| <b>Precondiciones:</b> el actor es cualquier persona no identificada y debe existir una cuenta activa y aprobada para este usuario.                                                                                                                                                                                                                                                                                                                                                                                                                  |                |
| <b>Requisitos no funcionales:</b> -                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                |
| <b>Flujo de Eventos:</b> <ol style="list-style-type: none"> <li>1. El usuario accede a la pantalla de login.</li> <li>2. Ingresa credenciales (usuario y contraseña) y hace clic en “Start” para iniciar sesión.</li> <li>3. El sistema valida: la existencia del usuario, contraseña y estado de la cuenta (debe ser “aprobada”).</li> <li>4. Si las credenciales son correctas y la cuenta está activa: el sistema crea una sesión de usuario.</li> <li>5. Si las credenciales son incorrectas: muestra un mensaje de error específico.</li> </ol> |                |
| <b>Postcondiciones:</b> Se activa la sesión del usuario en el sistema.                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                |
| <b>Interfaz Gráfica:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                |



3. El usuario modifica los datos deseados.
4. El sistema valida en tiempo real los cambios realizados y actualiza la información en la base de datos.
5. El sistema muestra un mensaje de confirmación: “Perfil actualizado correctamente”.

**Postcondiciones:** Los datos quedan actualizados en la base de datos.

**Interfaz Gráfica:**



*Ilustración 5 – Menú principal del usuario – Perfil*

## Mi perfil

Usuario: **qwerty**

Correo electrónico

qwerty@cualquiera.com

URL de foto de perfil



https://ejemplo.com/mi-foto.png

Copia y pega la dirección de una imagen de internet

CAMBIAR CONTRASEÑA (DEJAR EN BLANCO PARA MANTENER LA ACTUAL)

Nueva contraseña

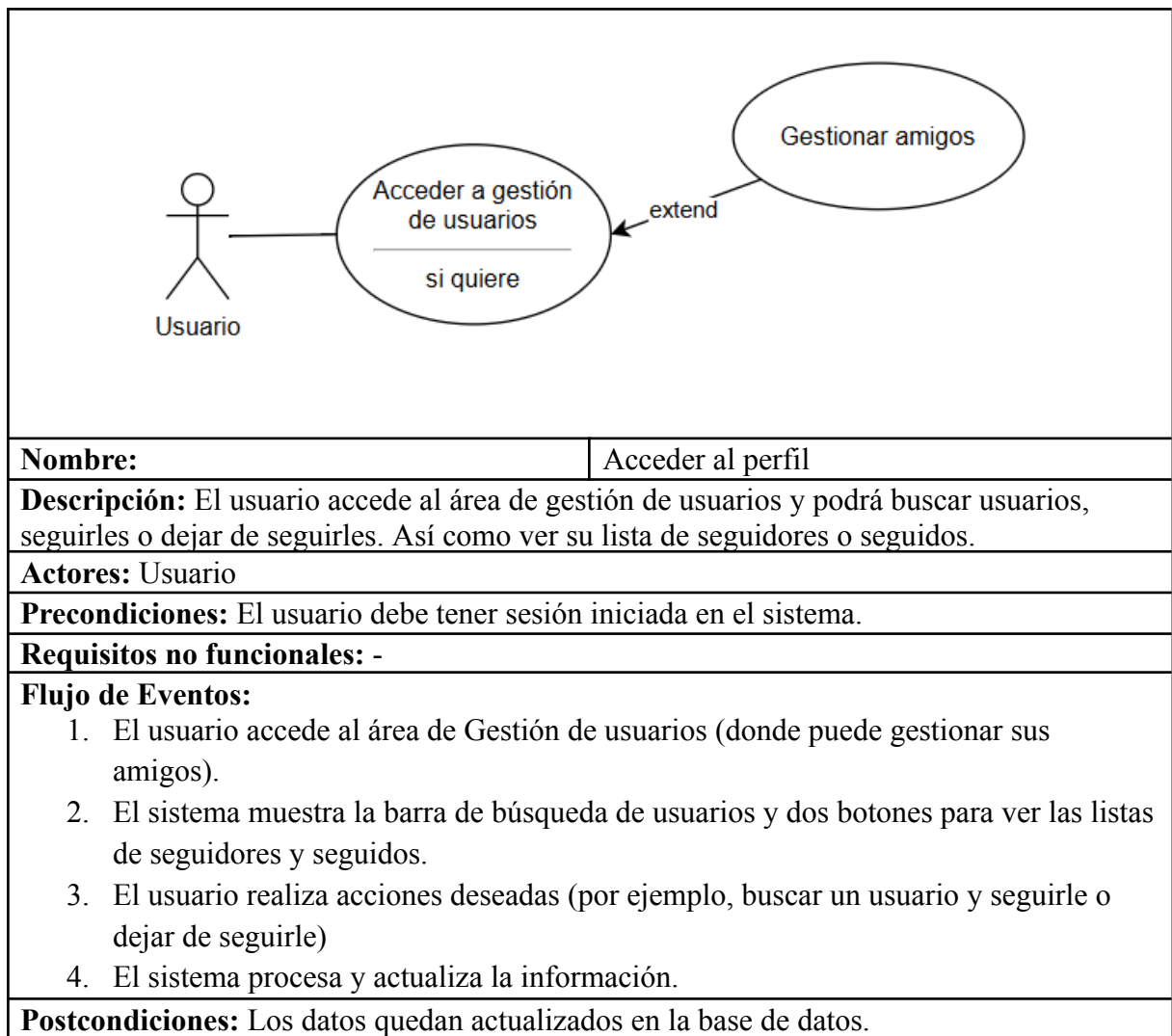
Mínimo 8 caracteres

Confirmar nueva contraseña

Guardar cambios

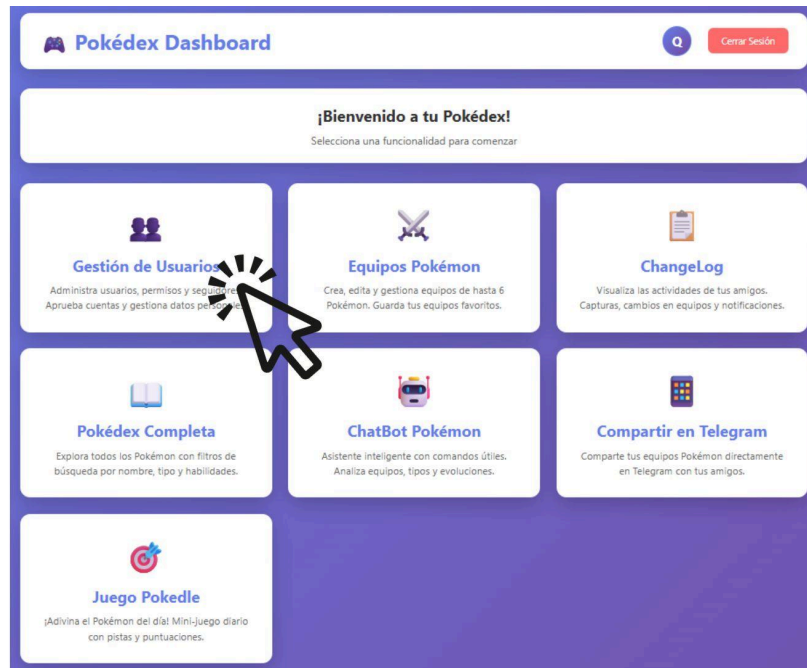
[← Volver al Dashboard](#)

*Ilustración 6 – Pantalla para editar datos*

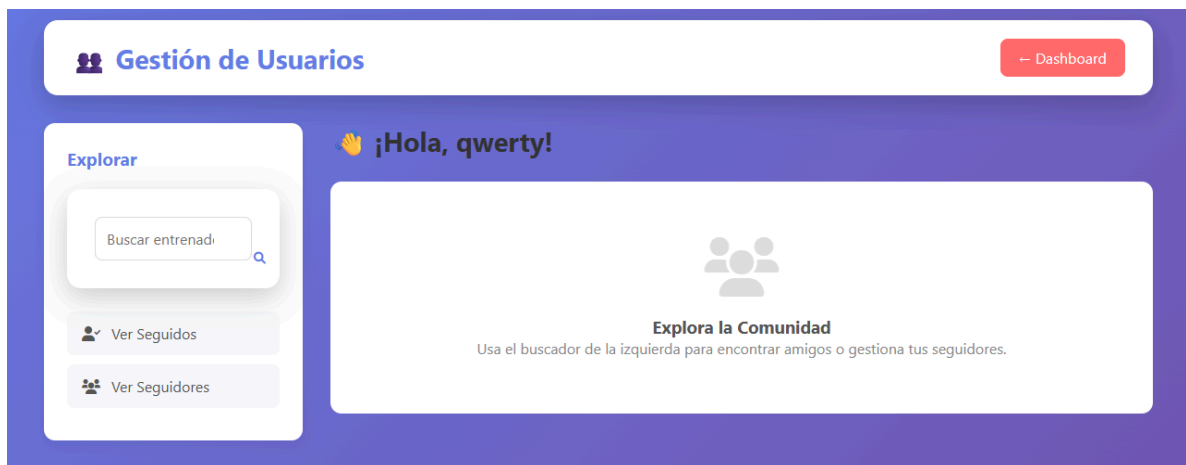




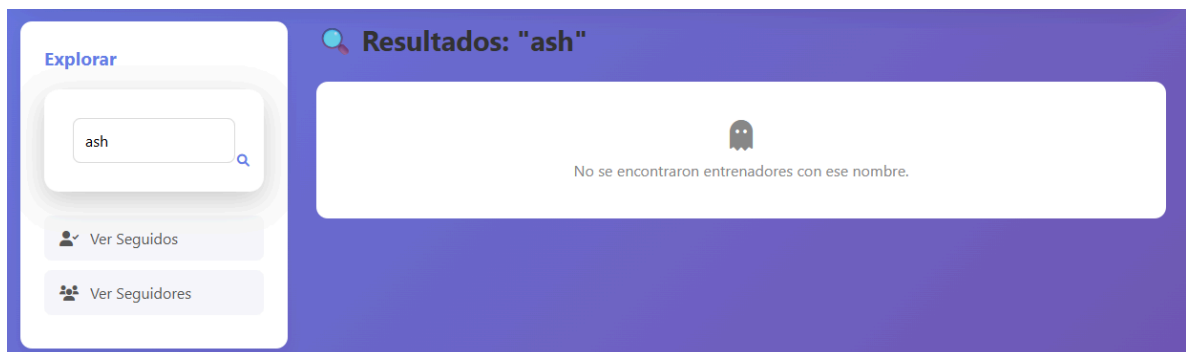
## Interfaz Gráfica:



*Ilustración 7 – Menú principal del usuario*



*Ilustración 8 - Gestión de amigos*



*Ilustración 8.1 - Pantalla de usuario no encontrado*

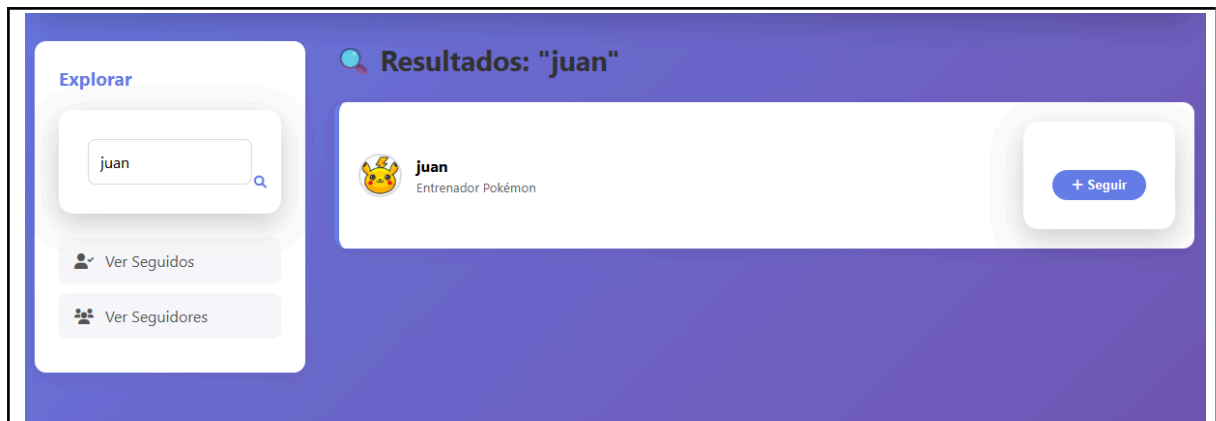


Ilustración 8.2 - “Botón Seguir”

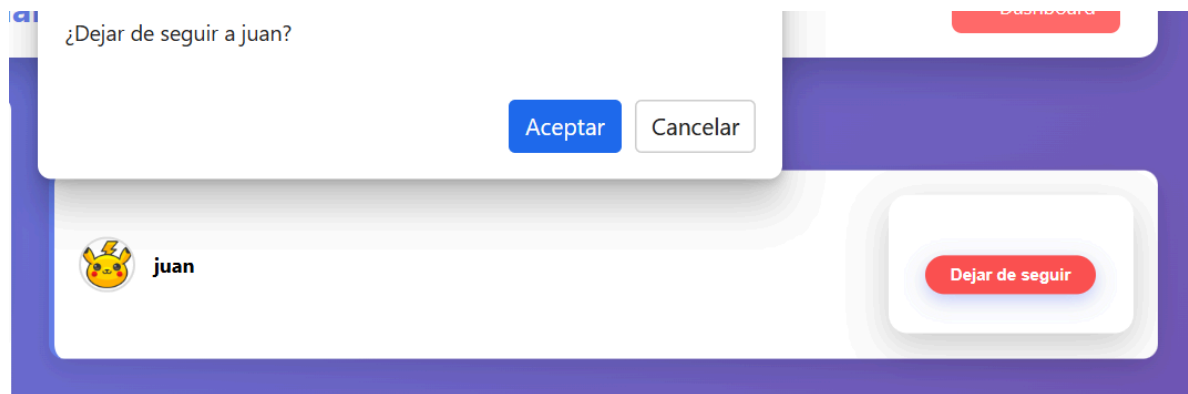
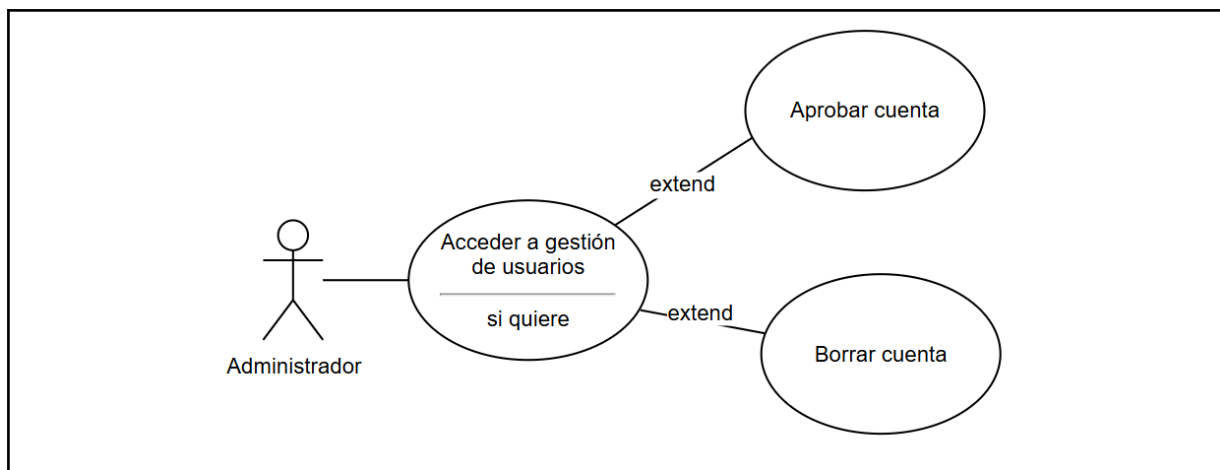


Ilustración 8.3 - Botón “Dejar de seguir”

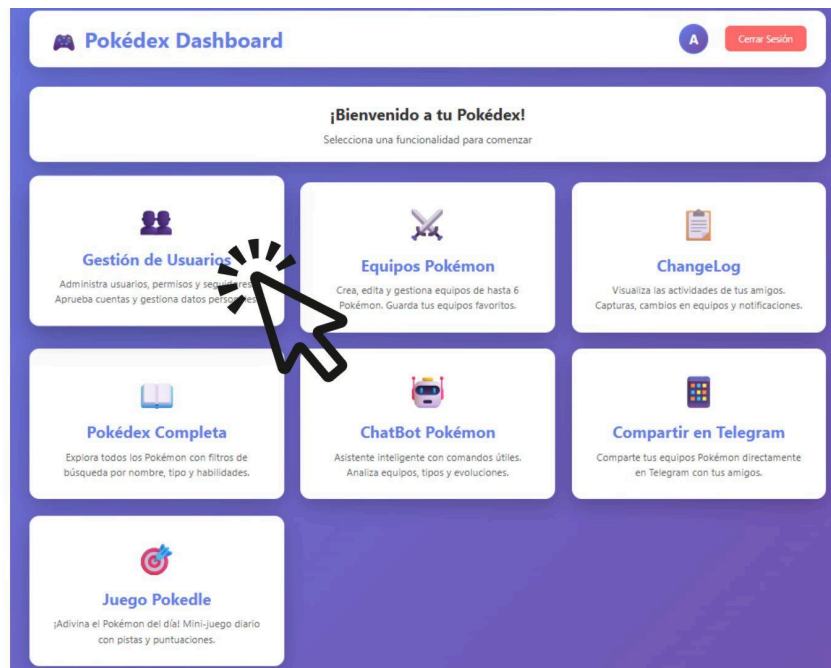


|                                                                                                                                   |                   |
|-----------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <b>Nombre:</b>                                                                                                                    | Gestionar cuentas |
| <b>Descripción:</b> El administrador podrá aprobar cuentas de usuarios pendientes para activarlas en el sistema o borrar cuentas. |                   |
| <b>Actores:</b> Administrador                                                                                                     |                   |
| <b>Precondiciones:</b> El administrador debe tener sesión iniciada y debe existir la cuenta que quiera aprobar o borrar.          |                   |
| <b>Requisitos no funcionales:</b> -                                                                                               |                   |

**Flujo de Eventos:**

1. El administrador accede al panel de administración (Gestión de usuarios).
1. El sistema muestra las distintas opciones que puede realizar el administrador:
  - a. Aprobar cuentas
    - i. El administrador revisa los datos de cada cuenta pendiente y puede dar a “Aprobar”.
    - ii. Al aprobar una cuenta: el sistema actualiza el estado a “aprobado” y quita la cuenta aprobada de la lista de pendientes.
  - b. Borrar cuentas
    - i. El administrador selecciona una cuenta a borrar.
2. El sistema muestra confirmación de la acción realizada.
3. El sistema valida los cambios y el administrador confirma modificaciones.
4. El sistema actualiza la base de datos.

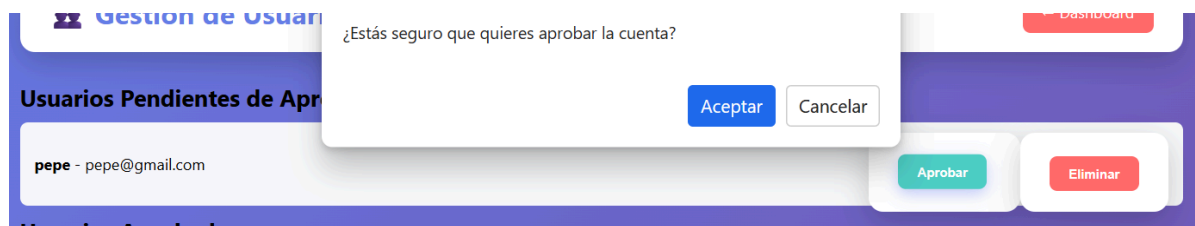
**Postcondiciones:** Se refleja los cambios realizados.

**Interfaz Gráfica:**

*Ilustración 9 - Menú principal del administrador*



*Ilustración 10 – Pantalla de gestión de cuentas*



*Ilustración 10.1 – Aprobar cuenta*

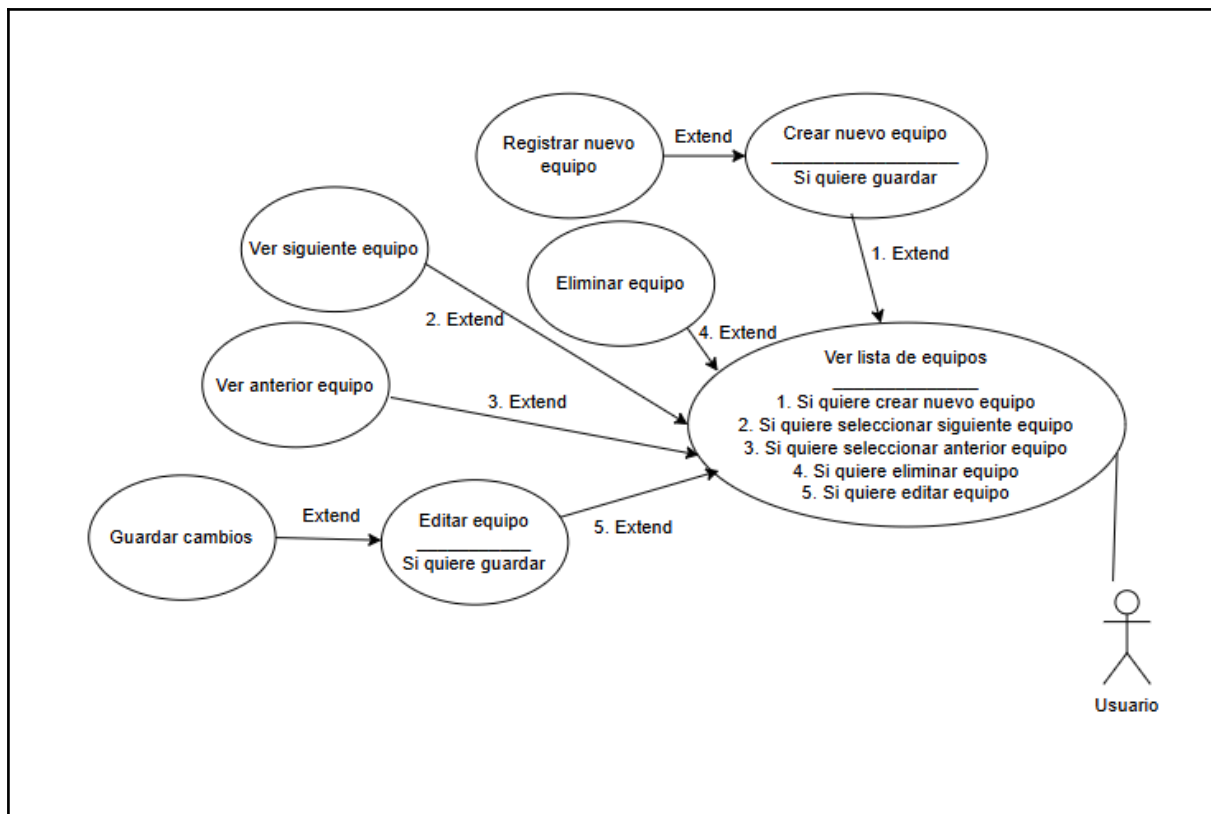
¿Estás seguro que quieres borrar la cuenta?

Aceptar

Cancelar

*Ilustración 10.2 - Eliminar cuenta*

## FUNCIONALIDAD 2: Crear Equipos Pokémon - Adrián Vinagre



**Nombre:**

Crear equipos Pokémon

**Descripción:** Permite ver un listado de todos los equipos Pokémon anteriormente creados. Además, permite editarlos, eliminarlos, e incluso crear un nuevo equipo desde cero.

**Actores:** Usuario

**Precondiciones:** Estar identificado como usuario

**Requisitos no funcionales:** Se tiene un límite máximo de 10 equipos Pokémon almacenados por usuario. No se puede superar la cantidad de 6 Pokémon en un equipo.

**Flujo de Eventos:**

1.- El Usuario selecciona “Ver equipos” en el menú principal. (Ilustración 11)

2.- Se muestra al usuario la lista de equipos (que haya creado anteriormente). El usuario puede ver los distintos equipos de la lista deslizando hacia abajo. En cada equipo, aparecerá un mensaje que anima al usuario a añadir Pokémon a su equipo, si ese está vacío. Por el contrario, aparecerán 6 ranuras destinadas a Pokémon, si al menos hay un Pokémon en el equipo. En cada ranura, se puede observar la foto del Pokémon, su nombre, y su especie, si hay un Pokémon en la ranura. Si no hay ningún Pokémon en la ranura, aparecerá un mensaje de “vacío” (Ilustraciones 12 ,19).

[Si el usuario pulsa sobre “Crear nuevo equipo”]

3.- Se crea un nuevo equipo y se añade al final de la lista de equipos, en la parte inferior de la aplicación. Aparece un mensaje que confirma que se ha creado el

equipo

correctamente. (Ilustraciones 20, 21)

[Si el usuario pulsa sobre “Eliminar equipo”]

4.- Se muestra al usuario una pantalla en donde el sistema se asegura de que el usuario realmente quiere eliminar el equipo  
(Ilustración 13)

[Si el usuario responde “Aceptar”]

4.- a. Se borra el equipo de su lista de equipos, y se eliminan todos los Pokémons que formaban parte del mismo.

[Si el usuario responde “Cancelar”]

4.-b. Se cancela la operación y se regresa a la lista de equipos.

[Si el usuario pulsa sobre “Captura un Pokémon”]

5.- Se abrirá la lista de especies Pokémon, desde la que se pueden ver todos los detalles de cada especie Pokémon. (Ilustración 14)

[Si el usuario pulsa sobre “Ver detalles”]

5.a- Aparecerá una pantalla con las estadísticas y tipos del Pokémon. Además, aparecerá debajo un botón “Capturar”, para poder añadir el Pokémon a cualquiera de los equipos del usuario. (Ilustración 15)

[Si el usuario pulsa sobre “Capturar”]

5.a.a- Se abrirá una pantalla en la que se permite añadir el Pokémon a cualquier

ranura vacía de cualquier equipo que tenga el usuario. (Ilustración 16)

[Si el usuario pulsa sobre cualquier botón de “Añadir”]

5.a.a.a- Aparecerá una pantalla en la que se permite al usuario introducir el

nombre del Pokémon que quiere añadir al equipo seleccionado en la ranura seleccionada. (Ilustraciones 17,22)

[Si el usuario pulsa “Aceptar” tras introducir el nombre]

5.a.a.a.a- Se volverá a la lista de equipos, en la que ahora aparecerá el nuevo Pokémon en la ranura y equipo deseados, con su nombre. Además, arriba se muestra un mensaje que confirma si se ha podido realizar la operación o no.  
(Ilustraciones 18,23)

[Si el usuario pulsa sobre “Expulsar” en el botón rojo que sale en las ranuras en las que hay un Pokémon]

6.- Aparece un mensaje de confirmación para asegurarse de que se quiere expulsar el Pokémon del equipo.

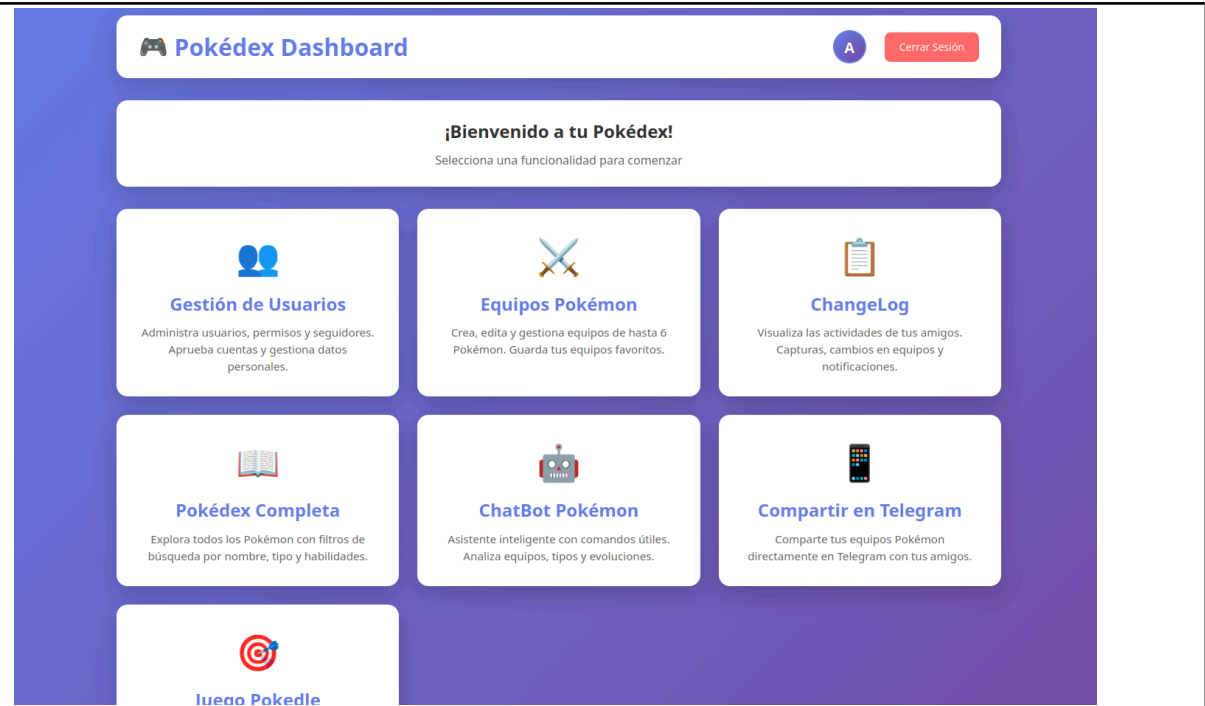
[Si el usuario pulsa sobre “Aceptar”]

6.a- Se expulsa al Pokémon del equipo, y se elimina al Pokémon del sistema.

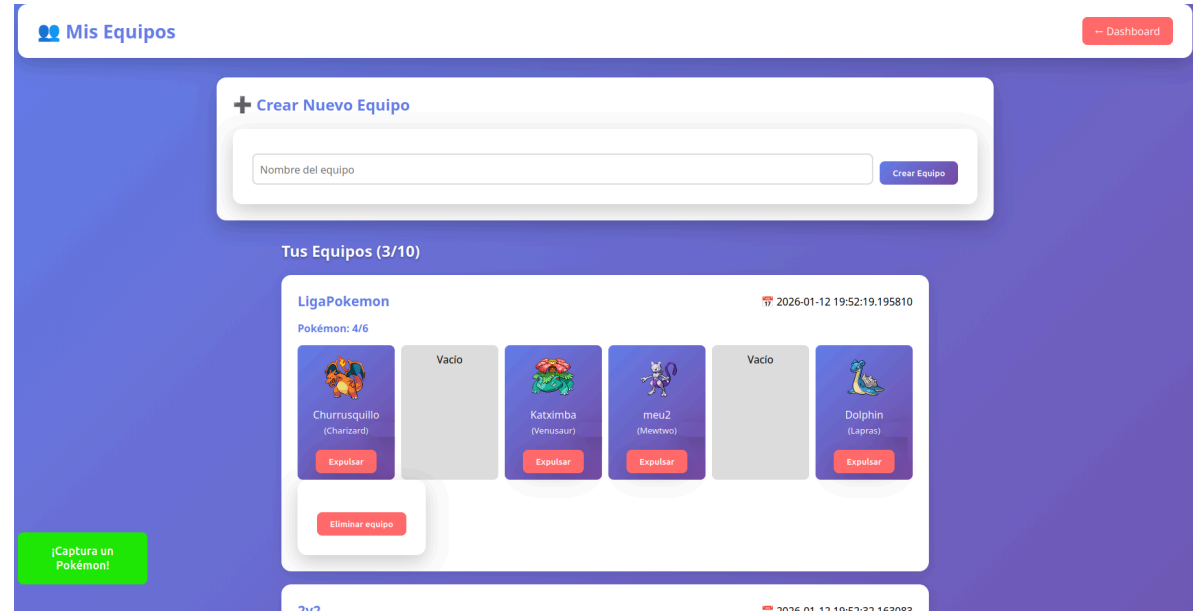
8.- a. En todo momento, excepto en cuadros de texto de confirmación, como los que aparecen al eliminar o guardar un equipo, el usuario puede pulsar la flecha de la esquina superior izquierda, lo cual le permite volver a la pantalla anterior.

**Postcondiciones:** Ninguna

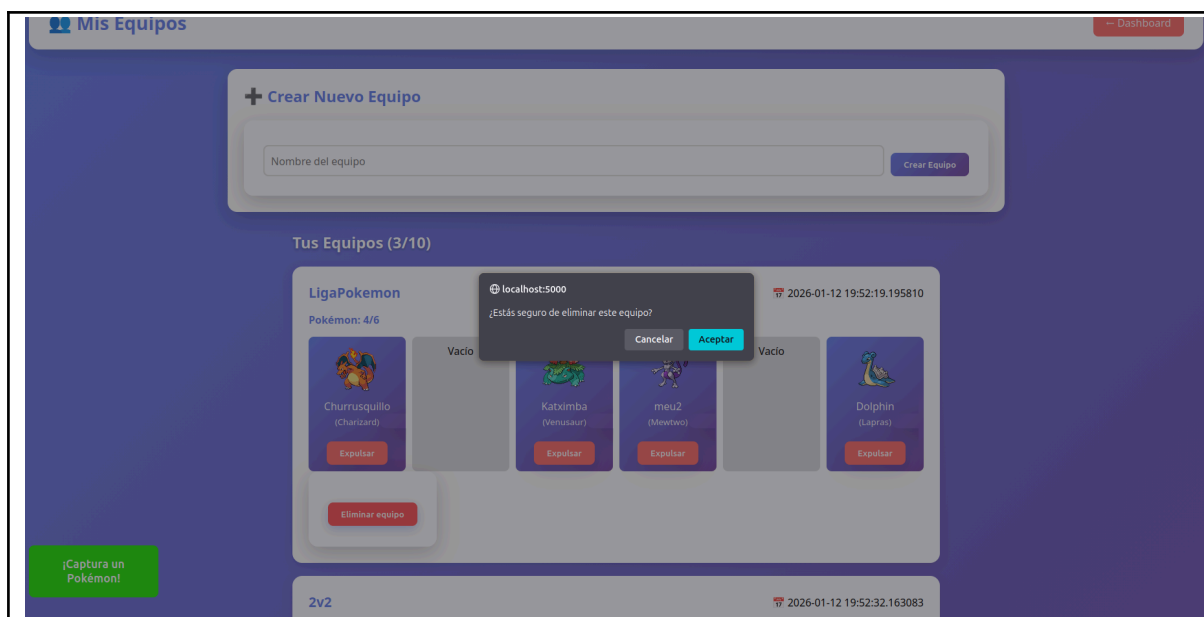
**Interfaz Gráfica:**



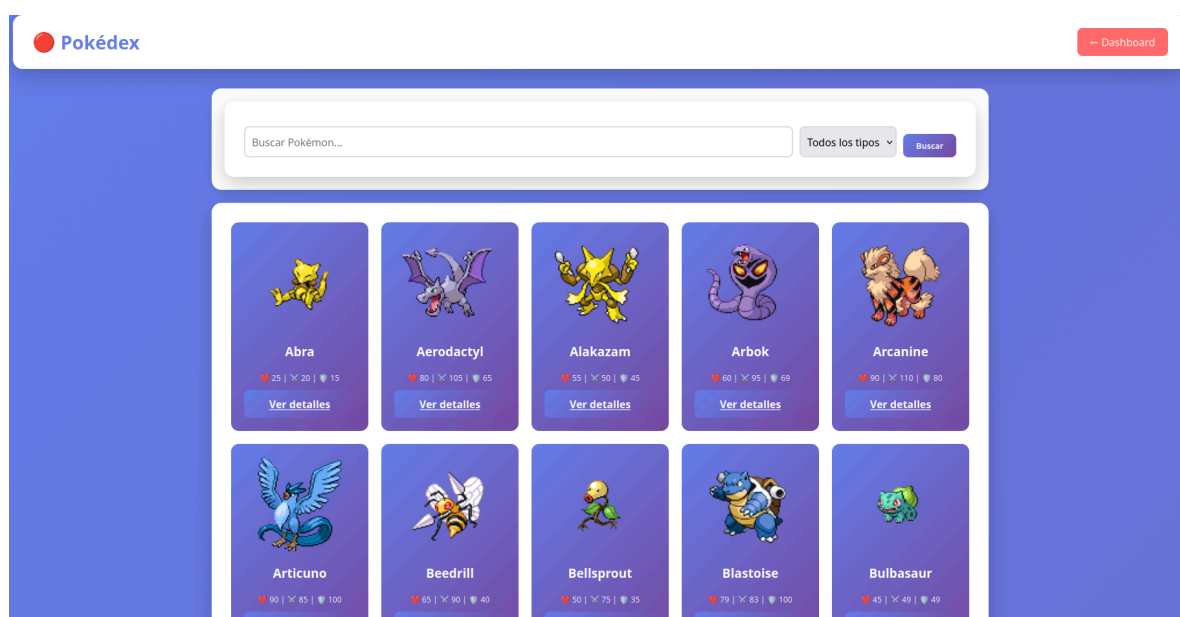
*Ilustración 11 – Menú principal*



*Ilustración 12 – Lista de equipos*



*Ilustración 13 – Eliminar equipo*



*Ilustración 14 – Lista de especies Pokémon (tras pulsar “Captura un Pokémon”)*



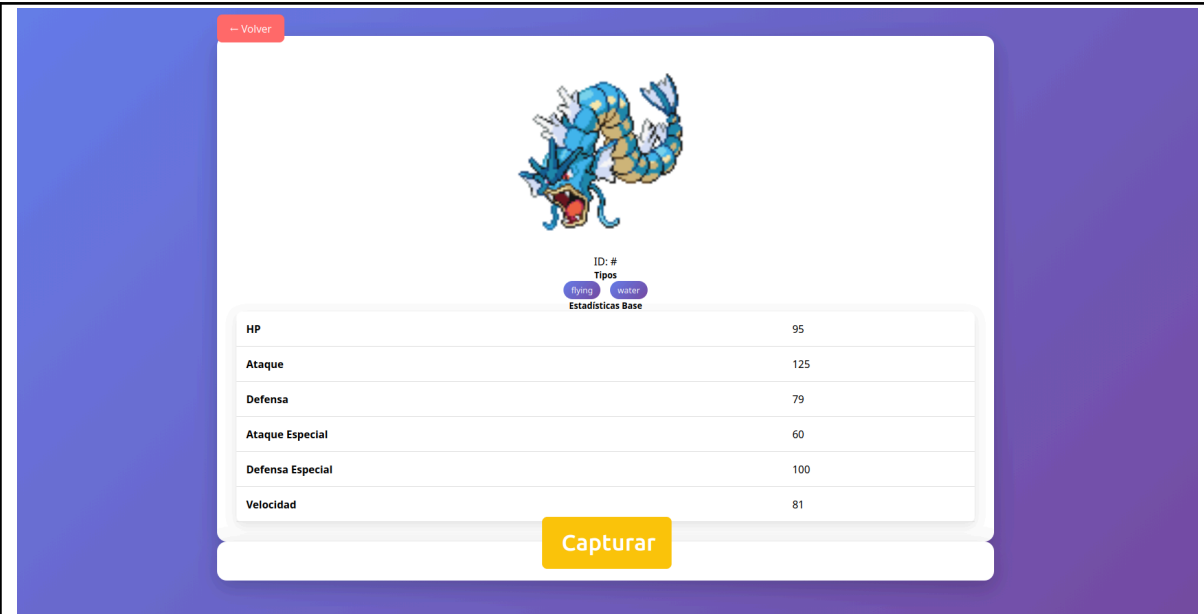


Ilustración 15 – Ver detalles de un Pokémon, opción “Capturar”

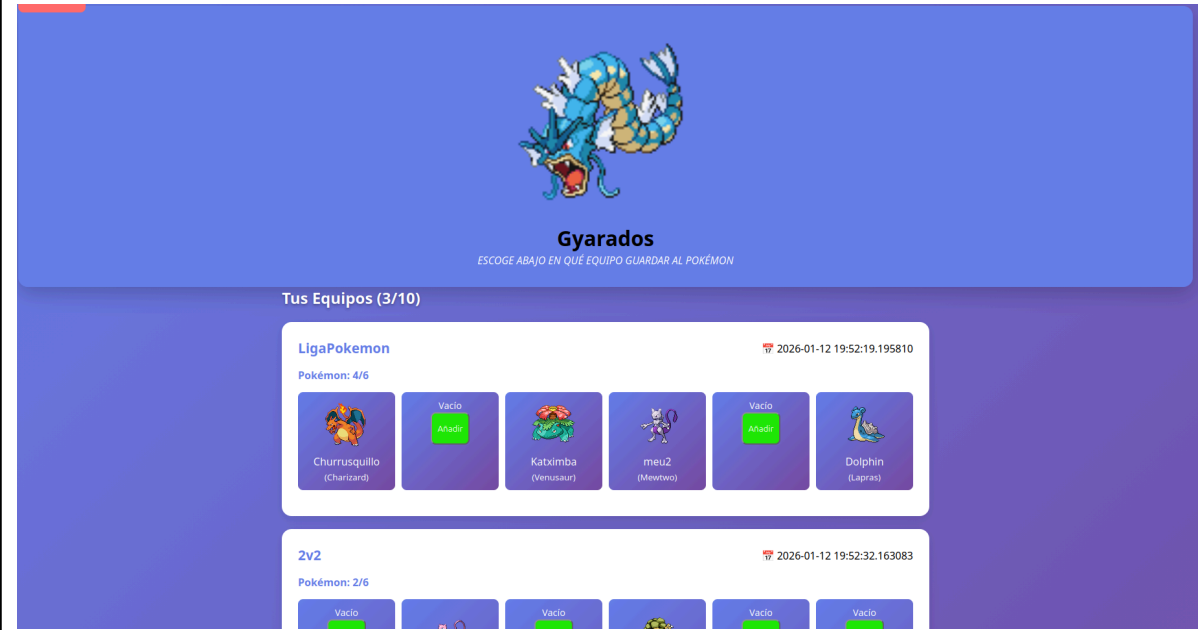
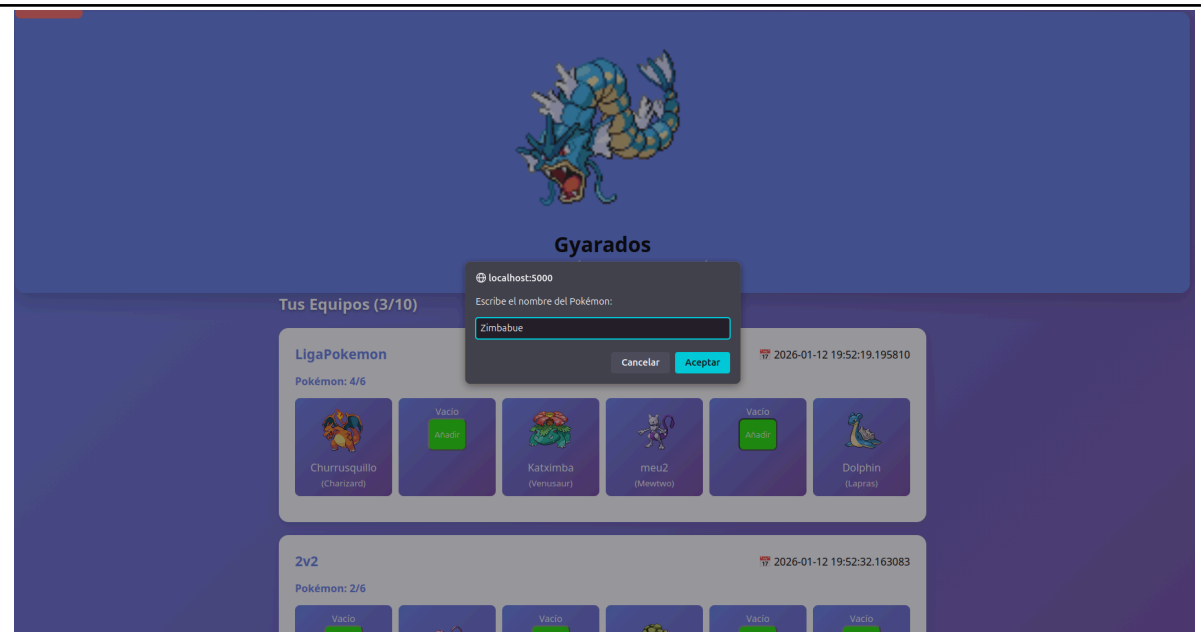
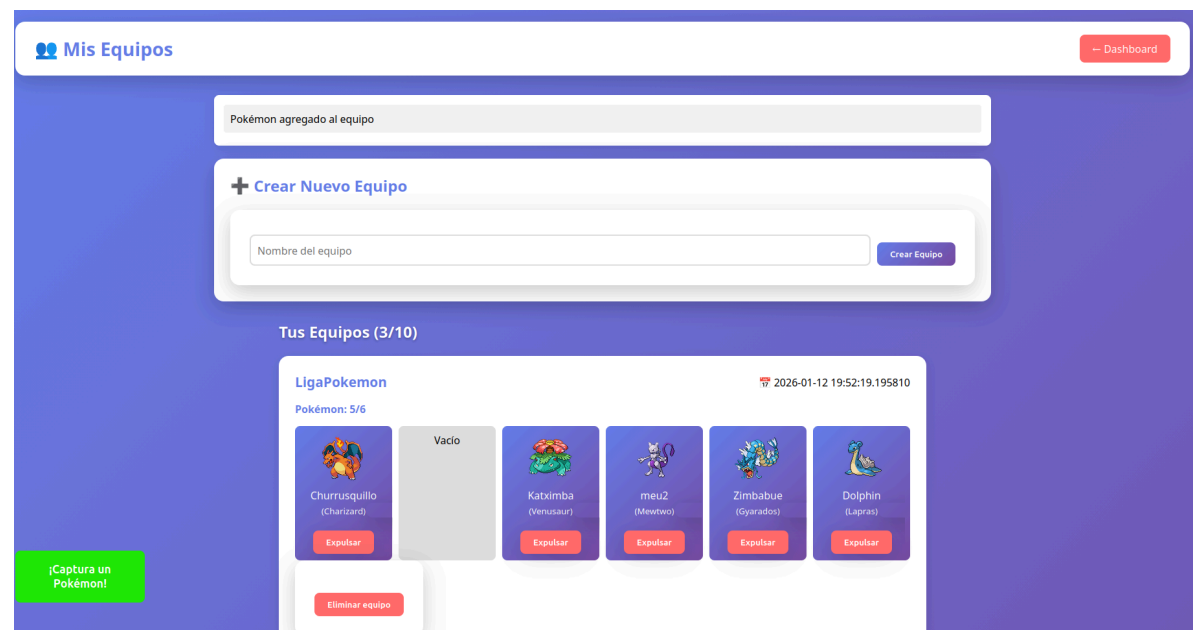


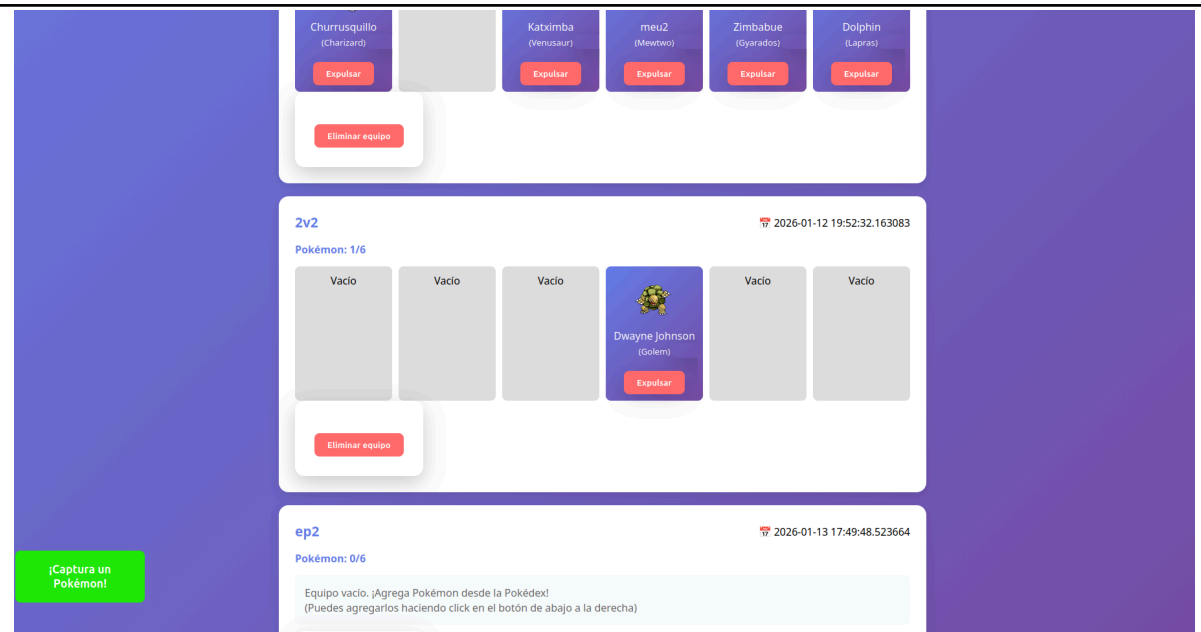
Ilustración 16 – Botón añadir



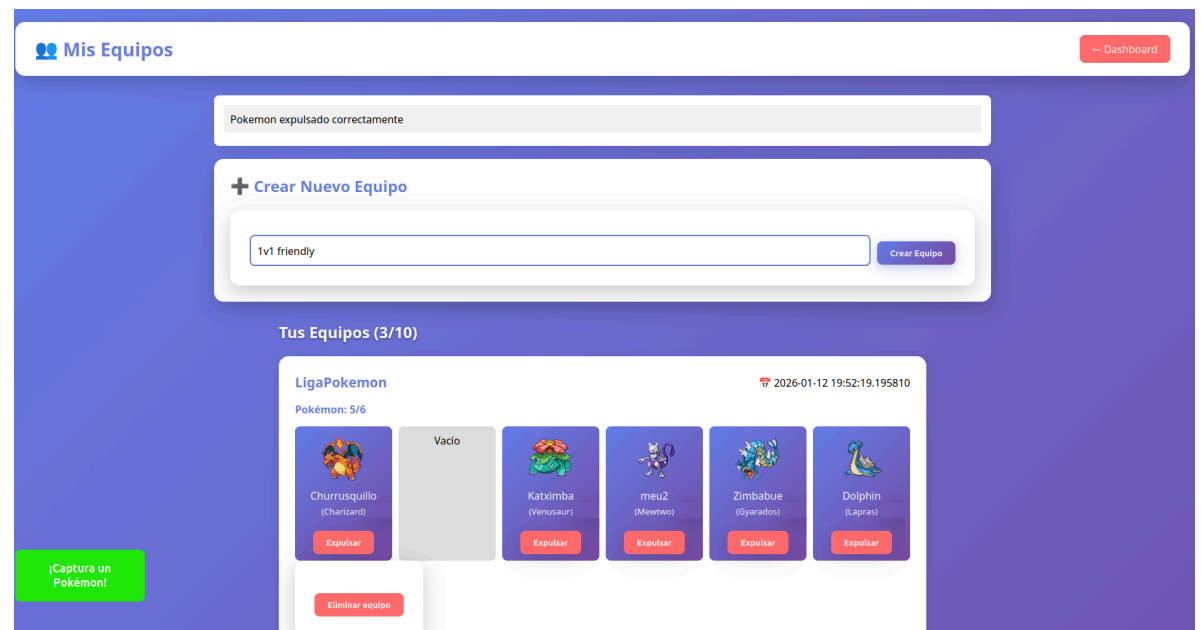
*Ilustración 17 – Elección de nombre de Pokémon tras pulsar “Añadir”*



*Ilustración 18 – Pokémon añadido al equipo*



*Ilustración 19 – Visualización del resto de equipos (Scroll down)*



*Ilustración 20 – Crear nuevo equipo introduciendo su nombre (botón)*

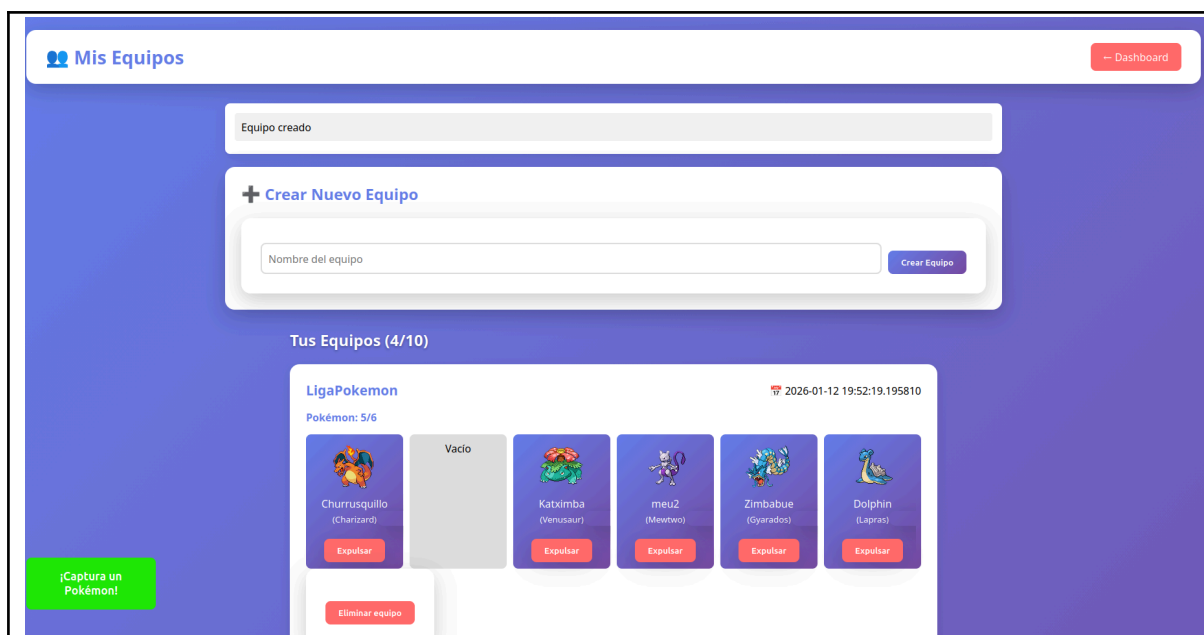


Ilustración 21 – Mensaje de correcta creación de nuevo equipo

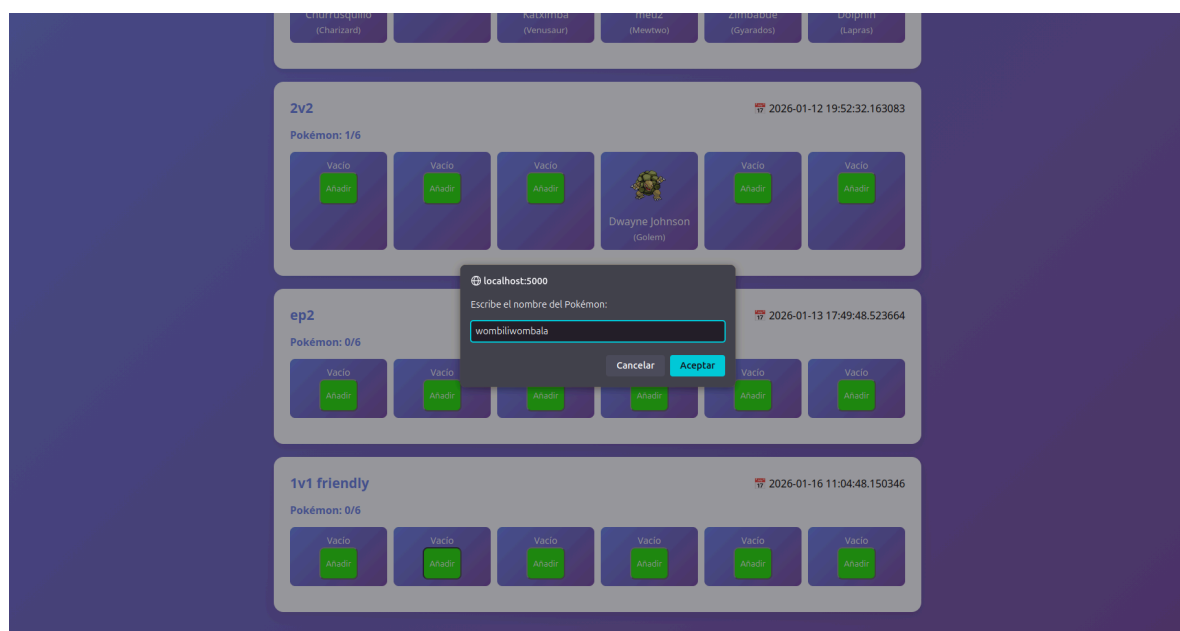
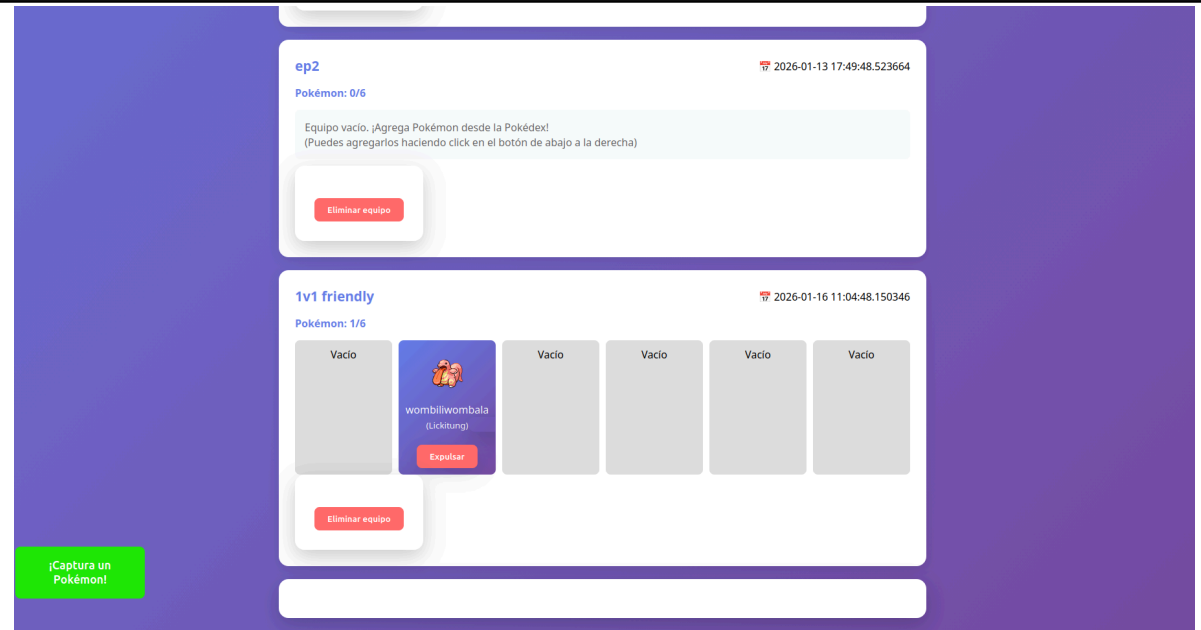
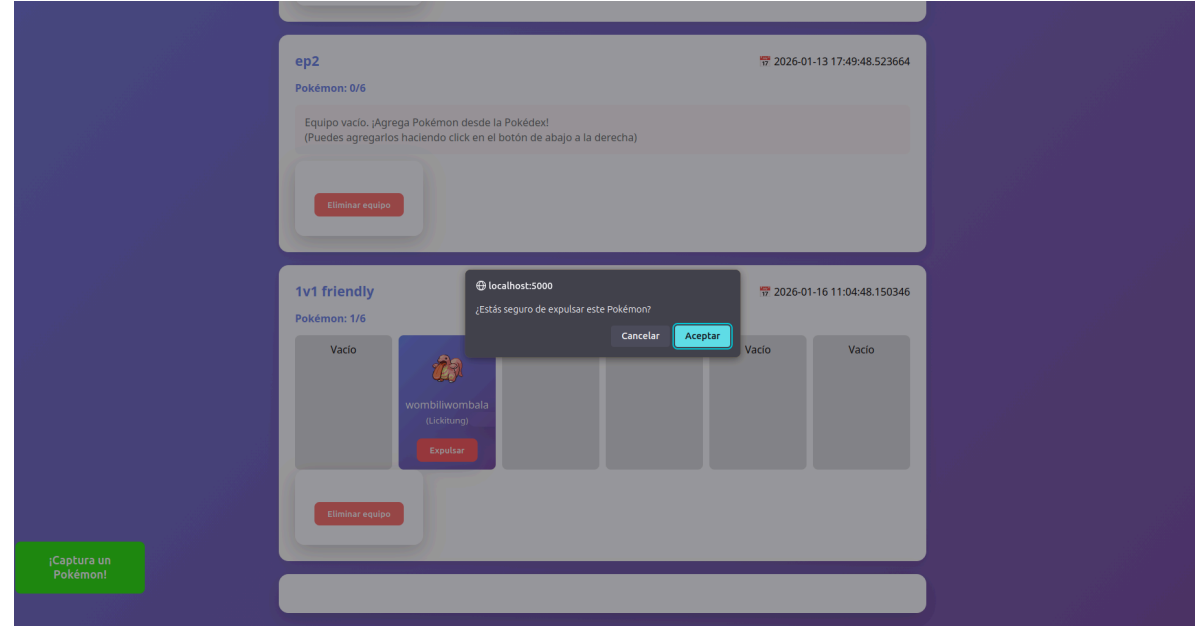


Ilustración 22 – Pokémon añadido en ranura vacía en equipo nuevo



*Ilustración 23 – Pokémon añadido a equipo vacío*



*Ilustración 24 – Botón expulsar*



*Ilustración 26 – Botón expulsar (resultado)*

FUNCIONALIDAD 3: ChangeLog - Aitana Niño

The diagram shows a stick figure actor labeled 'Usuario' connected to a use case circle labeled 'Ver ChangeLog'. Inside the circle is a horizontal line with 'Si quiere' below it. Another use case circle labeled 'Aplicar filtro' is connected to the 'Ver ChangeLog' circle with an arrow labeled 'Extend'.

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nombre:                    | Ver ChangeLog                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Descripción:               | Permite al usuario consultar los cambios realizados en el equipo Pokémon o las capturas de otros usuarios a los que sigue                                                                                                                                                                                                                                                                                                                                                                              |
| Actores:                   | Usuario                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Precondiciones:            | Estar identificado                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Requisitos no funcionales: | Se almacenarán las últimas 20 notificaciones                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Flujo de Eventos:          | <div>1. El usuario accede a la opción seleccionándola en el menú principal. (Ilustración 7)</div> <div>2. El sistema consulta la base de datos asociada a los eventos de los usuarios que sigue.</div> <div>3. El sistema muestra la lista de eventos en orden cronológico. (Ilustración 27)</div> <div>4. Si el usuario lo desea, podrá filtrar las notificaciones en función del usuario o por tipo de notificación (usuario seguido, nueva captura o nuevo equipo). (Ilustración 28, 29 y 30)</div> |
| Postcondiciones:           | Se obtienen las notificaciones, todas o filtradas.                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| Interfaz Gráfica:          | <div><p>The screenshot shows a mobile app interface titled 'Actividad de Amigos'. At the top is a dropdown menu with 'Todas las actividades'. Below are five event cards for '@user2'. The first three are 'captura' (green) for 'Arcanine', 'Alakazam', and 'Voltorb'. The fourth is 'equipo' (red) for 'BalacalaoAlPilpil'. The fifth is 'seguido' (yellow) for 'ha empezado a seguir a user1'. Each card has a timestamp on the right.</p></div>                                                    |

Ilustración 27 – Lista de eventos

## Actividad de Amigos

Todas las actividades

Todas las actividades

Capturas

Nuevos Equipos

Nuevos Seguidores

Buscar usuario específico...

Ilustración 28 - Filtros de eventos

## Actividad de Amigos

Buscar usuario específico...

Escribe el nombre aquí...

Ilustración 29 - Buscar por usuario

### Actividad de Amigos

Capturas

@user2

captura

user2 ha capturado un Arcanine

2026-01-15 17:47:47

@user2

captura

user2 ha capturado un Alakazam

2026-01-15 17:47:36

@user2

captura

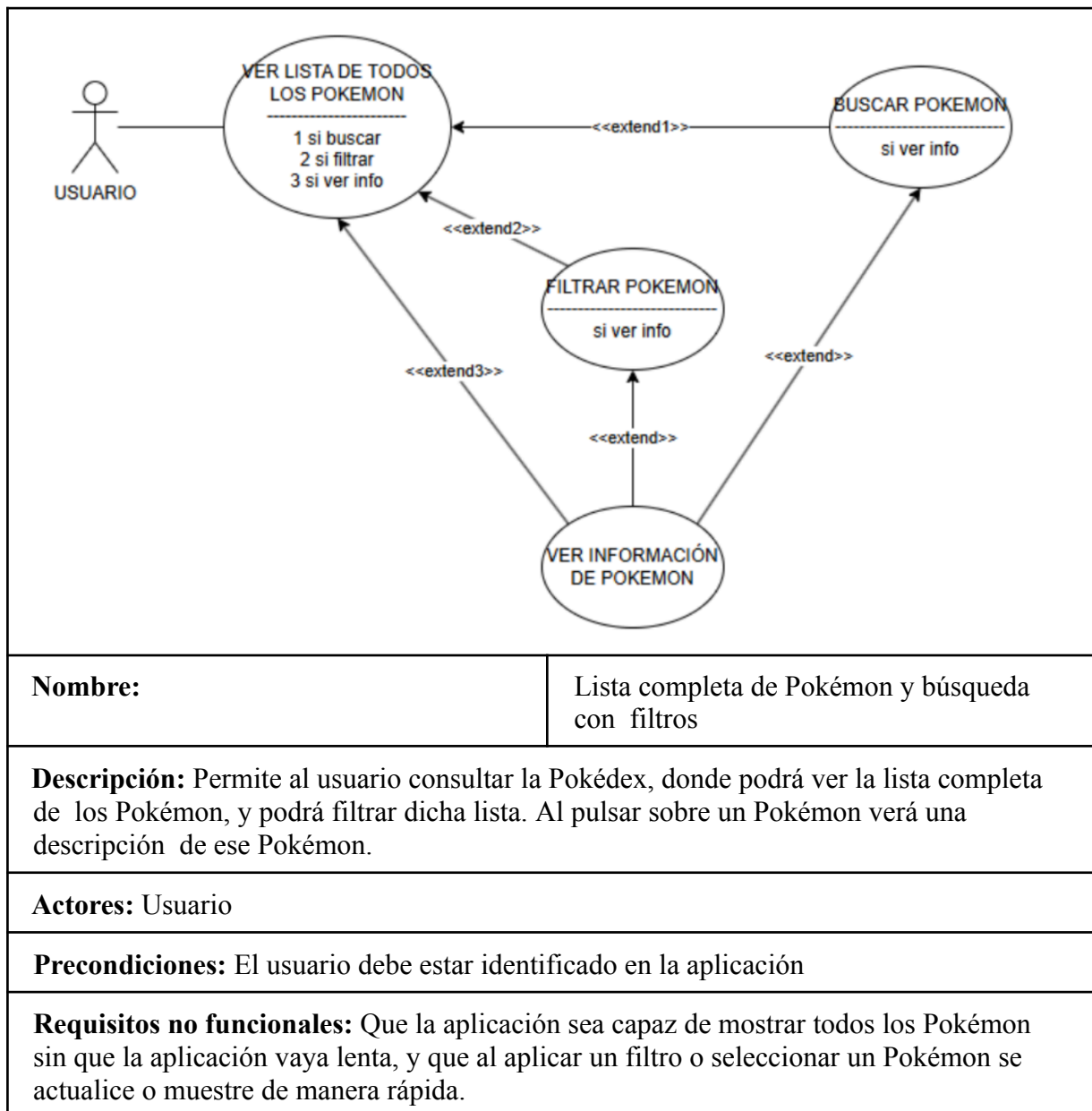
user2 ha capturado un Voltorb

2026-01-15 17:47:23

Ilustración 30 - Notificaciones con filtros



## FUNCIONALIDAD 4: Lista Completa de Pokémon y Búsquedas con Filtros – Ibai Munné



**Flujo de Eventos:**

1. El usuario selecciona en su menú principal la opción de ver la lista completa de los Pokémon, donde podrá ver la lista completa de los Pokémon (*Ver ilustración 1*). a. El usuario podrá filtrar los Pokémon que aparecen para poder realizar su búsqueda de una manera más rápida (a través de un botón), al pulsar el botón se abrirá un desplegable (o ventana aparte) con varias opciones de filtrado, tipo de Pokémon, si es legendario o no... (*Ver ilustración 2*).

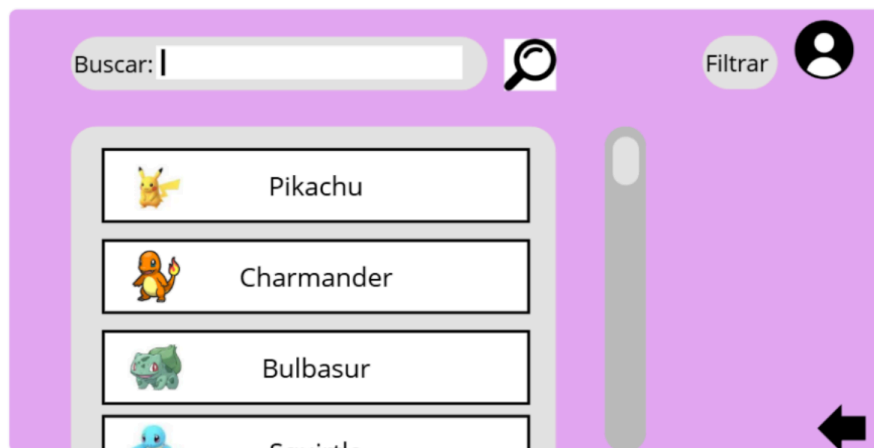
Al aplicar el filtro de búsqueda la lista de Pokémon se actualizará y mostrará solo los Pokémon con ese filtro. Si no existe ningún Pokémon con dichos filtros, la lista aparecerá vacía.

- b. El usuario además de filtrar podrá hacer una búsqueda de un Pokémon en concreto al escribir su nombre en el apartado de buscar.
- c. Cuando el usuario seleccione un Pokémon (ya sea habiendo utilizado un filtro/búsqueda o no), podrá ver la información completa de dicho Pokémon, donde se mostrará en una nueva ventana: imagen del Pokémon, peso, altura, tipos... (*Ver ilustración 3*).

**Postcondiciones:** Ninguna

## Interfaz Gráfica:

*Lista completa de Pokemon*



*Ilustración 31 – Lista completa de Pokémon*

*Filtrado de pokemons*



*Ilustración 32 – Filtrado de Pokémon*



Ilustración 33 – Ver información de Pokémon

## FUNCIONALIDAD 5: ChatBot

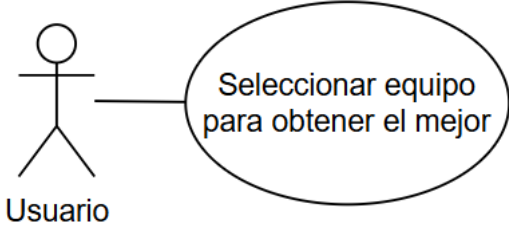
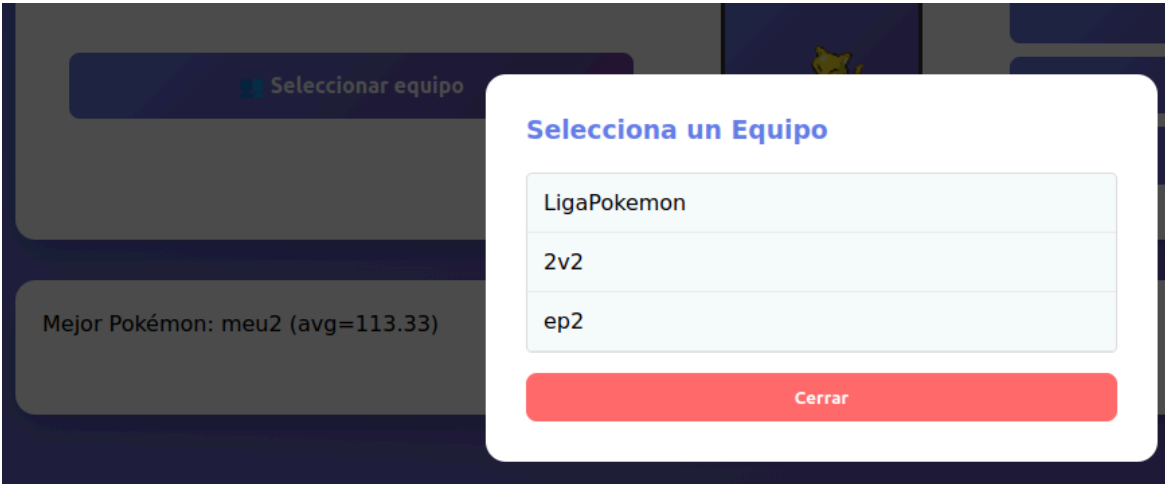
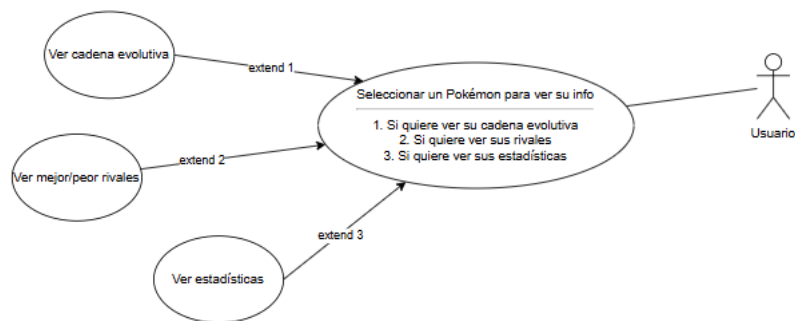
|                                                                                                                                                                                  |                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <pre>graph LR     Usuario((Usuario)) --- UC((Seleccionar equipo para obtener el mejor))</pre> |                                                                                                                                                                                                                                                                     |
| <b>Nombre:</b>                                                                                                                                                                   | Seleccionar equipo para obtener mejor                                                                                                                                                                                                                               |
| <b>Descripción:</b>                                                                                                                                                              | Permite al usuario consultar el mejor Pokémon de un equipo suyo.                                                                                                                                                                                                    |
| <b>Actores:</b>                                                                                                                                                                  | Usuario                                                                                                                                                                                                                                                             |
| <b>Precondiciones:</b>                                                                                                                                                           | <ul style="list-style-type: none"><li>- Si el usuario no tiene equipos el botón estará apagado, es decir; no podrá usar el caso de uso.</li></ul>                                                                                                                   |
| <b>Requisitos no funcionales:</b>                                                                                                                                                |                                                                                                                                                                                                                                                                     |
| <b>Flujo de Eventos:</b>                                                                                                                                                         | <ol style="list-style-type: none"><li>1. El usuario selecciona en el menú del ChatBot el botón “seleccionar equipo”.</li><li>2. A continuación, podrá ver la lista completa de sus equipos</li><li>3. Tras seleccionar uno, el sistema devolverá el mejor</li></ol> |
| <b>Postcondiciones:</b>                                                                                                                                                          | <ul style="list-style-type: none"><li>- Devuelve el Pokémon con mayor nivel, en caso de haber empate, devuelve uno.</li></ul>                                                                                                                                       |
| <b>Interfaz Gráfica:</b>                                                                                                                                                         |                                                                                                                                                                                 |

Ilustración 34 – Seleccionar equipo ChatBot



**Nombre:** Seleccionar un Pokémon para ver su info

**Descripción:** El usuario elige un Pokémon escribiendo su nombre

**Actores:** usuario

**Precondiciones:**

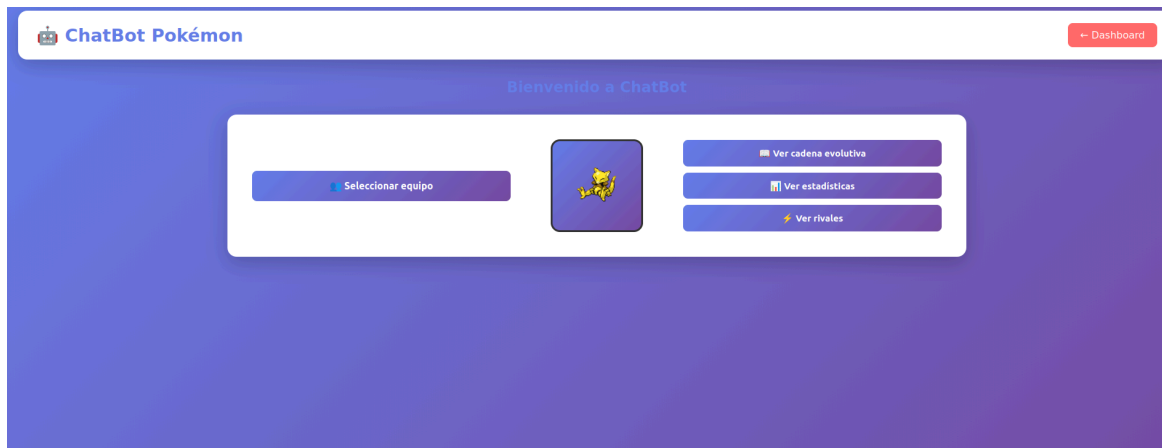
**Requisitos no funcionales:**

**Flujo de Eventos:**

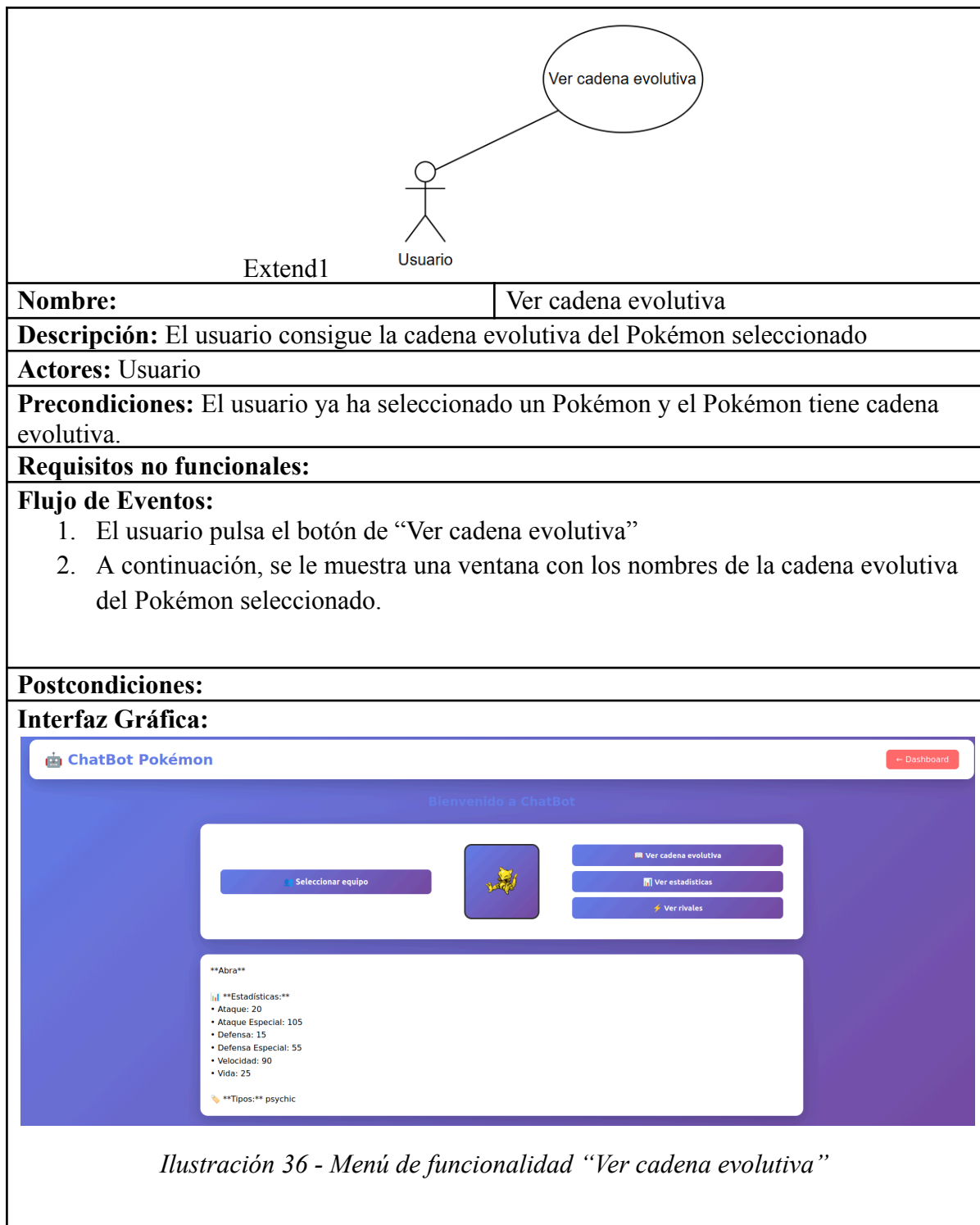
1. El usuario pulsa el botón de “selecciona Pokémon”
2. A continuación, se le muestra un desplegable para que escriba el Pokémon

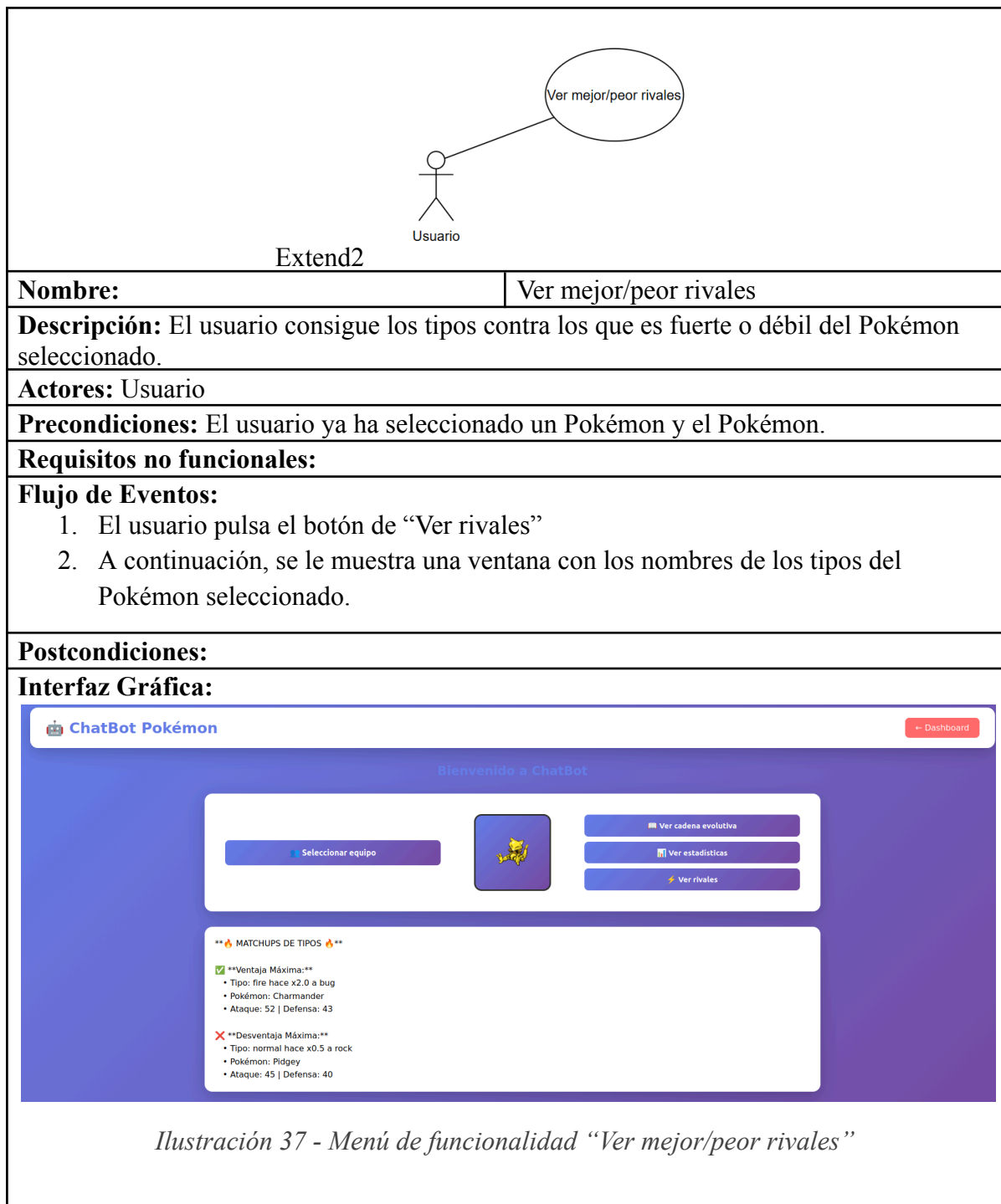
**Postcondiciones:** Si el Pokémon existe aparecerá una foto de este y se habilitarán correspondientes botones de abajo (pasarán de apagados a encendidos).

**Interfaz Gráfica:**

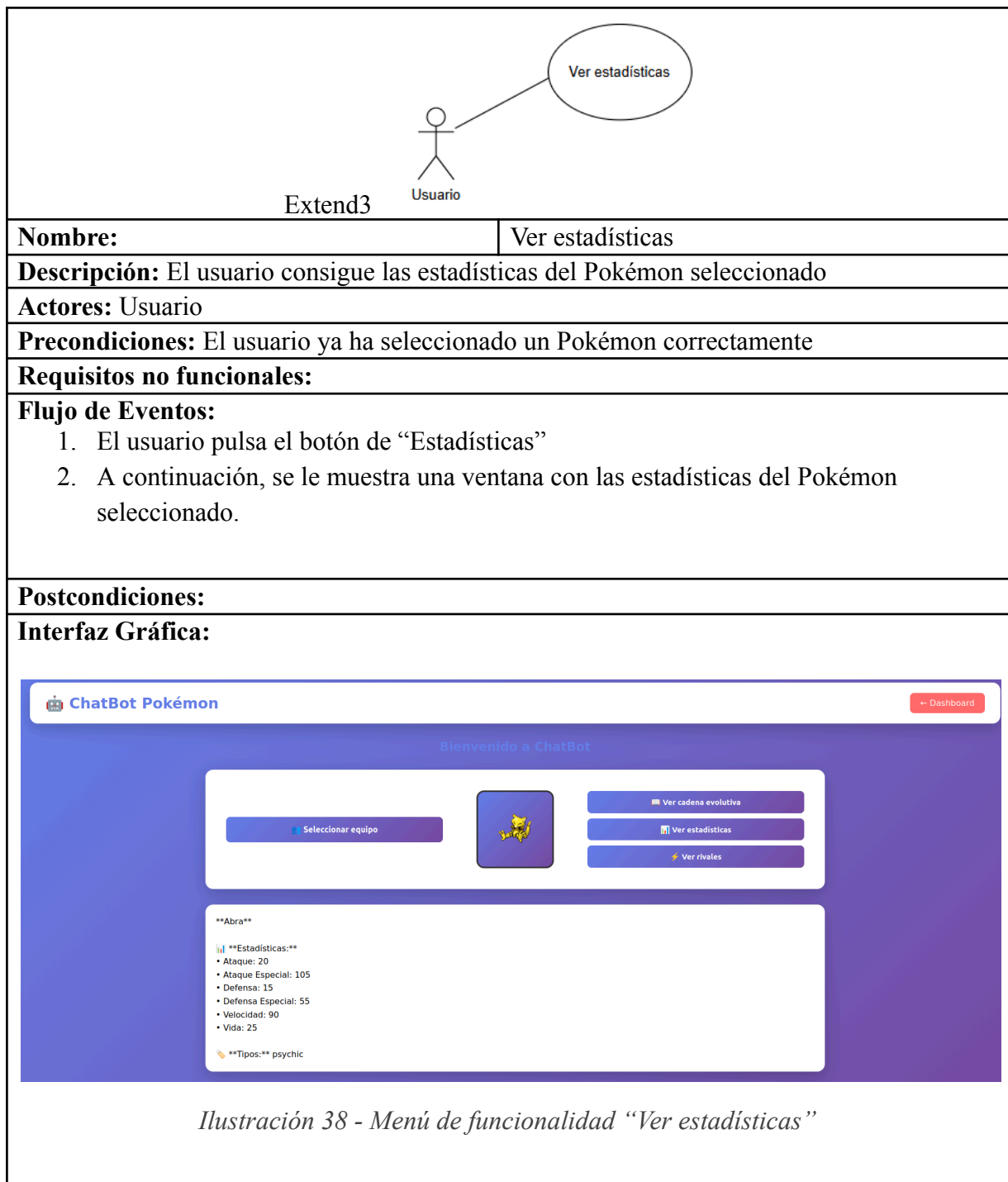


*Ilustración 35 - Menú de ChatBot una vez elegido el Pokémon*

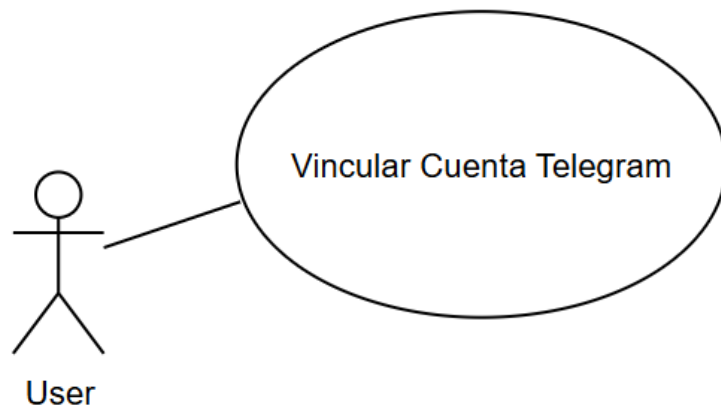








## FUNCIONALIDAD 6: Bot de Telegram



**Nombre:**

Vincular cuenta de Telegram

**Descripción:** Permite al usuario asociar su usuario de telegram con su cuenta de la aplicación.

**Actores:** Usuario

**Precondiciones:** El usuario debe estar identificado en la aplicación y tener cuenta de Telegram

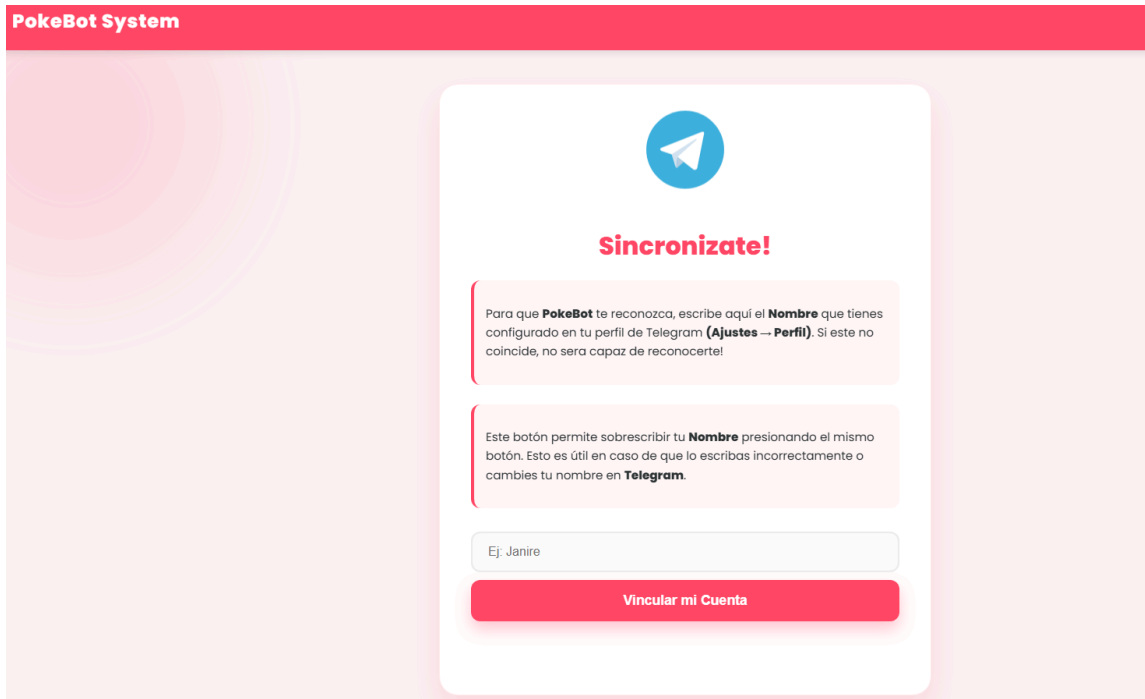
**Requisitos no funcionales:** -

**Flujo de Eventos:**

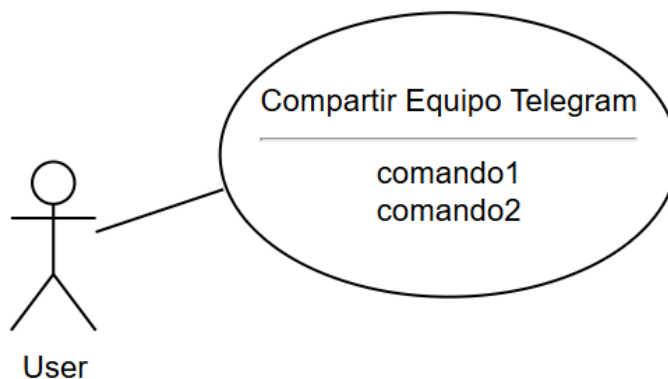
1. El usuario escribe un nombre que corresponder con su username de Telegram
- 2 El usuario pulsa “Vincular mi Cuenta”

**Postcondiciones:** El bot reconoce al usuario

**Interfaz:**



*Ilustración 39 - Funcionalidad Telegram*



**Nombre:**

Compartir mediante Telegram

**Descripción:** Telegram (accionado por el usuario dentro de un contexto de Telegram) consulta los equipos de un usuario y los comparte

**Actores:** Usuario Telegram

**Precondiciones:** El usuario debe estar identificado en la aplicación, estar en un grupo de Telegram con el Bot y el Bot debe estar encendido.

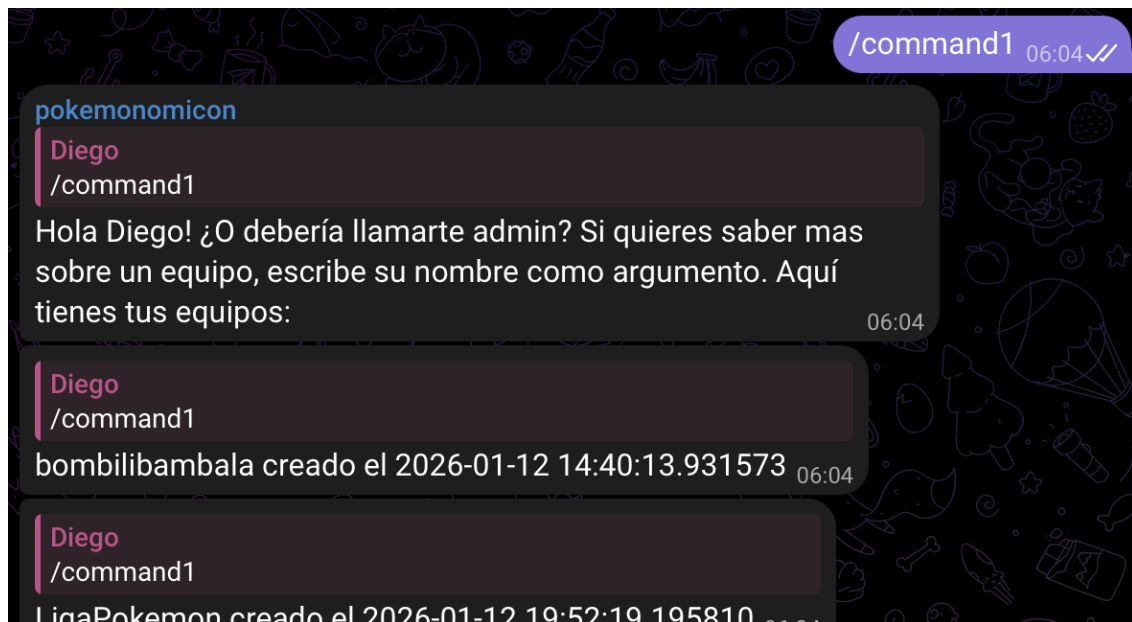
**Requisitos no funcionales:** -

**Flujo de Eventos:**

1. El usuario escribe un comando que es leído por el Bot de Telegram
- 2a. Es el comando 2. Se ponen a jugar a piedra papel tijera versión Pokemon.
- 2b. Es el comando 1. El Bot consulta y muestra los equipos del Usuario.
- 2c. Es el comando 1 con el nombre del equipo. El Bot consulta y muestra el equipo especificado del Usuario.

**Postcondiciones:** El bot reconoce al usuario

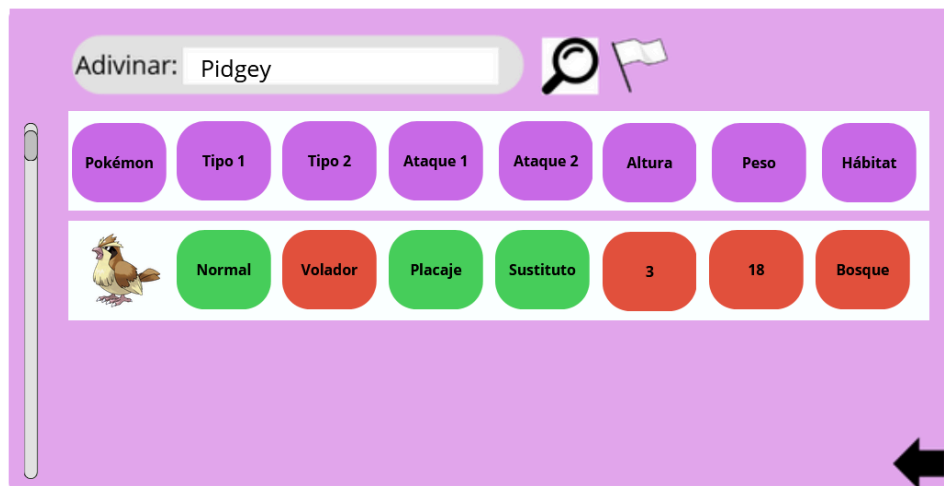
**Interfaz:**



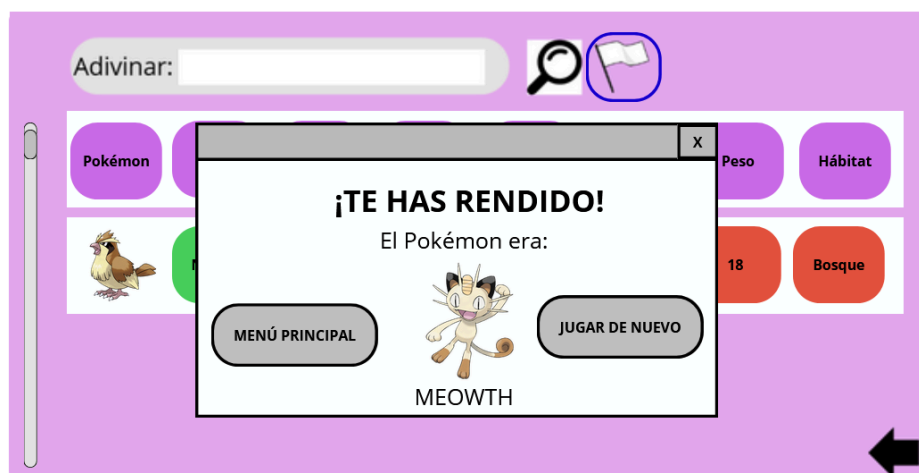
*Ilustración 40 - Interfaz en Telegram*

## FUNCIONALIDAD 7: Pokedle -

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <pre> graph LR     Usuario((Usuario)) --- UC1((Jugar a Pokedle<br/>1. Si quiere rendirse<br/>2. Si quiere adivinar Pokémon))     UC1 -- extend2 --&gt; UC2((Escribir nombre Pokémon))     UC1 -- extend1 --&gt; UC3((Rendirse y mostrar Pokémon a adivinar))         </pre> <p>The diagram shows a stick figure actor labeled 'Usuario' connected to a central use case 'Jugar a Pokedle'. This central use case contains two sub-steps: '1. Si quiere rendirse' and '2. Si quiere adivinar Pokémon'. Two other use cases, 'Escribir nombre Pokémon' and 'Rendirse y mostrar Pokémon a adivinar', are connected to the central use case via arrows labeled 'extend2' and 'extend1' respectively.</p>                            |                 |
| <b>Nombre:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Jugar a Pokedle |
| <b>Descripción:</b> El usuario puede jugar a adivinar un Pokémon que se elige aleatoriamente. Con cada intento aparecen pistas que ayudan a adivinarlo.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                 |
| <b>Actores:</b> Usuario                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                 |
| <b>Precondiciones:</b> Estar identificado en la plataforma correctamente como usuario.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                 |
| <b>Requisitos no funcionales:</b> -                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                 |
| <b>Flujo de Eventos:</b> <ol style="list-style-type: none"> <li>1. En el menú principal el usuario pincha en el botón de “Jugar Pokedle”. El programa escoge un Pokémon al azar.</li> <li>2. El usuario introduce el nombre de un Pokémon en el campo de texto y pulsa enter en su teclado. Se muestra el Pokémon adivinado, y si no es el correcto, se marcan las casillas que coinciden con el Pokémon adivinado en verde, y en rojo las casillas que no coincidan, a modo de pista. (Ilustración 41)</li> <li>3. El usuario puede continuar introduciendo nombres o rendirse (Ilustración 42,43).</li> <li>4. Al finalizar, el usuario puede pulsar el botón de menú principal o jugar de nuevo (Ilustración 44).</li> </ol> |                 |
| <b>Postcondiciones:</b> el sistema muestra la pantalla de resultados (victoria o rendición) o la pantalla de juego con pistas actualizadas.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                 |
| <b>Interfaz Gráfica:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                 |



*Ilustración 41 - Pistas Pokedle*



*Ilustración 42 - Opción de rendirse*



*Ilustración 43 - Pokémon adivinado*



*Ilustración 44 - Ir a menú principal o jugar de nuevo*

### 3. MODELO DE DOMINIO

#### Diagrama completo

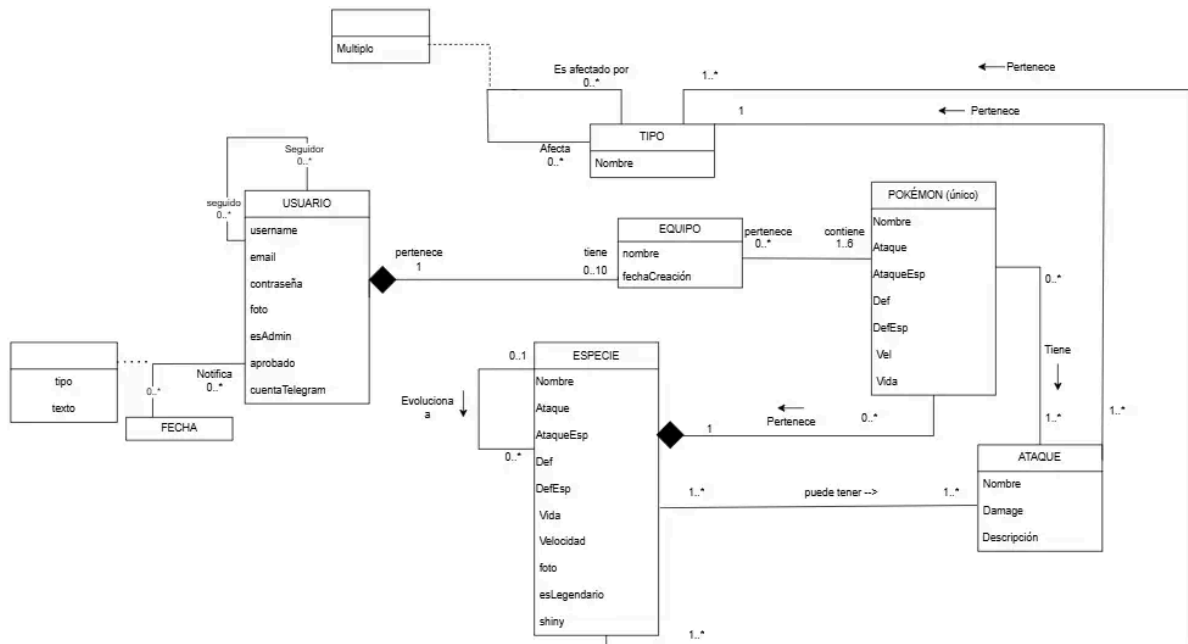


Ilustración 45 - Diagrama de modelo de dominio

#### Entidades

| Usuario     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción | Es la representación de la persona que está usando nuestra aplicación.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Porqué      | Es el actor “main”, necesario para seguir su rumbo en nuestra aplicación.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Atributos   | <ul style="list-style-type: none"> <li>● username: nombre del usuario.</li> <li>● email: correo electrónico del usuario.</li> <li>● esAdmin: booleano que indica si el usuario es administrador o no (con más privilegios respecto al usuario normal).</li> <li>● aprobado: booleano que indica si la cuenta ha sido aprobada por un administrador.</li> <li>● contraseña: conjunto de caracteres y números que sirven de clave de acceso privada del usuario a la aplicación.</li> <li>● foto: Una imagen que el usuario puede establecer si así lo desea, para distinguirse algo más del resto.</li> </ul> |



| Equipo      |                                                                                                                                                              |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción | Son listas de Pokémon que tienen guardados los jugadores, para, en caso de realizar un combate, tener una lista de 6 pokémon con los que participar.         |
| Porqué      | Cada jugador puede tener guardados entre 0 y 10 equipos, por lo que debemos de tener esto para guardar unos pokémon específicos a modo de lista.             |
| Atributos   | <ul style="list-style-type: none"> <li>• Nombre: nombre que el usuario ha puesto a dicho equipo</li> <li>• FechaCreación: fecha en la que se creó</li> </ul> |

| Especie     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción | <p>Son instancias de Pokémon con las estadísticas base fijas.<br/> <i>ej: Los pikachu tienen 100 de daño de base.</i><br/>         Esto quiere decir que todos los pikachu tendrán esa estadística predeterminada/predefinida. (nos la da la API)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Porqué      | Tenemos que tener una lista (o base de datos) con todos los pokemon para no tener que estar consultando a la API constantemente. Así, al generar una instancia específica de un Pokémon, podemos consultar la información de la especie, y generar un espécimen único a partir de ella.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Atributos   | <ul style="list-style-type: none"> <li>• Nombre: nombre de la especie del Pokémon. Por ejemplo, Pikachu, Charmander, Squirtle, Bulbasaur ...</li> <li>• Ataque: Estadística de daño físico que tiene la especie Pokémon como base. Es decir, el daño que hará con ataques cuerpo a cuerpo normales, como “placaje” o “mordisco”.</li> <li>• AtaqueEsp: Estadística de daño especial que tiene la especie Pokémon como base. Es decir, el daño que hará con ataques especiales, como “puño trueno” o “colmillo ígneo”.</li> <li>• Def: Estadística de defensa que tiene la especie Pokémon como base. Es decir, la reducción de daño que tendrá contra ataques cuerpo a cuerpo normales, como “placaje” o “mordisco”.</li> <li>• DefEsp: Estadística de defensa especial que tiene la especie Pokémon como base. Es decir, la reducción de daño que tendrá contra ataques especiales, como “puño trueno” o “colmillo ígneo”.</li> </ul> |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <ul style="list-style-type: none"> <li>● Velocidad: Estadística base que determina ciertos factores durante el combate, como qué Pokémon empieza atacando antes (generalmente es el que más velocidad tiene).</li> <li>● Vida: Estadística que determina la cantidad de vida que tiene el Pokémon de base.</li> <li>● Foto: Imágen que sirve para identificar visualmente al Pokémon (sprite).</li> <li>● esLegendario: booleano que identifica si el pokémon es legendario o no.</li> <li>● shiny: booleano que identifica si el pokémon es shiny (variocolor) o no.</li> </ul> |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| Pokémon (único) |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción     | <p>Es un Pokémon con estadísticas y habilidades propias, que el jugador ha atrapado al crear un equipo. Cada jugador atrapa un Pokémon diferente, el cual tendrá sus propias habilidades y estadísticas acorde a su especie (existe la probabilidad de que dos Pokemons únicos tengan las mismas estadísticas y/o habilidades).</p> <p><i>ej: Los pikachu tienen 100 de daño base (ESPECIE), <b>pero</b> nuestro pikachu tendrá por ejemplo 104 de daño.</i></p>                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Porqué          | <p>Necesitamos esto ya que tenemos que poder tener Pokémons con estadísticas y habilidades concretas, y distintas al resto de los Pokémon. Cada espécimen de Pokémon es distinto.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Atributos       | <ul style="list-style-type: none"> <li>● Nombre: Nombre único del espécimen de Pokémon concreto. Por ejemplo, mi pikachu se llama Juan.</li> <li>● Ataque: Estadística de daño físico que tiene el Pokémon concreto. Es decir, el daño que hará con ataques cuerpo a cuerpo normales, como “placaje” o “mordisco”.</li> <li>● AtaqueEsp: Estadística de daño especial que tiene el Pokémon concreto. Es decir, el daño que hará con ataques especiales, como “puño trueno” o “colmillo ígneo”.</li> <li>● Def: Estadística de defensa que tiene el Pokémon concreto. Es decir, la reducción de daño que tendrá contra ataques cuerpo a cuerpo normales, como “placaje” o “mordisco”.</li> <li>● DefEsp: Estadística de defensa especial que tiene el Pokémon concreto. Es decir, la reducción de daño que tendrá contra ataques especiales, como “puño trueno” o “colmillo ígneo”.</li> </ul> |

|  |                                                                                                                                                                                                                                                                                                                        |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <ul style="list-style-type: none"> <li>• Velocidad: Estadística base que determina ciertos factores durante el combate, como qué Pokémon empieza atacando antes (generalmente es el que más velocidad tiene).</li> <li>• Vida: Estadística que determina la cantidad de vida que tiene el Pokémon concreto.</li> </ul> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| Tipo        |                                                                                                                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción | Elemento el cual pertenece a un pokemon. Por ejemplo, tipo agua, tipo planta, tipo fuego, tipo acero, tipo fantasma...                                                                                                                 |
| Porqué      | Un tipo tiene sentido por sí solo. No comprende valores infinitos, sino que una serie de valores específicos, como los del ejemplo mencionado en la descripción. Se deben guardar las relaciones de debilidad y fortaleza entre tipos. |
| Atributos   | <ul style="list-style-type: none"> <li>• Nombre: el nombre del tipo. (planta, acero, fantasma...)</li> </ul>                                                                                                                           |

| Ataque      |                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Descripción | Una acción especial que pueden realizar algunos pokémon. Generalmente se utilizan en combate, pero pueden tener distintas aplicaciones en el día a día. Por ejemplo, en combate, un Pokémon puede utilizar el ataque 'Surf' para derrotar a su rival, o puede utilizarla para ayudar a su entrenador a navegar por el mar (Aunque en esta aplicación no existe el combate, ni tampoco un entorno en el que usar los ataques). |
| Porqué      | Se necesita tener en cuenta las posibles ataques que puede aprender una especie Pokémon, y los ataques concretos que tiene un espécimen Pokémon.                                                                                                                                                                                                                                                                              |
| Atributos   | <ul style="list-style-type: none"> <li>• Nombre: el nombre del ataque. (Ejemplo: Surf, Disparo demora, Látigo cepa, Lariat oscuro...)</li> <li>• Damage: Los puntos de daño que infringe el ataque en combate.</li> <li>• Descripción: Un texto informativo sobre los efectos del ataque.</li> </ul>                                                                                                                          |

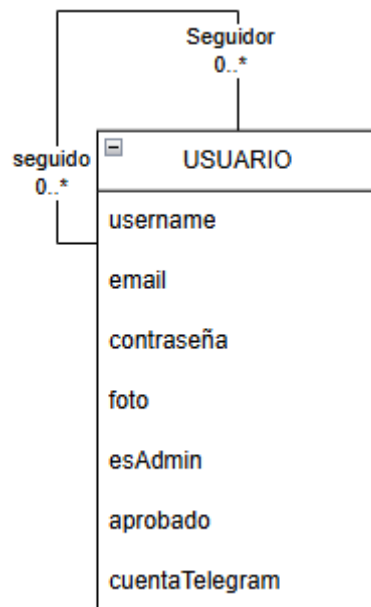
| Fecha       |                                                          |
|-------------|----------------------------------------------------------|
| Descripción | La fecha en la que un usuario notifica a sus seguidores. |

|           |                                                                                                                                                                                                   |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Porqué    | Un usuario puede notificar a sus seguidores varias veces. Si un usuario, por ejemplo, cambia su equipo 5 veces en los últimos 5 minutos, se ha tenido que notificar cada cambio a sus seguidores. |
| Atributos | <ul style="list-style-type: none"> <li>Fecha: Día, año, mes, hora y minuto en el que usuario A notifica a sus seguidores.</li> </ul>                                                              |

## Relaciones

### 1. Usuario-Usuario

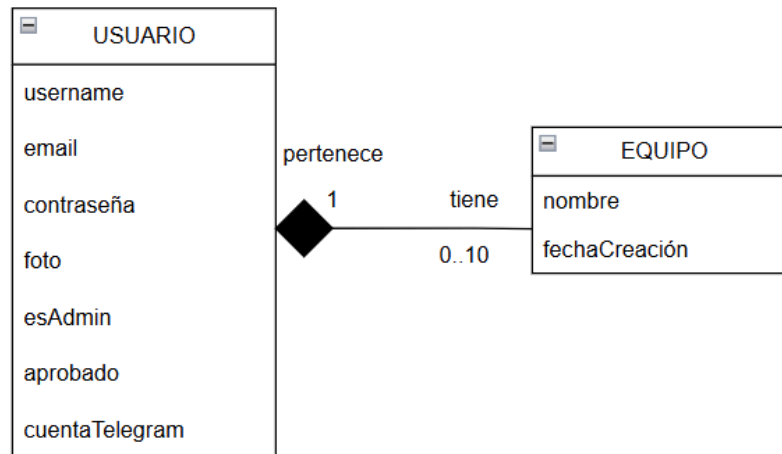
Los usuarios pueden seguir a otros usuarios (0..\*) y ser seguidos por varios usuarios (0..\*). Se quiere guardar quién sigue a quién. La fecha se guarda mediante la notificación. Solo si un usuario deja de seguir a otro, lo puede volver a seguir.



*Ilustración 46 - Relación usuario-usuario*

### 2. Usuario-Equipo

Un usuario puede tener de 0 a 10 equipos, pero un equipo sólo puede pertenecer a un usuario en específico. Se quiere almacenar qué equipos tiene un usuario, y cada equipo a quién pertenece.

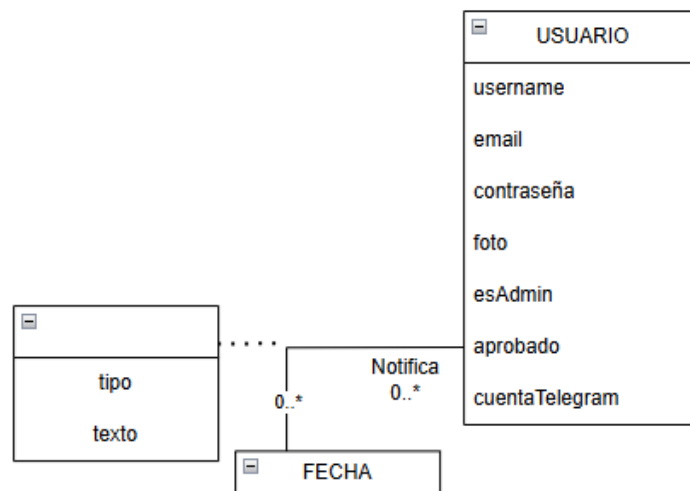


*Ilustración 47 - Relación usuario-equipo*

### 3. Usuario-Fecha

Un usuario puede notificar a sus seguidores en una fecha o ninguna o varias veces. También se guarda el tipo de notificación y el texto, ya que se debe enviar un texto predeterminado a la notificación dependiendo de lo que se haya cambiado.

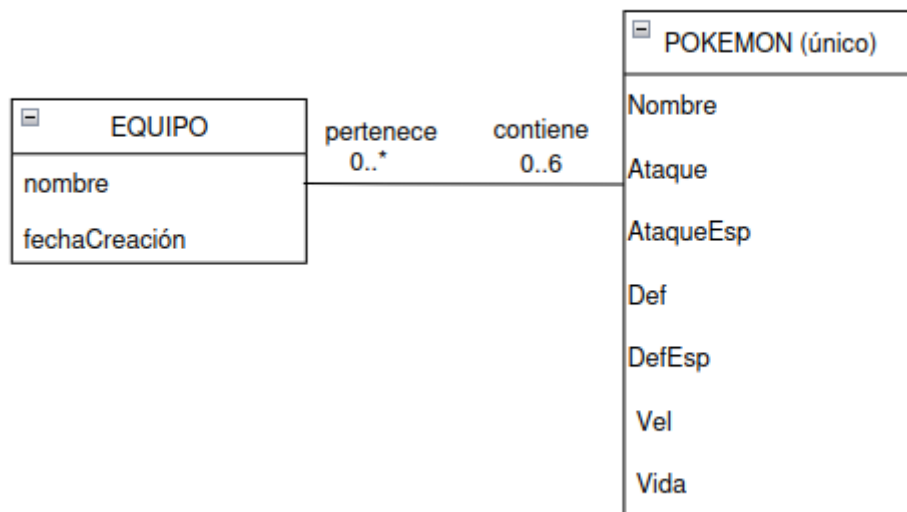
Por ejemplo, si la notificación es de tipo de creación de equipo, se tendrá que notificar a sus seguidores con el texto predeterminado correspondiente (“¡Usuario A ha creado un equipo!”). Puede haber muchos tipos de notificación: modificación de equipo, creación de equipo, seguimiento a usuario, etc.



*Ilustración 48 - Relación notificación*

### 4. Equipo-Pokémon

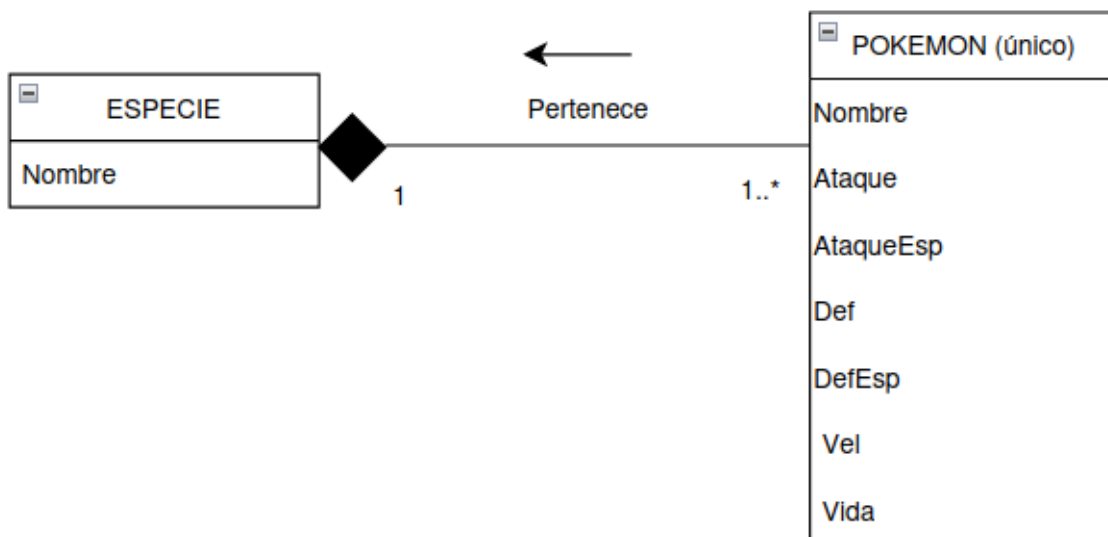
Un equipo puede estar formado como máximo de 6 pokémons. Pero un pokémon puede pertenecer a varios equipos. Así se almacena cada instancia de Pokémon y en qué equipo está.



*Ilustración 49 - Relación equipo-pokémon*

## 5. Pokémon-Especie

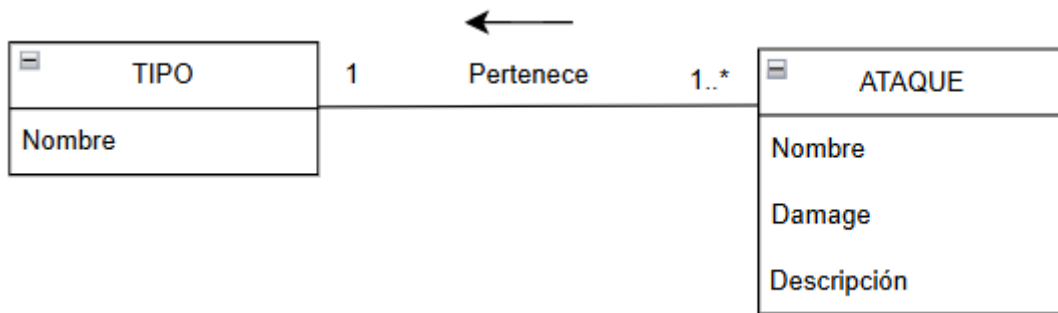
Un pokémon pertenece a una única especie, pero pueden existir varios pokémons pertenecientes a una misma especie.



*Ilustración 50 - Relación pokémon-especie*

## 6. Ataque-Tipo

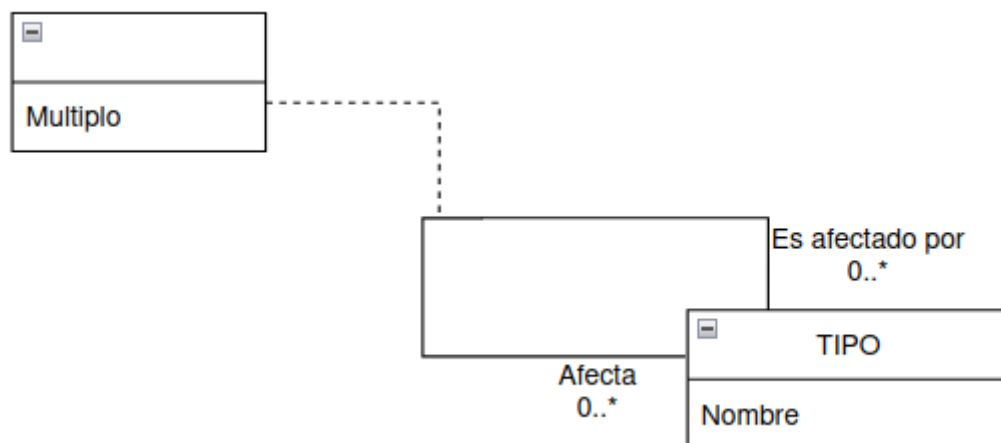
Una habilidad puede ser de un único tipo, pero hay varias habilidades de un mismo tipo.



*Ilustración 51 - Relación ataque-tipo*

## 7. Tipo -Tipo

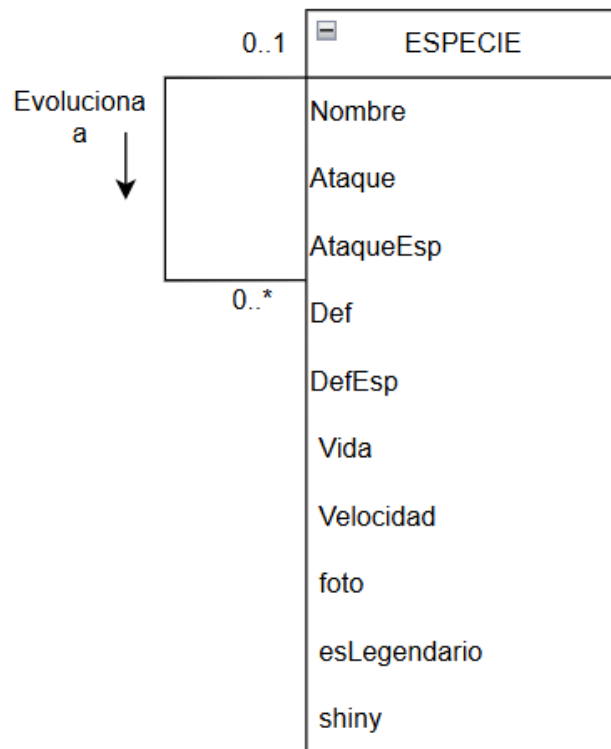
Un tipo afecta a ninguno o varios tipos, y de la misma manera puede ser afectado por ninguno o varios tipos. Si un tipo afecta a otro, lo hace con un multiplicador, que puede ser x2, x0.5 o x0.



*Ilustración 52 - Relación tipo-tipo*

## 8. Especie - Especie

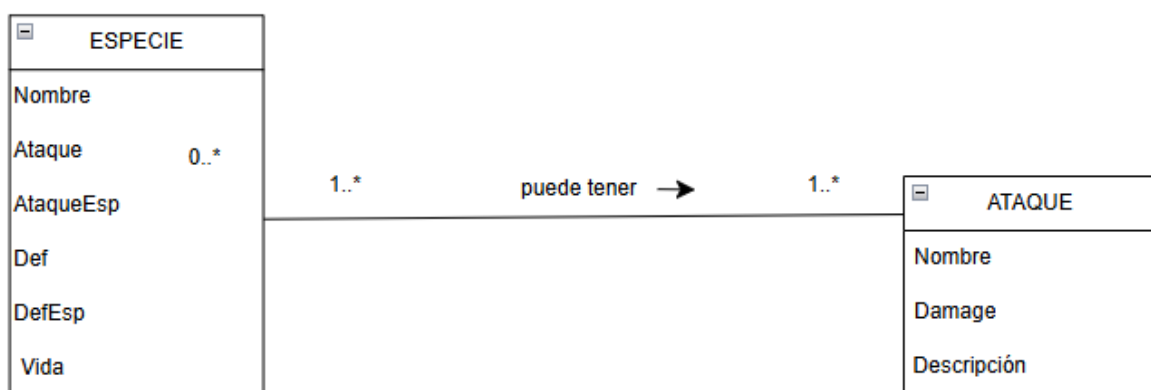
Una especie pokémon puede evolucionar a ninguna, o a varias otras especies. Una especie pokémon puede ser la evolución de ninguna o de una especie.



*Ilustración 53 - Relación evolución*

## 9. Especie - Ataque

Una especie pokémon tiene varias posibles habilidades que puede aprender. Una habilidad la pueden llegar a aprender una o varias especies pokémon.

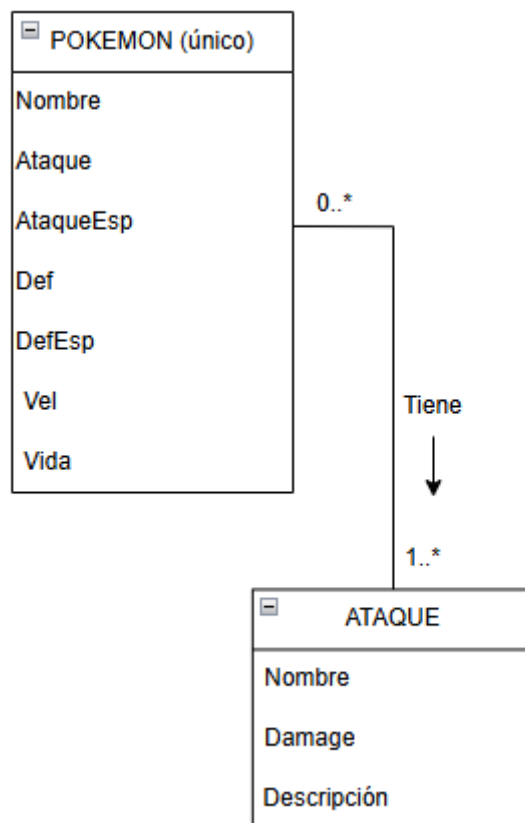


*Ilustración 54 - Relación especie-ataque*



## 10. Pokémon- Ataque

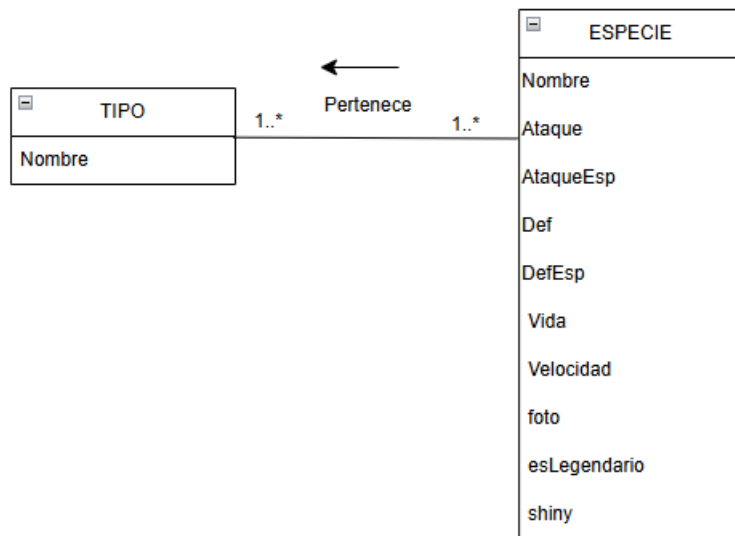
Un pokémon puede tener una o varias habilidades aprendidas. Una habilidad la pueden haber aprendido uno o varios pokémon.



*Ilustración 55 - Relación pokémon-ataque*

## 11. Especie - Tipo

Una especie puede pertenecer a uno o más tipos, y existe una especie o más de cada tipo.



*Ilustración 56 - Relación especie-tipo*

## 4. DIAGRAMA DE CLASES Y ESQUEMA DE BASE DE DATOS

### 1. DIAGRAMA DE BASE DE DATOS

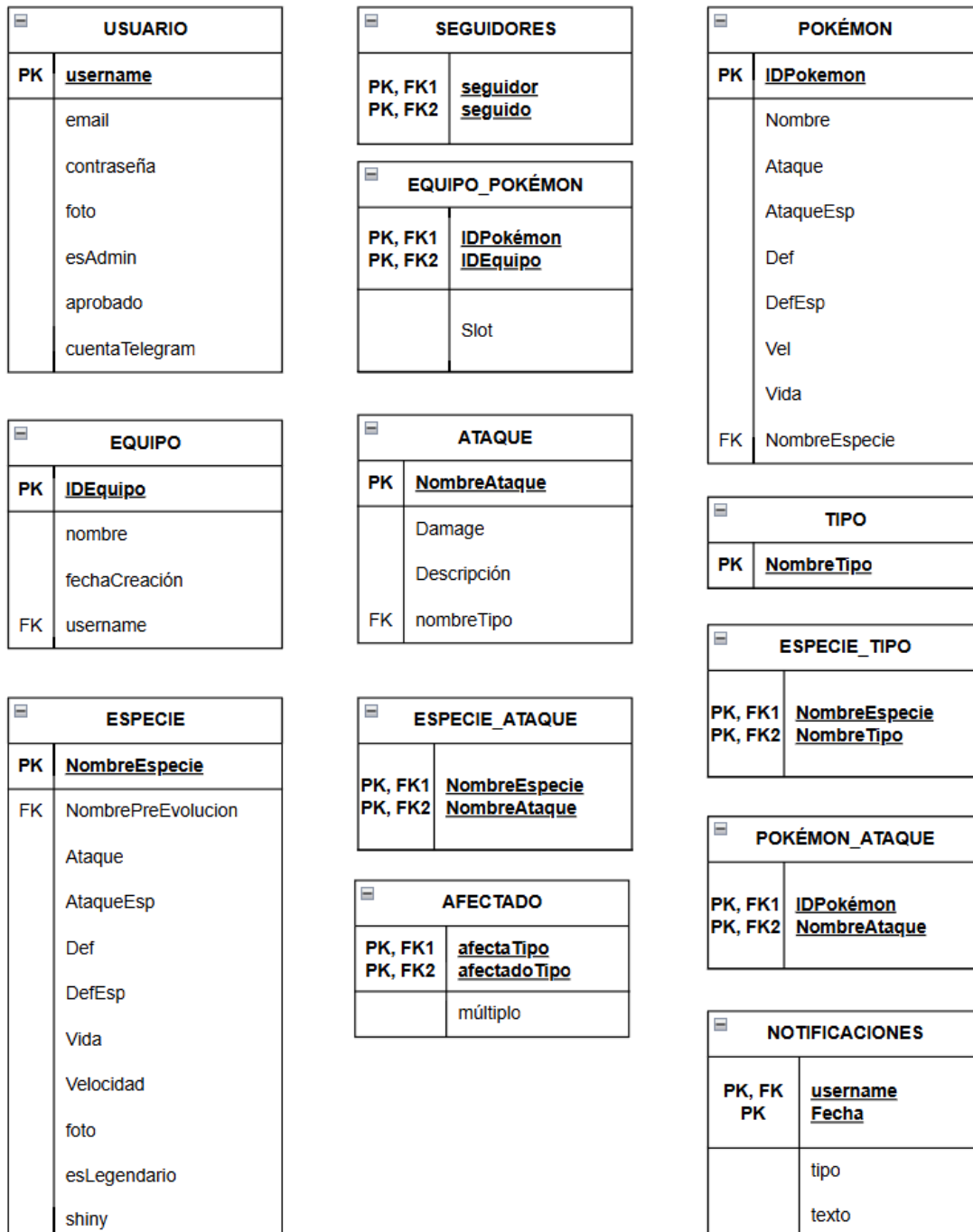
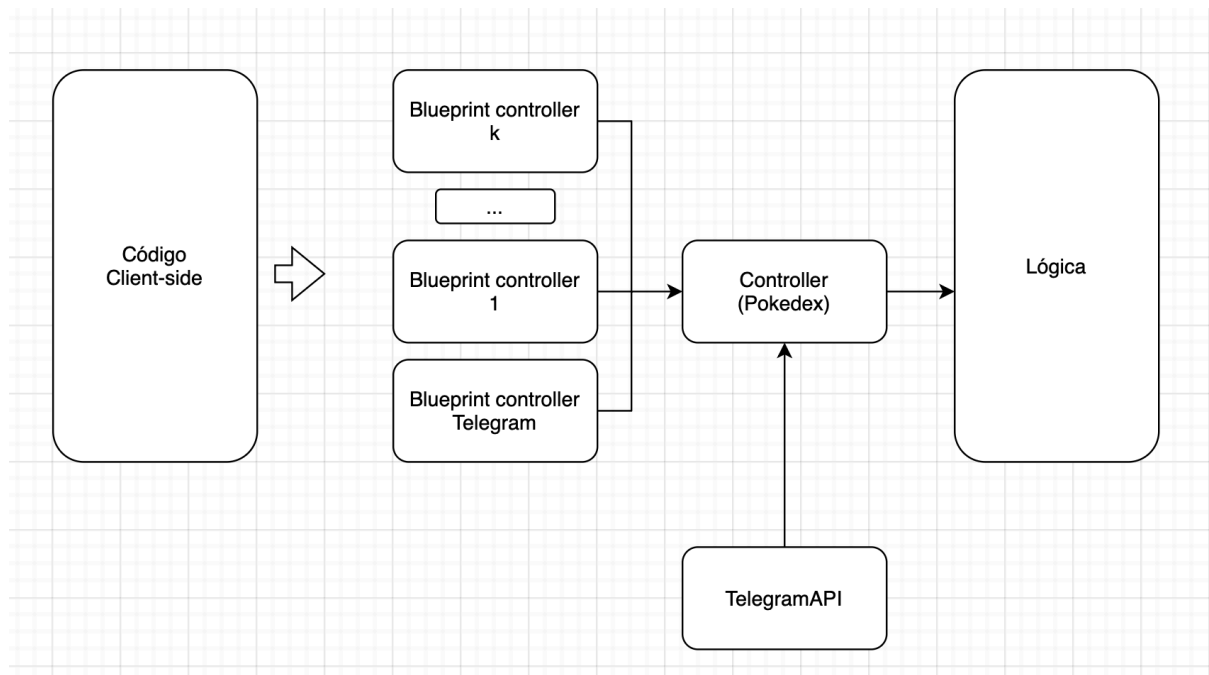


Ilustración 57 - Diagrama de la BD

El diagrama de clases ha cambiado un poco. Aunque en la siguiente figura no se muestra, el controlador Pokedex ahora se coordina junto a más controladores que reciben POST del frontend y realizan las llamadas a Pokedex. Esto se aprecia mejor en el diagrama siguiente de Telegram.



## 2.1 Sistema Telegram + rework de controladores



*Ilustración 59*

El controlador de Telegram se encarga de recibir el JSON del frontend y realizar la llamada para asociar la cuenta de Telegram con la APP.

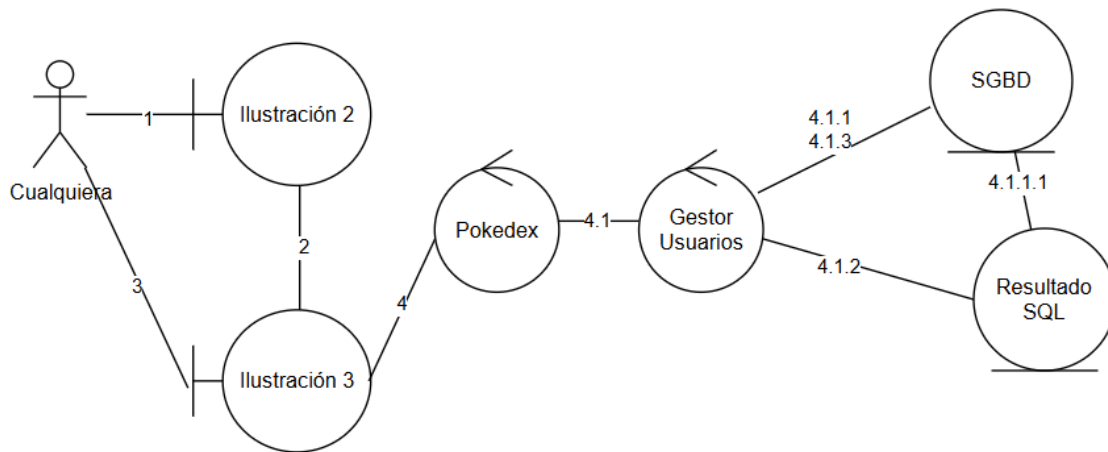
Por otro lado, TelegramAPI se conecta directamente con el controlador principal (Pokédex) pidiéndole la información necesaria para interactuar luego dentro de Telegram. Es la clase responsable de llevar a cabo la ejecución de los comandos de Telegram.

## 5. DIAGRAMAS DE COMUNICACIÓN

### 5.1 GESTIÓN DE USUARIOS

#### 5.1.1 Registrarse

Este caso de uso permite a un actor **Cualquiera** crear una cuenta de usuario, la cual se guardará en la base de datos con el estado inicial de “pendiente de aprobación”.



*Ilustración 60 - Diagrama Comunicación Registrarse*

1. En la página de inicio de sesión, pulsa "Registrarse"
2. new Ilustración3()
3. Pulsa "Registrarse"
4. registrarse(username, email, password, confirm\_password): string
  - 4.1. crearCuenta(username, email, password, confirm\_password): string
    - 4.1.1. execSQL(sql1): ResultadoSQL
      - 4.1.1.1. new ResultadoSQL()
    - 4.1.2. next(): bool
      - [SI FALSE]
    - 4.1.3. execSQL(sql2)

sql1: "SELECT username FROM USUARIO WHERE username=%username%"

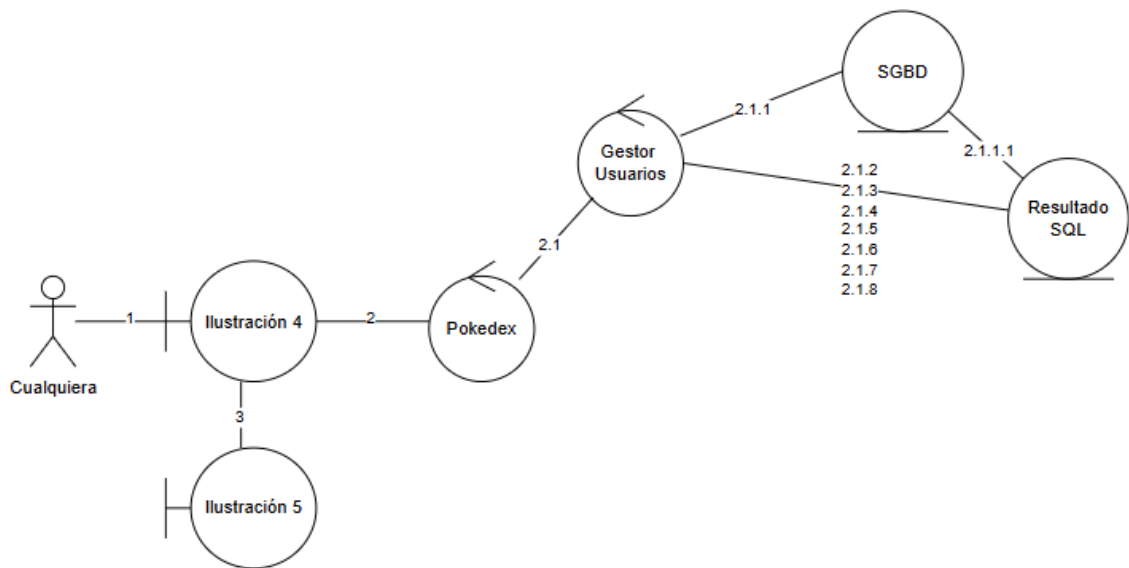
sql2: "INSERT INTO USUARIO(username, email, contraseña, esAdmin, aprobado) VALUES (%username%, %email% %contrasena% 0, 0)"

#### EXPLICACIÓN:

- El actor pulsa "Registrarse" en la vista de inicio de sesión (Ilustración 2) para dirigirse a la vista de registro (Ilustración 3), allí ingresa sus datos (nombre de usuario, email y contraseña) y pulsa registrarse.
- El controlador Pokédex ejecuta la función registrarse(...) que delega a GestorUsuarios.crearCuenta(...).
- Se verifica en la base de datos si el username ya existe (sql1), en caso afirmativo, el proceso termina y devuelve un mensaje de aviso, de lo contrario, continúa con el registro insertando los datos del usuario en la base de datos.

### 5.1.2 Iniciar sesión

El proceso de inicio de sesión se inicia con la interacción del actor **Cualquiera** y su objetivo es verificar la identidad del usuario y activar su sesión en el sistema.



*Ilustración 61 - Diagrama Comunicación Iniciar sesión*

1. Introduce los credenciales y pulsa "Iniciar sesión"
2. iniciarSesión(username, password): Tuple[JSON, string]
  - 2.1. validarUsuario(user, password): Tuple[JSON, string]
    - 2.1.1. execSQL(sql): ResultadoSQL
      - 2.1.1.1. new ResultadoSQL()
    - 2.1.2. next(): bool
    - 2.1.3. getString("username"): string
    - 2.1.4. getString("email"): string
    - 2.1.5. getString("foto"): string
    - 2.1.6. getString("contrasena"): string
    - 2.1.7. getBool("esAdmin"): bool
    - 2.1.8. getBool("aprobado"): bool

[si devuelve un None]

Falla el inicio de sesión y muestra el mensaje de error por pantalla

[si JSON no está vacío]

3. new Ilustración5(username)

sql="SELECT \* FROM USUARIO WHERE username=%user% AND contraseña=%pass% AND aprobado=1"

JSON = {username: string, email: string, contrasena: string, foto: string, esAdmin: bool, aprobado: bool}

string: es un mensaje que muestra el resultado del inicio de sesión (como, usuario o contraseña incorrectos, cuenta pendiente de aprobación o inicio de sesión exitoso)

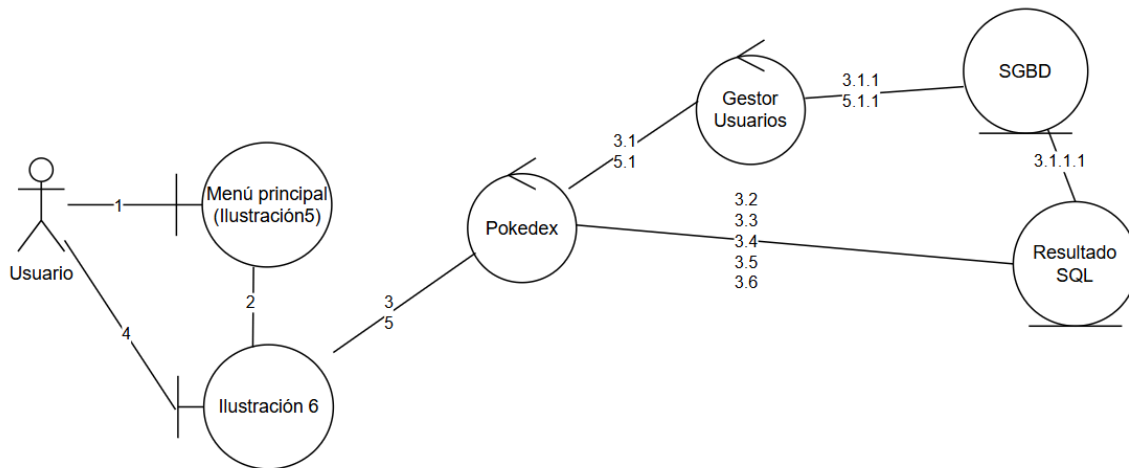
#### EXPLICACIÓN:

- En la vista login (Ilustración 4), el actor introduce sus credenciales y pulsa "Iniciar sesión".
- El sistema ejecuta una consulta SQL que verifica si las credenciales son correctas y si la cuenta está aprobada.

- Si lo que devuelve es un None (diccionario vacío) entonces muestra por pantalla el mensaje de error. Y si JSON (o diccionario) no está vacío entonces se abre la pantalla principal.

### 5.1.3 Actualizar datos

El siguiente diagrama muestra el flujo de acceder al perfil y modificar los datos del usuario logueado.



*Ilustración 62 - Diagrama Comunicación Actualizar datos*

1. Pulsa el icono de perfil
2. new Ilustración6(username)
3. buscarUsuarioLogueado(username): JSON
  - 3.1. buscarUsuarioLogueado(username): JSON
    - 3.1.1. execSQL(sql1): ResultadoSQL
      - 3.1.1.1. new ResultadoSQL()
  - 3.2. next()
  - 3.3. getString("username"): string
  - 3.4. getString("email"): string
  - 3.5. getString("contrasena"): string
  - 3.6. getString("foto"): string
4. Introduce nuevos cambios en los campos deseados y pulsa "Guardar cambios"
5. actualizarDatos(username, email, foto, password, confirm\_pass): Tuple[bool, string]
  - 5.1 actualizarDatos(username, email, foto, password, confirm\_pass): Tuple[bool, string]
    - 5.1.1. execSQL(sql2)

sql1: SELECT username, email, contrasena, foto FROM USUARIO WHERE  
username=%username%

sql2: "UPDATE USUARIO SET email=%email%, contraseña=%contraseña%, foto=%foto%  
WHERE username=%username%"

JSON = {username: string, email: string, contrasena: string, foto: string}

#### EXPLICACIÓN:

- En el menú principal (Ilustración 5) pulsa en el icono del perfil y se busca los datos del usuario para mostrar en pantalla.

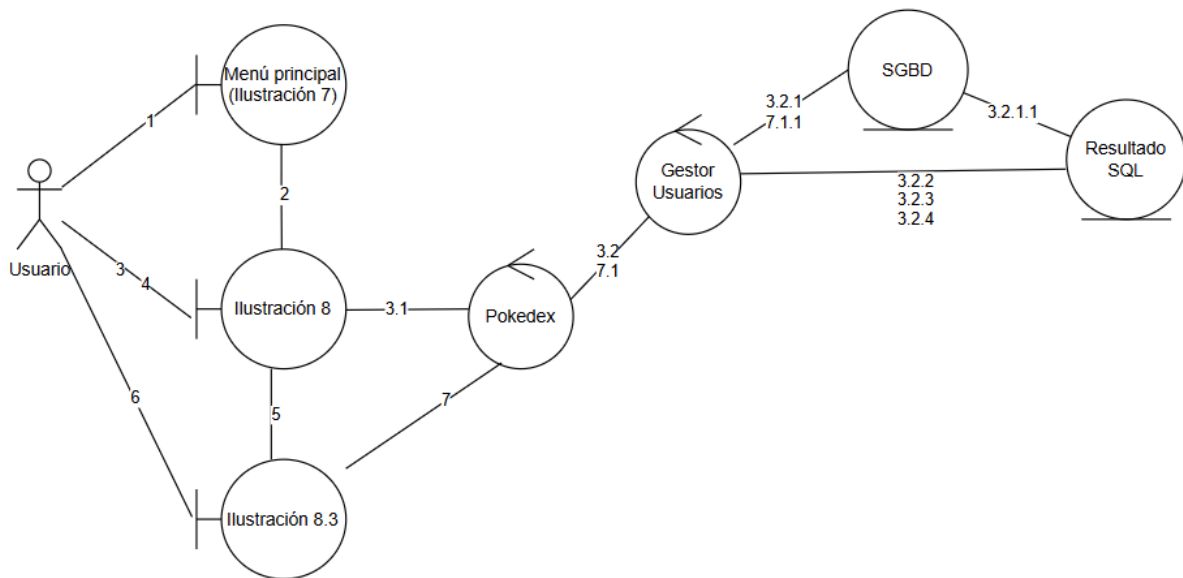


- El usuario introduce nuevos cambios y pulsa en “Guardar cambios”.
- Se ejecuta una sentencia UPDATE en la base de datos para modificar los datos del usuario en la tabla USUARIO.

### 5.1.4 Gestionar amigos

#### Listar usuarios seguidos y dejar de seguir

En el siguiente diagrama de comunicación se refleja el flujo de listar usuarios seguidos y dejar de seguir a un usuario seleccionado.



*Ilustración 63 - Diagrama Comunicación Gestionar amigos*

1. En la página principal, pulsa en "Gestión de usuarios"
2. new Ilustración8(username)
3. Pulsar en "Ver seguidos"
  - 3.1. get\_following(username): JSON
  - 3.2. get\_following(username): JSON
    - 3.2.1. execSQL(sql1): ResultadoSQL
      - 3.2.1.1 newResultadoSQL()
    - 3.2.2. next()
    - 3.2.3. getString("username")
    - 3.2.4. getString("foto")

[Si pulsa algo]

4. Pulsa "Dejar de seguir" sobre un usuario
5. new Ilustración8.3(username)
6. Pulsa en "Aceptar"
7. dejarDeSeguir(seguidor, seguido)
  - 7.1. dejarDeSeguir(seguidor, seguido)
    - 7.1.1. execSQL(sql3)

sql1: "SELECT u.username, u.foto FROM USUARIO u INNER JOIN SEGUIDORES s ON u.username=s.seguido WHERE s.seguidor=%username%"

Si se quiere dejar de seguir a un usuario:

sql3: "DELETE FROM SEGUIDORES WHERE seguidor=%seguidor% AND seguido=%seguido%"

Lo mismo para ver la lista de seguidores del usuario (los pasos del punto 3), varía un poco el SELECT y el método es get\_followers(username).

sql2: "SELECT u.username, u.foto FROM USUARIO u INNER JOIN SEGUIDORES s ON u.username=s.seguidor WHERE s.seguido=%username%"

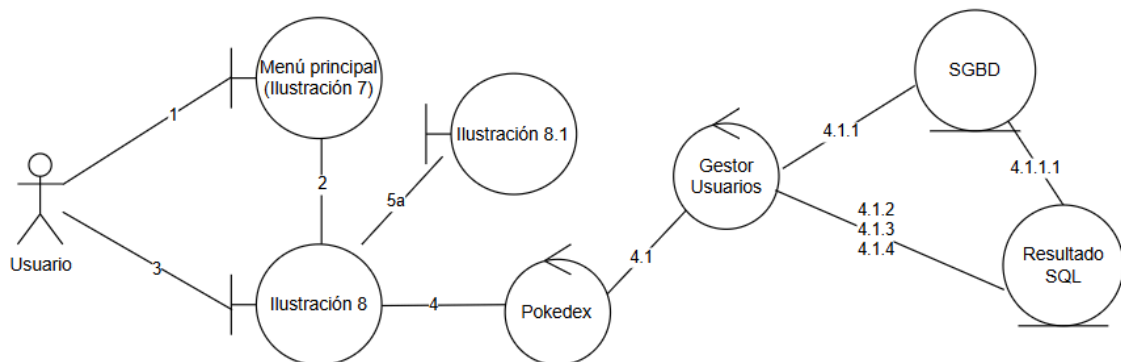
JSON: { [ {username: string, foto: string}, ... ] }

### EXPLICACIÓN:

El proceso de gestión de usuarios seguidos tiene dos fases:

- Listar usuarios seguidos (pasos 1 al 3.2.4)
  - El usuario en el menú principal pulsa “Gestionar amigos” y en la Ilustración 8 pulsa en “Ver seguidos”.
  - El sistema consulta la base de datos para obtener la lista de usuarios que sigue el usuario actual.
  - Se construye un JSON con los datos (username y foto) de los usuarios seguidos y se muestra la lista completa en la interfaz.
- Dejar de seguir (pasos 4 al 7.1.1)
  - El usuario selecciona un contacto de la lista de usuarios seguidos y pulsa “Dejar de seguir”.
  - Se ejecuta una sentencia DELETE en la tabla de SEGUIDORES para eliminar la relación.

### **Buscar usuario y seguirle**



*Ilustración 64 - Diagrama Comunicación Buscar y seguir usuario*

1. En la página principal, pulsa en "Gestión de usuarios"
2. new Ilustración8(username)
3. Introduce un nombre y pulsa en el icono de la lupa
4. buscarPorUsername(usuarioActual, usernameABuscar): JSON
  - 4.1. buscarPorUsername(usuarioActual, usernameABuscar): JSON
    - 4.1.1. execSQL(sql1): ResultadoSQL

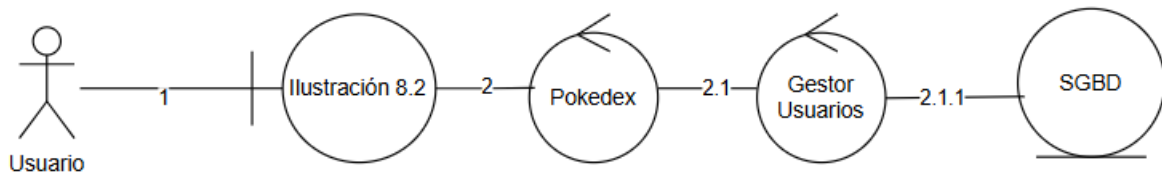
- 4.1.1.1. new ResultadoSQL()
- 4.1.2. next(): bool
- 4.1.3. getString("username"): string
- 4.1.4. getString("foto"): string

[Si JSON está vacío]

5a. new Ilustración8.1()

sql1: "SELECT username, foto FROM USUARIO WHERE username LIKE %usernameABuscar% AND username != %usuarioActual% AND aprobado =1 AND esAdmin = 0 ORDER BY USERNAME"

El siguiente diagrama representa cuando un usuario quiere seguir a otro entrenador tras la búsqueda.



*Ilustración 65 - Diagrama Comunicación Búsqueda*

[Si JSON no está vacío y pulsa algo]

1. Seleccionar usuario y pulsa "Seguir"
2. seguir(seguidor, seguido): bool
  - 2.1. seguir(seguidor, seguido): bool
    - 2.1.1. execSQL(sql2)

sql2: "INSERT INTO SEGUIDORES (seguidor, seguido) VALUES(%seguidor%, %seguido%)"

### EXPLICACIÓN:

El proceso de búsqueda y gestión de seguimiento tiene dos fases:

- Búsqueda de usuario
  - El usuario en el menú principal (Ilustración 7) pulsa “Gestión de usuarios” y en la barra de búsqueda (Ilustración 8) introduce el nombre del usuario a buscar.
  - El sistema consulta la base de datos usando consultas con LIKE para encontrar coincidencias parciales en usernames y se devuelve un JSON con los usuarios encontrados (username y foto) para mostrar en pantalla.
- Gestión de seguimiento
  - Selecciona un usuario y pulsa “Seguir”. Y se insertará en la tabla de seguidores una nueva relación de seguimiento.

## 5.1.5 Gestionar cuentas

### Obtener cuentas pendientes de aprobación y cuentas aprobadas

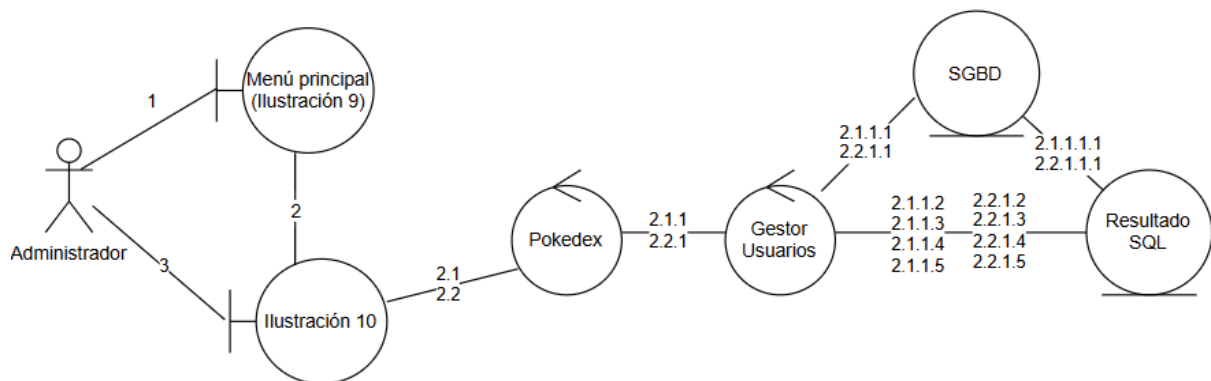


Ilustración 66 - Diagrama Comunicación Gestionar cuentas

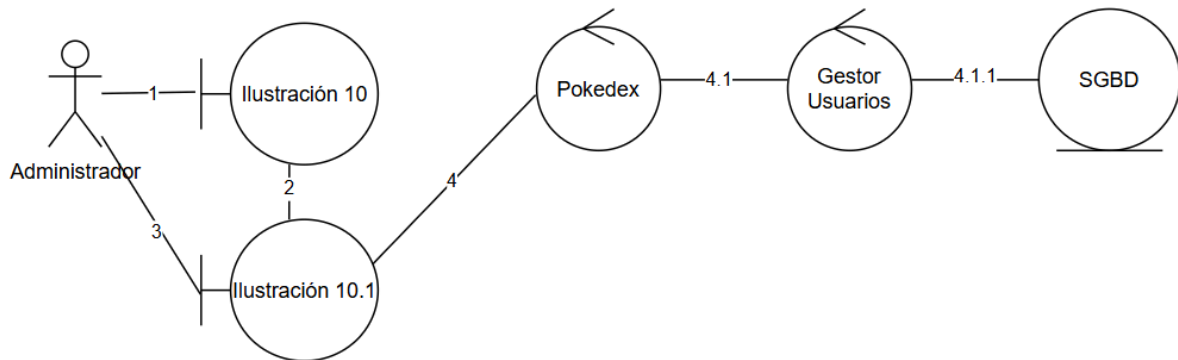
1. En el menú principal, pulsa "Gestión de usuarios"
2. new Ilustración10()
  - 2.1. obtenerCuentasPendientes(): JSON
    - 2.1.1. obtenerCuentasPendientes(): JSON
      - 2.1.1.1. execSQL(sql1): ResultadoSQL
        - 2.1.1.1.1. new ResultadoSQL()
      - 2.1.1.2. next()
      - 2.1.1.3. getString("username")
      - 2.1.1.4. getString("email")
    - 2.2. obtenerCuentasAprobadas(): JSON
      - 2.2.1. obtenerCuentasPendientes(): JSON
        - 2.2.1.1. execSQL(sql2): ResultadoSQL
          - 2.2.1.1.1. new ResultadoSQL()
        - 2.2.1.2. next()
        - 2.2.1.3. getString("username")
        - 2.2.1.4. getString("email")

sql1: "SELECT username, email FROM USUARIO WHERE aprobado=0"

sql2: "SELECT username, email FROM USUARIO WHERE aprobado=1"

JSON: { [ {username: string, email: string}, ...] }

## Aprobar cuenta



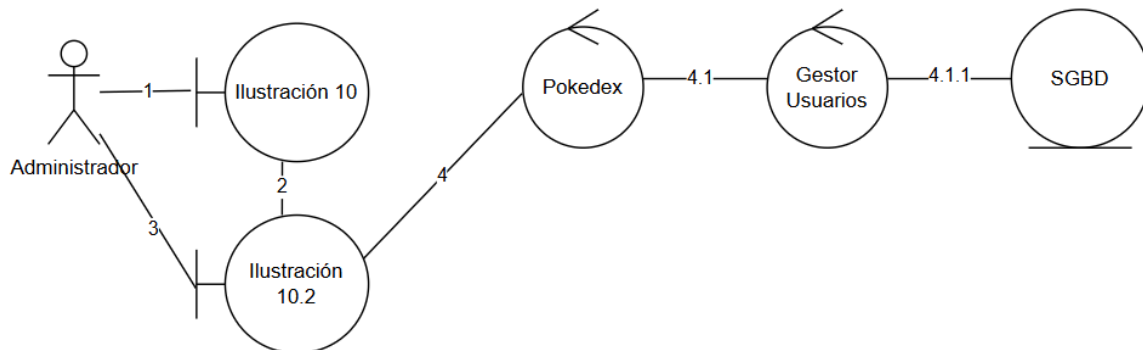
*Ilustración 67 - Diagrama Comunicación Aprobar cuenta*

[Si pulsa algo]

1. Seleccionar cuenta y pulsa "Aprobar"
2. new Ilustración10.1()
3. Pulsa "Aceptar"
4. aprobarCuenta(username): Tuple[bool. string]
  - 4.1. aprobarCuenta(username): Tuple[bool. string]
    - 4.1.1 execSQL(sql)

sql: "UPDATE USUARIO SET aprobado=1 WHERE username=%username%"

## Borrar cuenta



*Ilustración 68 - Diagrama Comunicación Borrar cuenta*

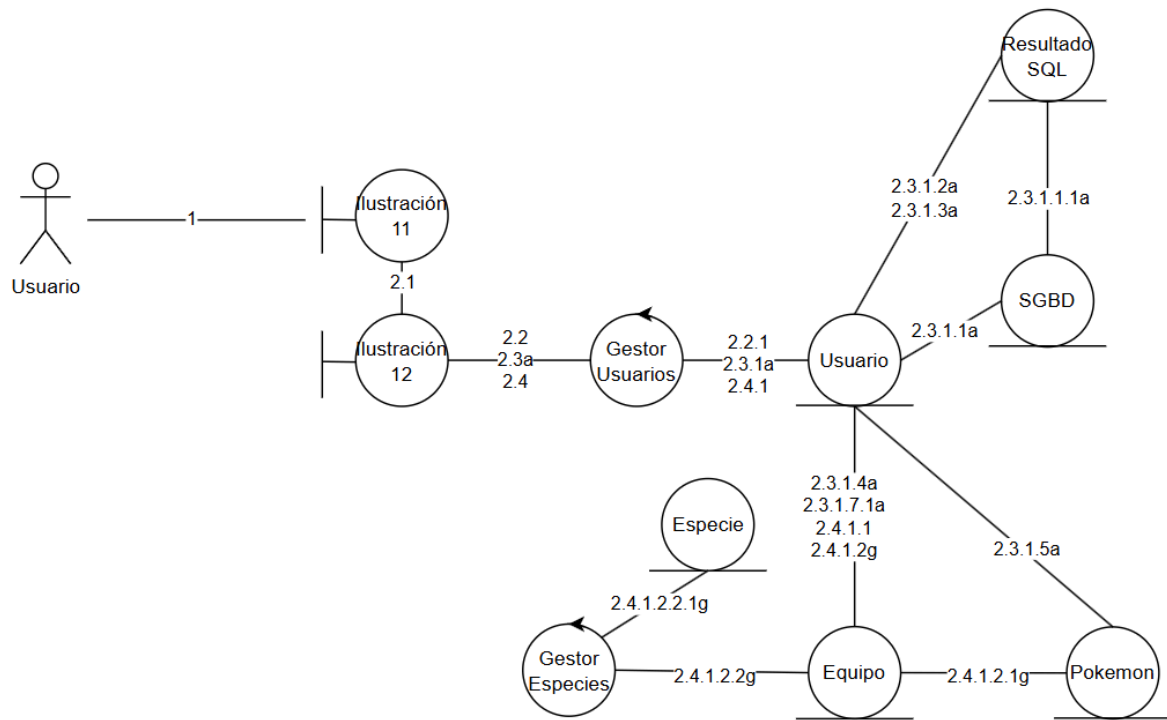
[Si pulsa algo]

1. Seleccionar cuenta y pulsa "Eliminar"
2. new Ilustración10.2()
3. Pulsa "Aceptar"
4. borrarCuenta(username): Tuple[bool. string]
  - 4.1. borrarCuenta(username): Tuple[bool. string]
    - 4.1.1 execSQL(sql)

sql: "DELETE FROM USUARIO WHERE username=%username%"

## 5.2 CREAR EQUIPOS POKÉMON

### 5.2.1 Ver un equipo Pokémon



*Ilustración 69 - Diagrama Comunicación Ver equipo*

1. Abrir aplicación para acceder al menú principal
2. Hacer click en el botón “Ver equipos”
  - 2.1 new Ilustración 12()
  - 2.2 existeEquipo(username,idequipo): bool
    - 2.2.1 existeEquipo(idequipo):bool
 [SI FALSE]
  - 2.3a cargarEquipo(username,idequipo):void
    - 2.3.1a cargarEquipo(idequipo):void
      - 2.3.1.1a execSQL(sql1)
        - 2.3.1.1.1a new ResultadoSQL():void
      - 2.3.1.2a next():bool
      - 2.3.1.3a getJSON():JSON1

```

2.3.1.4a nuevoEquipo= new Equipo(nombreEquipo)
2.3.1.5a nuevoPokemon = new Pokemon(nombrePokemon, nombreEspecie,
    ataque, ataqueEsp, defensa, defensaEsp, velocidad, vida, idpokemon,
    listaAtaques) <-- ejecutar por cada pokemon
2.3.1.6a addEquipo(nuevoEquipo):void
2.3.1.7a addPokemonAEquipo(nuevoPokemon,idequipo):void ← ejecutar por
    cada pokemon
2.3.1.7.1a addPokemon(nuevoPokemon):void
2.4 mostrarEquipo(username,idequipo):void
2.4.1 mostrarEquipo(idequipo):void
2.4.1.1 mismoId(idequipo):bool
[SI TRUE]
2.4.1.2g mostrarPokemons():void
2.4.1.2.1g getNombrePokemon():String ←por cada Pokemon
2.4.1.2.2g getImagen(nombrePokemon):image←por cada
    Pokemon
2.4.1.2.2.1g estaPokemon(nombrePokemon):bool

```

JSON1: {nombreEquipo:String,

[ {nombreEspecie:String, nombrePokemon:String, ataque:Int, ataqueespecial:Int, defensa:Int, defensaespecial:Int, velocidad:Int, vida:Int, idpokemon:Int, [ {nombreAtaque:String}, {} ] }, {} ] }

SQL1:

```

SELECT eq.nombreequipo, pok.nombreEspecie, pok.nombre, pok.ataque, pok.ataqueespecial,
pok.defensa, pok.defensaespecial, pok.velocidad, pok.vida, pok.IDPokemon, atq.nombre
FROM Equipo AS eq, Equipo_Pokemon AS eqpok, Pokemon AS pok, Ataque AS atq,
Pokemon_Ataque AS atqpok, Usuario AS usr
WHERE eqpok.idequipo = %idequipo% AND usr.username = %username% AND eq.idequipo =
eqpok.idequipo AND pok.idpokemon = eqpok.idpokemon AND atq.nombre = atqpok.nombreataque
AND usr.username = eq.username

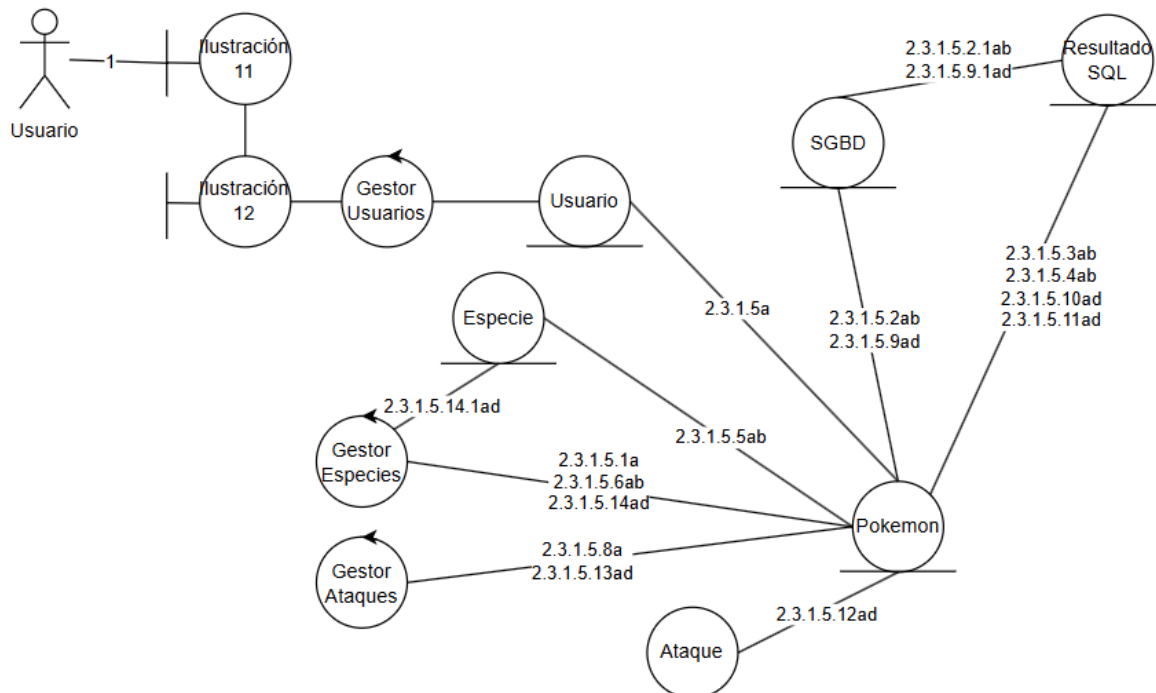
```

### EXPLICACIÓN:

- La creación de pokémons, equipos, tipos, y especies son algo complejas y ocupan mucho texto y espacio en los diagramas, así que aparecen por separado.

- Dado un id de equipo y un nombre de usuario, acceder al gestor de usuarios para ver si está el equipo a mostrar cargado o no.
- Encontrar al usuario en el gestor, y acceder al usuario dándole el id del equipo como parámetro para encontrar el equipo.
- Si no está (1), cargar equipo. Para ello, pedir a la base de datos el nombre del equipo, los atributos de los Pokémon que forman parte de él, y los nombres de esas especies. Además, por cada Pokémon se pide también una lista de nombres de ataques que tienen aprendidos.
- (1) Se crea un nuevo equipo con el nombre obtenido.
- (1) Por cada Pokémon, se crea un nuevo Pokémon con la información obtenida.
- (1) Se añade el nuevo equipo creado a la lista de equipos del usuario.
- (1) Por cada Pokémon, se añade al equipo recién creado.

### 5.2.1.1 Crear un Pokémon



*Ilustración 70 - Diagrama Comunicación Crear Pokémon*

2.3.1.5a nuevoPokemon = new Pokemon(nombrePokemon, nombreEspecie, ataque, ataqueEsp, defensa, defensaEsp, velocidad, vida, idpokemon, listaAtaques)

2.3.1.5.1a existeEspecie(nombreEspecie):bool

[SI FALSE]

2.3.1.5.2ab execSQL(sql2)



```

2.3.1.5.2.1ab new ResultadoSQL(): void
2.3.1.5.3ab next():bool
2.3.1.5.4ab getJSON():JSON2
2.3.1.5.5ab nuevaEspecie = new Especie(nombreEspecie, ataque, ataqueEsp,
defensa, defensaEsp, velocidad, vida, listaTipos):void
2.3.1.5.6ab addEspecie(nuevaEspecie):void
2.3.1.5.7a addPokemonAEspecie(nombrePokemon):void
2.3.1.5.8a existeAtaque(nombreAtaque):bool <-- ejecutar por cada ataque (*)
[SI FALSE]
2.3.1.5.9ad execSQL(sql3)<-- (*)
2.3.1.5.9.1ad new ResultadoSQL():void
2.3.1.5.10ad next():bool<-- (*)
2.3.1.5.11ad getJSON():JSON3<-- (*)
2.3.1.5.12ad nuevoAtaque = new Ataque(nombreAtaque, damage,
descripcion, nombreTipo):void<-- (*)
2.3.1.5.13ad addAtaque(nuevoAtaque):void<-- (*)
2.3.1.5.14ad addAtaqueAEspecie(nombreEspecie, nombreAtaque):void<--(*)
2.3.1.5.14.1ad addAtaque(nombreAtaque):void

```

JSON2: {nombreEspecie:String, ataque:Int, ataqueEsp:Int, defensa:Int, defensaEsp:Int, Vida:Int, Velocidad:Int, esLegendario:bool, esShiny:bool, [{nombreTipo:String},{}]}

JSON3: {nombreAtaque:String, damage:Int, descripcion:String, nombreTipo:String}

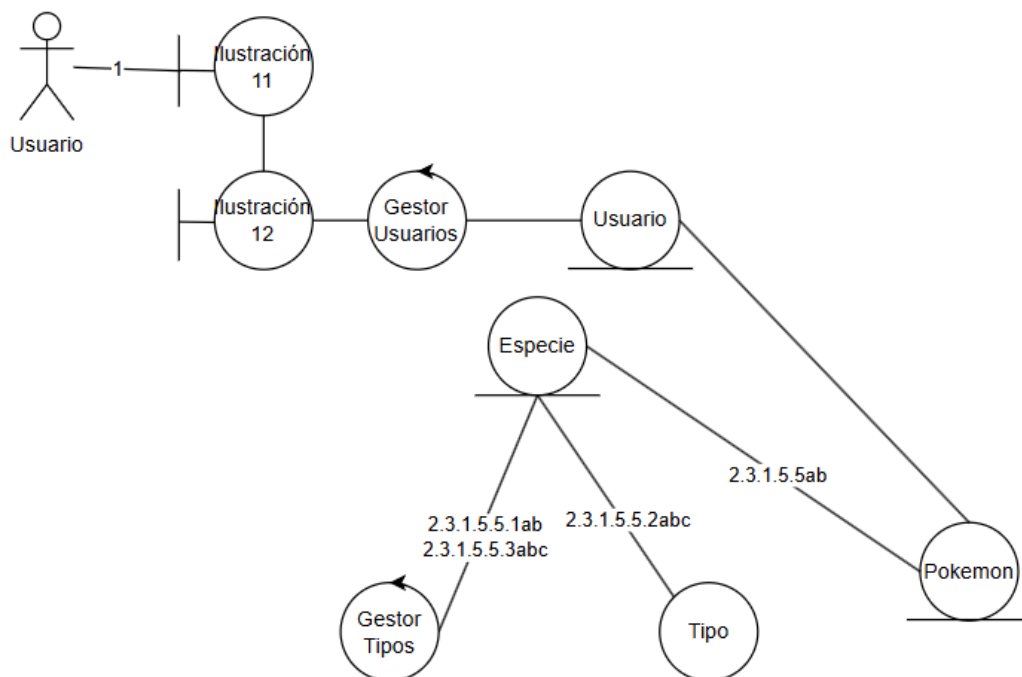
SQL2: SELECT e.NombreEspecie, e.ataque, e.ataqueespecial, e.defensa, e.defensaespecial, e.velocidad, e.vida, e.foto, e.esLegendario, e.esShiny, t.nombretipo  
FROM Especie AS e, Tipo AS t, Especie\_Tipo AS et  
WHERE e.NombreEspecie = %nombreEspecie% AND e.nombreEspecie = et.nombreEspecie AND t.nombreTipo = et.nombreTipo

SQL3: SELECT a.nombreAtaque, a.damage, a.descripcion, a.nombreTipo  
FROM Ataque AS  
WHERE a.nombreAtaque = %nombreAtaque%

### EXPLICACIÓN

- Crear un Pokémon pasándole sus estadísticas, y una lista de nombres de ataques.
- Comprobar si ya está cargada la especie a la que pertenece. Si no (1), hay que cargarla.
- (1) Pedir a la base de datos las estadísticas de la especie, y una lista de los nombres de los tipos que tiene (fuego, agua, planta...)
- (1) Crear la nueva especie. Añadir la nueva especie al gestor de especies.
- (1) Añadir el nombre del nuevo Pokémon a la especie.
- Por cada ataque, comprobar si existe en el gestor de ataques, dándole un nombre de ataque al gestor. Si no existe (2), pedir a la base de datos los atributos del ataque.
- (2) Crear nuevo ataque con la información obtenida. Guardar el ataque en el gestor de ataques.
- (2) Añadir el nombre del ataque a la lista de ataques de la especie.

### 5.2.1.2 Crear una especie



*Ilustración 71 - Diagrama Comunicación Crear especie*

2.3.1.5.5ab new Especie(nombreEspecie, ataque, ataqueEsp, defensa, defensaEsp,  
velocidad, vida, listaTipos):void

2.3.1.5.5.1ab existeTipo(nombreTipo):bool <-- ejecutar por cada tipo

(\*)

[SI FALSE]

2.3.1.5.5.2abc nuevoTipo = new Tipo(nombreTipo):void<--(\*)

2.3.1.5.5.3abc addTipo(nuevoTipo):void<--(\*)

## EXPLICACIÓN

- Comprobar si existen los tipos de la especie. Para ello, Por cada tipo, acceder al gestor de tipos con el nombre del tipo y ver si está. Si no está, crear un nuevo tipo y añadirlo al gestor de tipos. También añadir su nombre a la lista de tipos de la especie (lista.add(nombreTipo);).

### 5.2.1.3 Crear un ataque

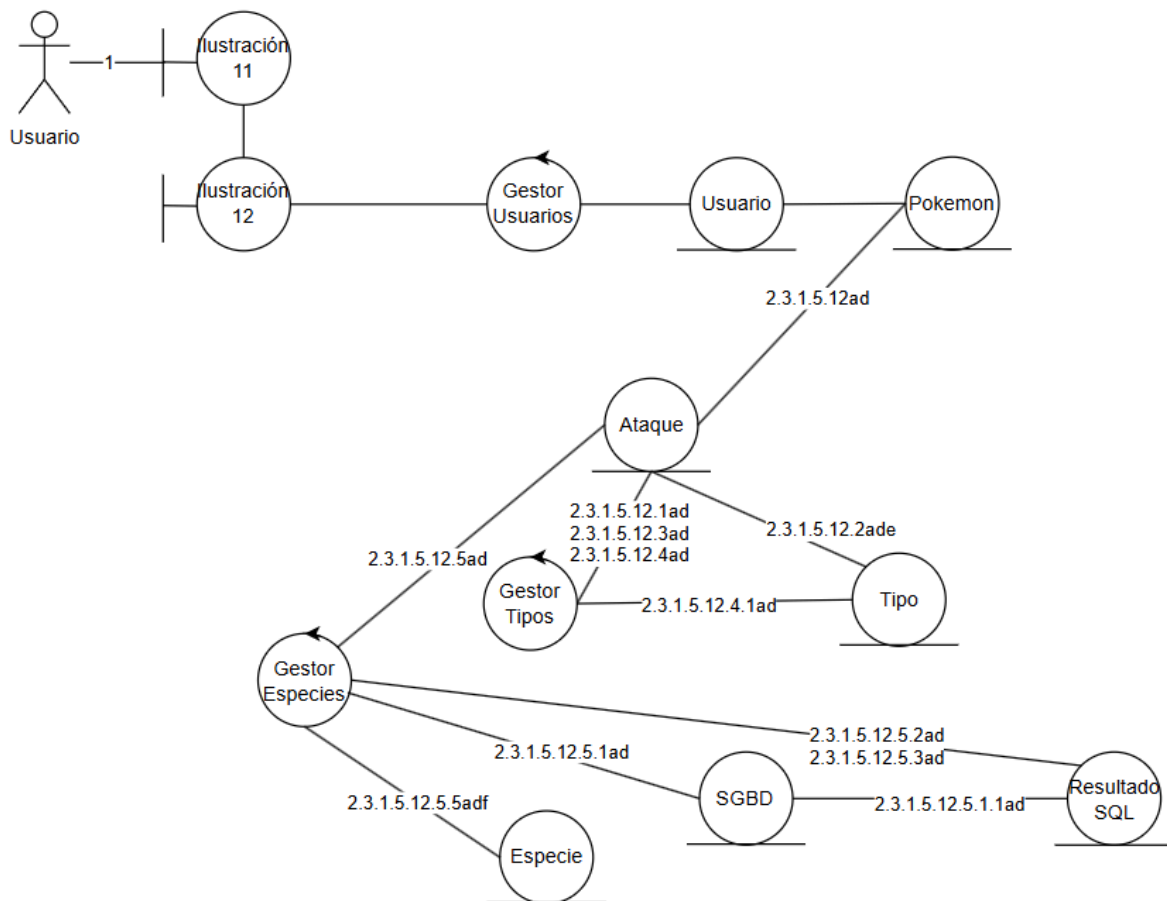


Ilustración 72 - Diagrama Comunicación Crear ataque

2.3.1.5.12ad nuevoAtaque = new Ataque(nombreAtaque, damage,  
descripcion, nombreTipo):void

2.3.1.5.12.1ad existeTipo(nombreTipo):bool

[SI FALSE]

2.3.1.5.12.2ade new Tipo(nombreTipo):void

2.3.1.5.12.3ade addTipo(nuevoTipo):void

2.3.1.5.12.4ad addAtaqueATipo(nombreAtaque, nombreTipo):void

2.3.1.5.12.4.1ad addAtaqueATipo(nombreAtaque):void

2.3.1.5.12.4.1.1ad addAtaque(nombreAtaque):void

2.3.1.5.12.5ad addAtaqueAEspecie(nombreAtaque):void

2.3.1.5.12.5.1ad execSQL(sql4)

2.3.1.5.12.5.1.1ad new ResultadoSQL():void

2.3.1.5.12.5.2ad next():bool

2.3.1.5.12.5.3ad getJSON():JSON4

2.3.1.5.12.5.4ad existeEspecie(nombreEspecie):bool <-- por  
cada especie

[SI TRUE]

2.3.1.5.12.5.5adf addAtaqueAEspecie(nombreAtaque,  
nombreEspecie):void<-- por cada especie

JSON4: {[{nombreEspecie:String},{}]}

SQL4: SELECT e.NombreEspecie

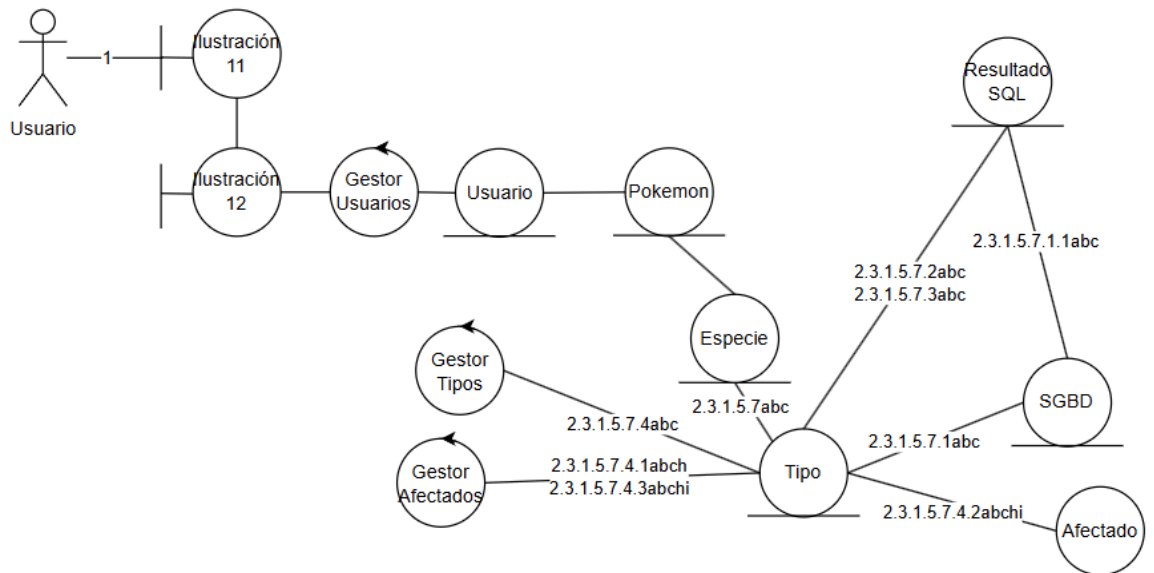
FROM Especie AS e, Ataque AS a , Especie\_Ataque AS ea

WHERE a.NombreAtaque = %nombreAtaque% AND ea.NombreEspecie = e.NombreEspecie AND  
ea.NombreAtaque = a.NombreAtaque

## EXPLICACIÓN

- Comprobar si existe el tipo del ataque, accediendo al gestor de tipos con el nombre del tipo. Si no existe, crear el nuevo tipo y añadirlo al gestor de tipos.
- Añadir el nombre del ataque a la lista de ataques del tipo al que pertenezca. Para ello, dado el nombre del tipo y del ataque, acceder al gestor de tipos.
- Añadir el nombre del ataque a la lista de ataques de las especies a las que pertenezca. Para ello, pedir a la base de datos una lista con los nombres de todas las especies que puedan aprender el ataque.
- Por cada especie que pertenezca a la lista obtenida, comprobar si existe. Si existe, añadir el ataque.

### 5.2.1.4 Crear un tipo



*Ilustración 73 - Diagrama Comunicación Crear tipo*

2.3.1.5.7abc new Tipo(nombreTipo):void

2.3.1.5.7.1abc execSQL(sql5)

2.3.1.5.7.1.1abc new ResultadoSQL()

2.3.1.5.7.2abc next():bool

2.3.1.5.7.3abc getJSON():JSON5

2.3.1.5.7.4abc existeTipo(otroTipo):bool <--Por cada otro tipo

[SI TRUE]

2.3.1.5.7.4.1abch existeAfectado(nombreTipo, otroTipo, multiplo):bool

[SI FALSE]

2.3.1.5.7.4.2abchi nuevoAfectado = new Afectado(nombreTipo, otroTipo,  
multiplo):void

2.3.1.5.7.4.3abchi addAfectado(nuevoAfectado):void

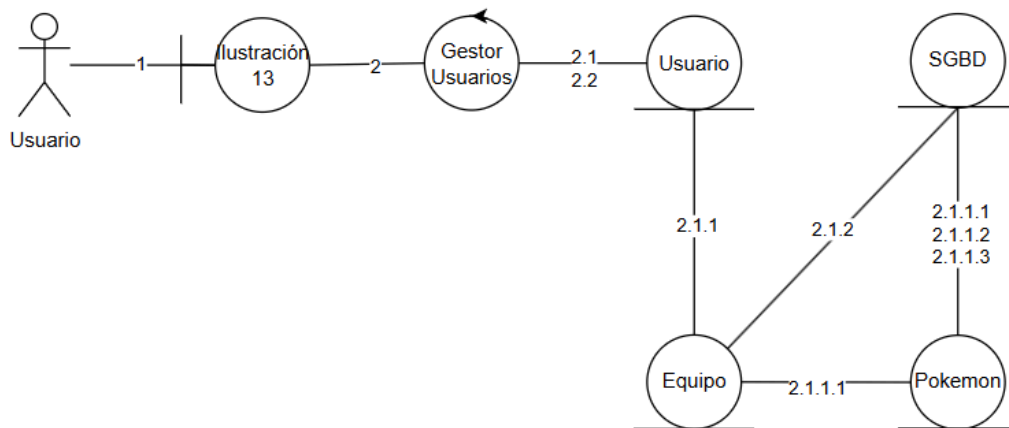
JSON5: {tipoAfecta:String, tipoAfectado:string, multiplo:Float}

SELECT \* FROM Afectado AS a WHERE a.afectaTipo = %nombreTipo% OR a.afectadoTipo = %nombreTipo%

EXPLICACIÓN:

- Para crear un tipo nuevo, dado un nombre de tipo, pedir a la base de datos todas las relaciones entre tipos en las que el tipo forma parte.
- Comprobar si existe el otro tipo que forma parte de la relación. Si existe, comprobar también si existe ya la relación, accediendo al gestor de afectados, proporcionando los nombres de los dos tipos y el múltiplo.
- Si no existe, crear nuevo afectado y añadirlo al gestor de afectados.

## 5.2.2 Eliminar un equipo Pokémon



*Ilustración 74 - Diagrama Comunicación Eliminar equipo*

1. Desde la interfaz de visualización de un equipo, hacer click en el botón "Eliminar" y luego en el botón "Sí"
2. eliminarEquipo(username,idequipo):void
  - 2.1 eliminarEquipo(idequipo):void
    - 2.1.1 eliminarPokemons():void
      - 2.1.1.1 eliminar(idequipo):void ←por cada pokemon
      - 2.1.1.2 execSQL(sql1)<-- por cada ataque
      - 2.1.1.3 execSQL(sql2)
      - 2.1.1.4 execSQL(sql3)
    - 2.1.2 execSQL(sql4)
  - 2.2 mostrarEquipo(idEquipo):void

SQL1: DELETE FROM Pokemon\_Ataque WHERE IDPokemon = %idPokemon% AND NombreAtaque = %nombreAtaque%

SQL2: DELETE FROM Equipo\_Pokemon WHERE IDPokemon = %idPokemon% AND IDEquipo = %idEquipo%

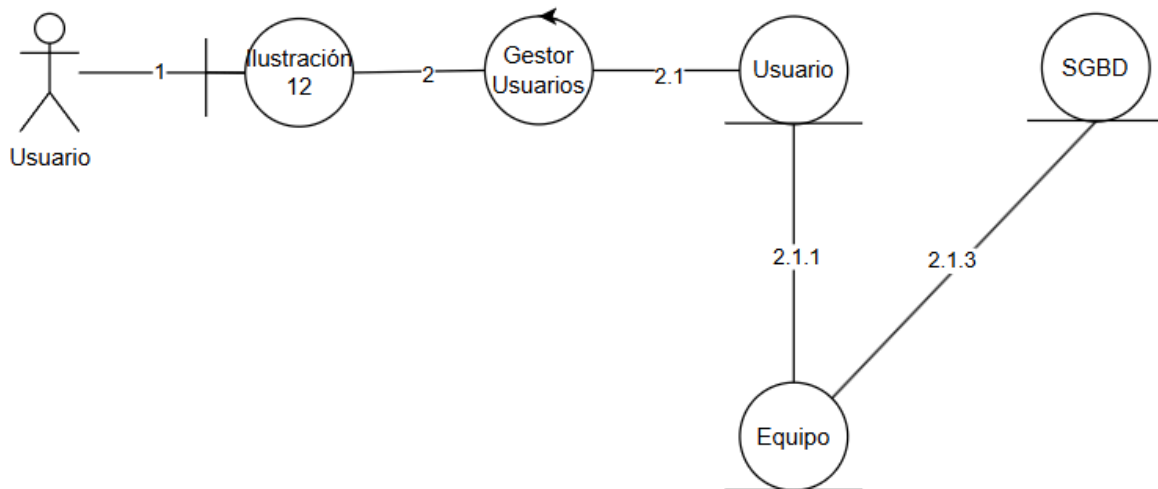
SQL3: DELETE FROM Pokemon WHERE IDPokemon = %idPokemon%

SQL4: DELETE FROM Equipo WHERE IDEquipo = %idEquipo%

### EXPLICACIÓN:

- Eliminar el equipo que se muestra actualmente, accediendo al gestor de usuarios y dándole el nombre de usuario y el id del equipo a eliminar. Se recorren los usuarios hasta encontrar el indicado.
- Acceder al usuario encontrado y dado un id, que elimine ese equipo suyo. Para ello, primero hay que eliminar los Pokémon del equipo.
- Eliminar los Pokémon consiste en recorrer la lista de Pokémon del equipo, y por cada Pokémon, eliminarlo. Para ello, dado el id del equipo, se accede al Pokémon, y se recorren los nombres de sus ataques. Por cada nombre de ataque, se accede a la base de datos y se elimina la relación entre el Pokémon y el ataque (el ataque sigue estando en el gestor de ataques).
- Para eliminar un Pokémon también se usa el id del Pokémon y del equipo. Se accede a la base de datos para eliminar la relación entre ellos dos.
- Después de esto, se regresa al equipo en cuestión y se borran todos los Pokémon de su lista de Pokémon, y se accede a la base de datos para eliminar el equipo.
- Por último, se regresa al usuario, y se elimina el equipo de su lista de equipos, reorganizando el número de todos los equipos y mostrando el nuevo equipo de número 1.

### 5.2.3 Crear un nuevo equipo Pokémon



*Ilustración 75 - Diagrama Comunicación Crear equipo*

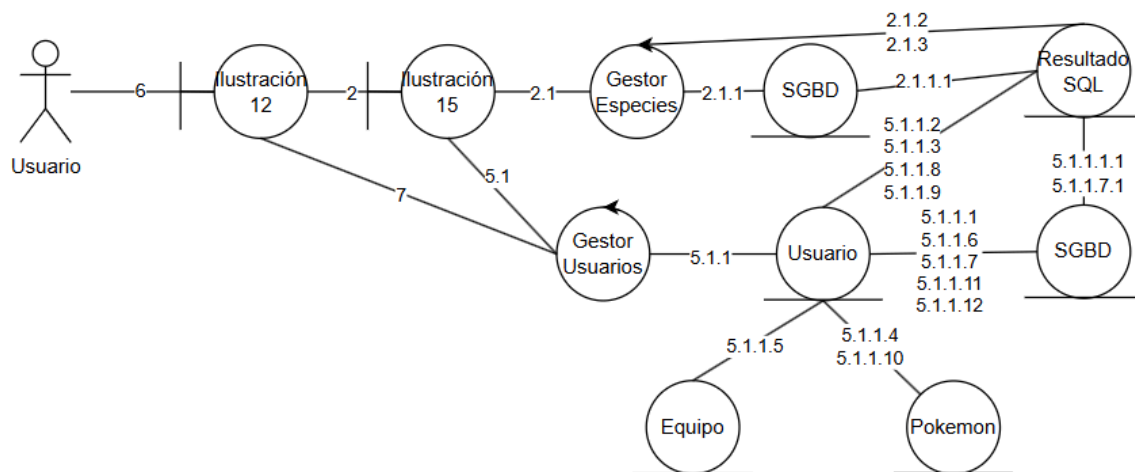
1. Desde la interfaz de visualización de un equipo, hacer click en el botón "Crear nuevo equipo" y luego en el botón "Sí"
2. crearEquipoNuevo(username, nombreEquipo):void
  - 2.1 crearEquipoNuevo(nombreEquipo):void
    - 2.1.1 nuevoEquipo = new Equipo(nombreEquipo):void
    - 2.1.2 addEquipo(nuevoEquipo):void
    - 2.1.3 execSQL(sql1):void

SQL1: INSERT INTO Equipo(nombre, fechaCreacion, username) VALUES %nombreequipo%, %fecha%, %username%

#### EXPLICACIÓN:

- Crear un equipo nuevo, accediendo al gestor de usuarios, y dándole un nombre de usuario y un nombre para el nuevo equipo.
- Buscar al usuario y acceder al mismo, pasándole el nombre del nuevo equipo.
- Crear el nuevo equipo, añadirlo a la lista de equipos del usuario, y acceder a la base de datos pasándole los datos del equipo. El ID se generará automáticamente en la BD, la fecha también se generará automáticamente.

#### 5.2.4 Añadir un Pokémon a un equipo



*Ilustración 76 - Diagrama Comunicación Añadir Pokémon*

1. Desde la interfaz de visualización de un equipo, hacer click en el botón "Editar" y luego en el botón "Sí"
2. new Ilustracion15():void
  - 2.1 mostrarListaEspecies():void
    - 2.1.1 execSQL(sql3)
      - 2.1.1.1 new ResultadoSQL()
    - 2.1.2 next():bool
    - 2.1.3 getJSON():JSON3
3. Hacer click en una ranura del equipo Pokémon
4. Hacer click en una de las especies de la lista
5. Hacer click sobre el botón "Añadir" y luego sobre "Guardar"
  - 5.1 addPokemonAEquipo(username, idEquipo, nombreEspecie, numRanura)



```

5.1.1 addPokemonAEquipo(idEquipo, nombreEspecie, numRanura)
    5.1.1.1 execSQL(sql1)
        5.1.1.1.1 new ResultadoSQL()
    5.1.1.2 next():bool
    5.1.1.3 getJSON():JSON1
    5.1.1.4 nuevoPokemon = new Pokemon(nombrePokemon,
        nombreEspecie, ataque, ataqueEsp, defensa,
        defensaEsp, velocidad, vida, listaAtaques)
    5.1.1.5 addPokemon(nuevoPokemon, numRanura):void
    5.1.1.6 execSQL(sql2)
    5.1.1.7 execSQL(sql4)
        5.1.1.7.1 new ResultadoSQL()
    5.1.1.8 next():bool
    5.1.1.9 getIDPokemon():Int
    5.1.1.10 cambiarID(idPokemon):void
    5.1.1.11 execSQL(sql5) ← por cada ataque
    5.1.1.12 execSQL(sql6)

```

```
6. new Ilustracion12():void
```

```
7. mostrarEquipo(username,idequipo):void
```

```
JSON1: {ataque:Int, ataqueespecial:Int, defensa:Int, defensaespecial:Int, velocidad:Int, vida:Int,
    [{NombreAtaque:String},{}]}
```

```
JSON3: [{NombreEspecie:String, foto:String},{}]}
```

```
SQL1: SELECT e.ataque, e.ataqueespecial, e.defensa, e.defensaespecial, e.velocidad, e.vida,
a.NombreAtaque
```

```
FROM Especie AS e, Ataque AS a, Especie_Ataque AS ea
```

```
WHERE e.NombreEspecie = %nombreEspecie% AND e.NombreEspecie = ea.NombreEspecie AND
a.NombreAtaque = ea.NombreAtaque
```

```
SQL2: INSERT INTO Pokemon(Nombre, Ataque, AtqueEsp, Def, DefEsp, Vel, Vida,
NombreEspecie) VALUES
```

```
%nombrePokemon%, %ataque%, %ataqueEsp%, %defensa%, %defensaEsp%, %velocidad%,
%vida%, %nombreEspecie%
```

```
SQL3: SELECT NombreEspecie, foto FROM Especie
```

```
SQL4: SELECT IDPokemon FROM Pokemon AS p WHERE p.ataque = %ataque% AND
p.ataqueEsp = %ataqueEsp% AND p.Def = %defensa% AND p.DefEsp = %defensaEsp% AND p.vel
= %velocidad% AND p.vida = %vida% AND p.NombreEspecie = %nombreEspecie%
```

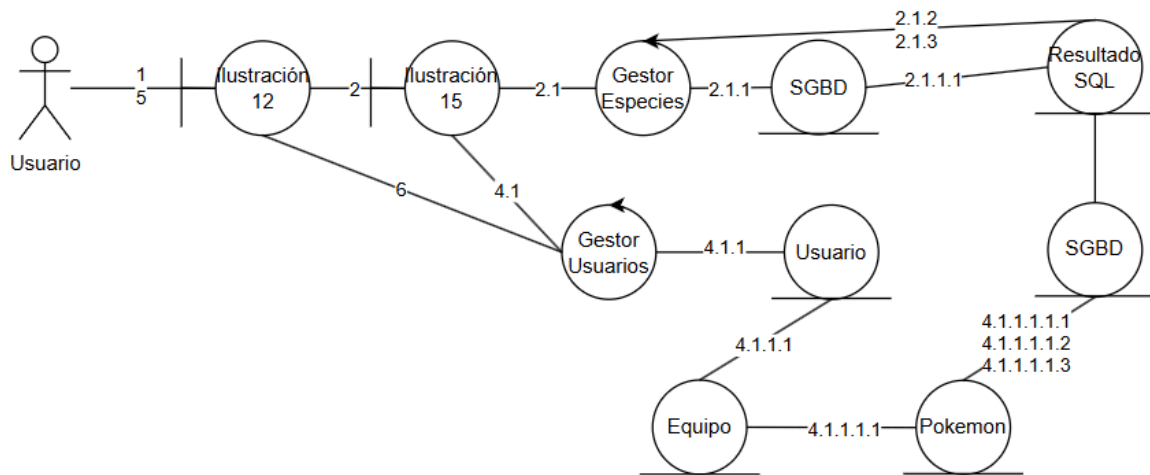
```
SQL5: INSERT INTO Pokemon_Ataque(IDPokemon, NombreAtaque) VALUES %idPokemon%,
%nombreAtaque%
```

SQL6: INSERT INTO Equipo\_Pokemon(IDPokemon, IDEquipo) VALUES %idPokemon%, %idEquipo%

#### EXPLICACIÓN:

- PARTE DE LA FUNCIONALIDAD “EDITAR UN EQUIPO”. La idea es que se guardan las ejecuciones que se deben hacer, y al dar click sobre “Guardar” se ejecutan. Si se sale de la interfaz sin guardar, se descartan los cambios y se borran las ejecuciones.
- Abrir la interfaz de edición de equipos, en la que se muestra una lista de todas las especies Pokémon entre las que escoger, y sus respectivas fotos. Para ello, entrar a la base de datos y obtener un JSON con una lista de nombres y fotos de las especies.
- Para añadir un Pokémon, seleccionar una ranura del equipo, y una especie de Pokémon de la lista de especies, en la interfaz gráfica. Si se selecciona el botón añadir, cambia la foto y nombre del Pokémon que se encontraba en la ranura seleccionada, por los de la especie seleccionada, pero solo gráficamente. Cuando se hace click en el botón guardar, se ejecutan los cambios, si no, se cancelan.
- Para guardar los cambios, con el nombre del usuario, el ID del equipo, el nombre de la especie seleccionada, y la ranura del equipo seleccionada, acceder al gestor de usuarios y buscar al usuario.
- Una vez encontrado el usuario, acceder a él con el nombre de la especie seleccionada, el ID del equipo, y el número de la ranura seleccionada. Pedir a la base de datos las estadísticas de la especie, y una lista de los nombres de sus posibles ataques.
- A partir de las estadísticas de la especie, generar unas estadísticas aleatorias para un Pokémon individual, y escoger 4 ataques aleatorios que pueda aprender la especie. Crear un nuevo Pokémon con esta información.
- Acceder al equipo en cuestión y sustituir el nuevo Pokémon por el que haya en la ranura seleccionada.
- Acceder a la base de datos y guardar las estadísticas y ataques del nuevo Pokémon. En la base de datos se genera un ID para el Pokémon, para que sea distinto del resto.
- Acceder a la base de datos para obtener el ID generado. Para ello, se busca el id del Pokémon con las mismas estadísticas generadas aleatoriamente y la misma especie. Técnicamente podría no ser el mismo Pokémon, pero sería muy raro. (se podría evitar añadiendo un gestor de Pokémon).
- Una vez obtenido el ID, se le añade al Pokémon. Además, por cada ataque, añadir una relación de Pokémon\_Ataque a la BD, y también añadir una relación de Equipo\_Pokémon (hace falta el ID del pokémon para estas relaciones).
- Por último, se regresa a la interfaz de visualización del equipo, y mostrar el equipo dado el ID del equipo y el nombre del usuario (método explicado en anteriores funcionalidades).

## 5.2.5 Eliminar un Pokémon de un equipo.



*Ilustración 77 - Diagrama Comunicación Eliminar Pokémon*

1. Desde la interfaz de visualización de un equipo, hacer click en el botón "Editar" y luego en el botón "Sí"

2. new Ilustracion15():void

2.1 mostrarListaEspecies():void

2.1.1 execSQL(sql3)

2.1.1.1 new ResultadoSQL()

2.1.2 next():bool

2.1.3 getJSON():JSON3

3. Hacer click en una ranura del equipo Pokémon

4. Hacer click sobre el botón "Expulsar" y luego sobre "Guardar"

4.1 expulsarPokemonDeEquipo(username, idEquipo, numRanura):void

4.1.1 expulsarPokemonDeEquipo(idEquipo, nombreEspecie,  
numRanura):void

4.1.1.1 expulsarPokémon(numRanura):void

4.1.1.1.1 expulsar(idEquipo):void

4.1.1.1.1.1 execSQL(sql1):void <--por cada ataque

4.1.1.1.1.2 execSQL(sql2):void

4.1.1.1.1.3 execSQL(sql3):void

5. new Ilustracion12():void

6. mostrarEquipo(username,idequipo):void

SQL1: DELETE FROM Pokemon\_Ataque WHERE IDPokemon = %idPokemon% AND NombreAtaque = %nombreAtaque%

SQL2: DELETE FROM Equipo\_Pokemon WHERE IDPokemon = %idPokemon% AND IDEquipo = %idEquipo%

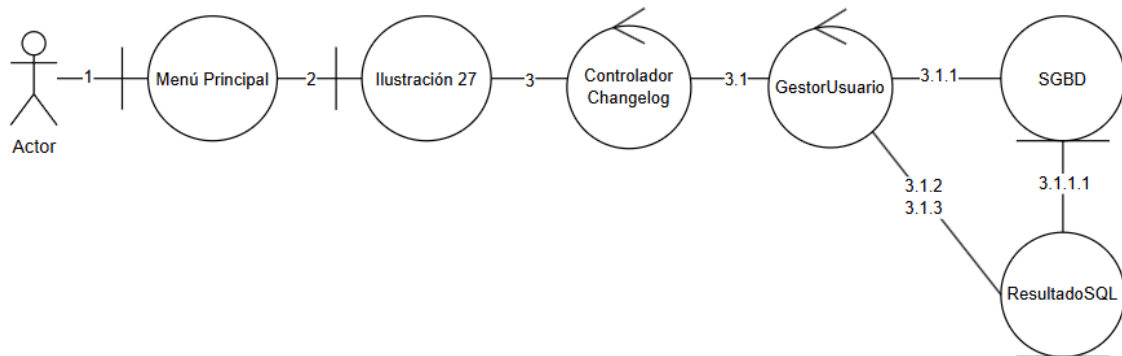
SQL3: DELETE FROM Pokemon WHERE IDPokemon = %idPokemon%

### EXPLICACIÓN

- PARTE DE LA FUNCIONALIDAD “EDITAR UN EQUIPO”. La idea es que se guardan las ejecuciones que se deben hacer, y al dar click sobre “Guardar” se ejecutan. Si se sale de la interfaz sin guardar, se descartan los cambios y se borran las ejecuciones.
- Abrir la interfaz de edición de equipos, en la que se muestra una lista de todas las especies Pokémon entre las que escoger, y sus respectivas fotos. Para ello, entrar a la base de datos y obtener un JSON con una lista de nombres y fotos de las especies.
- Seleccionar una ranura del equipo en la interfaz gráfica. Si se selecciona el botón añadir, borra la foto y nombre del Pokémon que se encontraba en la ranura seleccionada, pero solo gráficamente. Cuando se hace click en el botón guardar, se ejecutan los cambios, si no, se cancelan.
- Para expulsar el pokémon, se accede al gestor de usuarios, con un ID de equipo, un nombre de usuario, y un número de ranura (del 1 al 6) que ha sido seleccionado anteriormente. Se encuentra el usuario, y se accede al usuario con el ID del equipo y el número de ranura. Una vez encontrado el equipo, se accede al mismo con el número de ranura y se le indica que modifique el Pokémon que se encuentra ahí.
- Dado el ID del equipo se accede al Pokémon, y se recorren los nombres de sus ataques. Por cada nombre de ataque, se accede a la base de datos y se elimina la relación entre el Pokémon y el ataque (el ataque sigue estando en el gestor de ataques).
- Para eliminar un Pokémon también se usa el id del Pokémon y del equipo. Se accede a la base de datos para eliminar la relación entre ellos dos. Luego se vuelve a acceder a la BD para eliminar el Pokémon en si.
- Por último, se regresa a la interfaz de visualización del equipo, y mostrar el equipo dado el ID del equipo y el nombre del usuario (método explicado en anteriores funcionalidades).

## 5.3 CHANGELOG

### 5.3.1 Ver notificaciones



*Ilustración 78 - Diagrama Comunicación Ver notificaciones*

1. El usuario selecciona la opción "Changelog"
2. `new vistaChangelog()`
3. `obtenerNotificaciones(username): JSON1`
  - 3.1. `obtenerNotificaciones(username): JSON1`
    - 3.1.1. `execSQL (sql1): ResultadoSQL`
      - 3.1.1.1. `new ResultadoSQL()`
    - 3.1.2. `next() : bool`
    - 3.1.3. `getJSON(): JSON1`

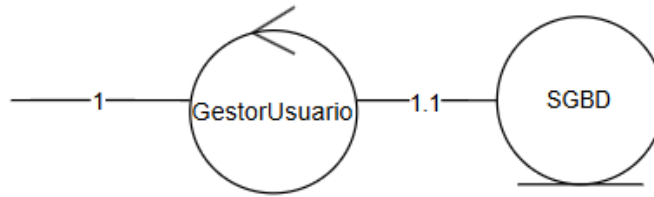
JSON1: { [ { username: string, fecha: date, tipo: string, texto: string } , ...] }

sql1: "SELECT n.username, n.fecha, n.tipo, n.texto  
FROM NOTIFICACIONES n  
INNER JOIN SEGUIDORES s ON n.username = s.seguido  
WHERE s.seguidor = %username%  
ORDER BY n.fecha DESC  
LIMIT 20"

#### EXPLICACIÓN:

- Al seleccionar la opción de ver el Changelog, se crea una nueva `VistaChangelog()`, que corresponde a la ilustración 27. Conociendo el nombre del usuario que tiene sesión iniciada, se obtiene de la base de datos las notificaciones generadas por los usuarios seguidos.
- Se devuelven las últimas 20 notificaciones a la vista. Después, será la propia vista la que filtre las notificaciones en función de la selección del usuario.

### 5.3.2 Generar notificaciones



*Ilustración 79 - Diagrama Comunicación Generar Notificación*

1. generarEvento( usuario, tipo, texto)
  - 1.1. execSQL (sql2)

sql2: "INSERT INTO notificaciones (username, fecha, tipo, texto)  
VALUES (%username%, datetime('now'), %tipo%, %texto%) "

EXPLICACIÓN:

- Cuando se genere un evento (un usuario sigue a otro, crea un nuevo equipo o añade un Pokémon a un equipo), se llamará a este método de GestorUsuario para guardar las notificaciones en la base de datos.

## 5.4 LISTA COMPLETA DE POKÉMON Y BÚSQUEDAS CON FILTRO

Ver Lista de Pokemon:

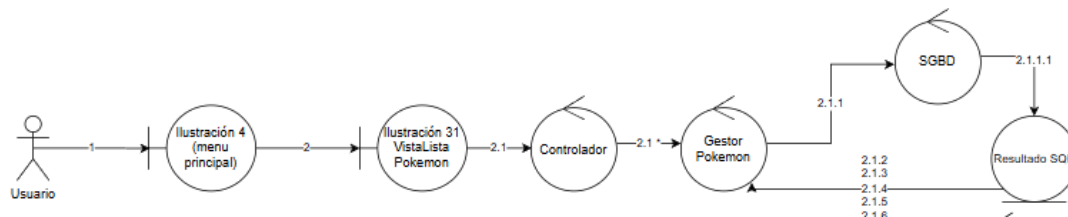


Ilustración 80 - Diagrama Comunicación Generar Notificación

Buscar Pokemon por nombre:

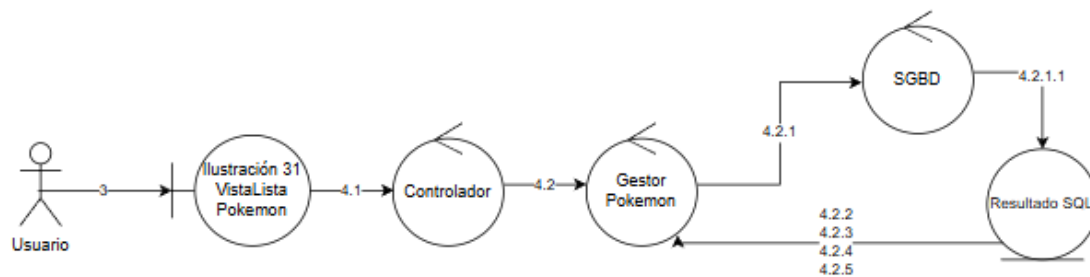


Ilustración 81 - Diagrama Comunicación Generar Notificación

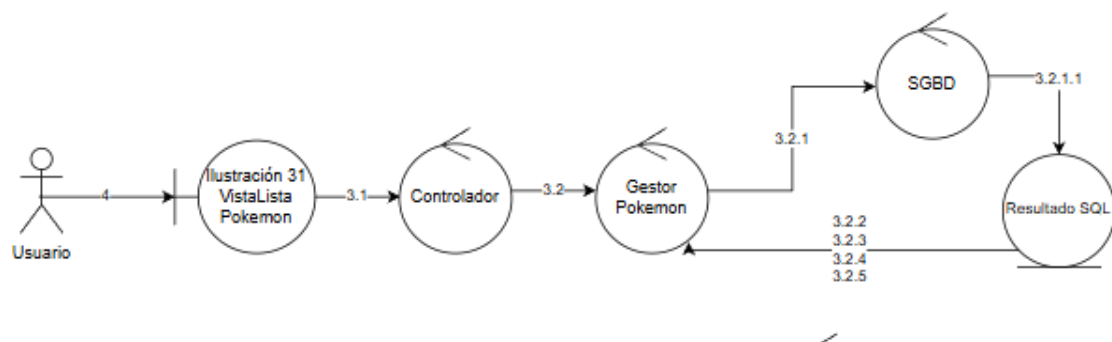
no me jodas

no es automatico?

bruh, pues lo hago yo si quieres, ya lo siento

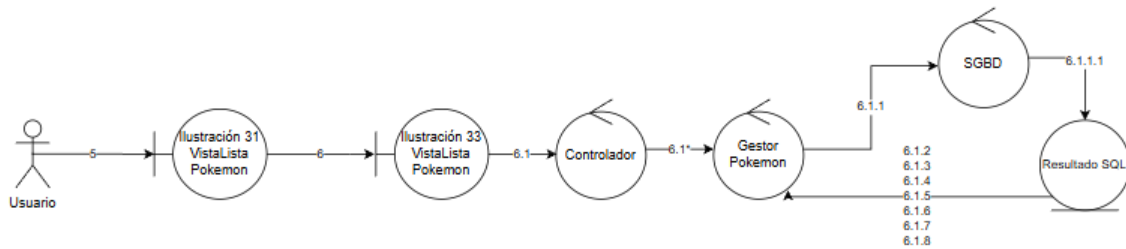
no, en google docs no se puede

Filtrar Pokemon por tipo:



## Ilustración 82 - Diagrama Comunicación Generar Notificación

Ver detalles de Pokemon:



## Ilustración 83 - Diagrama Comunicación Generar Notificación

1. [EL USUARIO PULSA POKÉDEX]
2. new VistaListaPokemon()
  - 2.1. cargarListaPokemon(filtro= " "): JSONLista
    - 2.1.1 execSQL(sql1): ResultadoSQL1
      - 2.1.1.1 new ResultadoSQL1()
    - 2.1.2 next()
    - 2.1.3 getVida()
    - 2.1.4 getDaño()
    - 2.1.5 getSprite()
    - 2.1.6 getNombre()
3. [EL USUARIO PULSA BUSCAR]
  - 3.1 aplicarFiltroBusqueda(nombrePokemon)
  - 3.2 cargarListaPokemon(filtro= nombrePokemon): JSONLista
    - 3.2.1 execSQL(sql3): ResultadoSQL3
      - 3.2.1.1 new ResultadoSQL3()
    - 3.2.2 next()
    - 3.2.3 getVida()
    - 3.2.4 getDaño()
    - 3.2.5 getSprite()
4. [EL USUARIO SELLECCIONA UN FILTRO]
  - 4.1 aplicarFiltroBusqueda(tipoPokemon)
  - 4.2 cargarListaPokemon(filtro= tipoPokemon): JSONLista
    - 4.2.1 execSQL(sql2): ResultadoSQL2



- 4.2.1.1 new ResultadoSQL2()
    - 4.2.2 next()
    - 4.2.3 getVida()
    - 4.2.4 getDaño()
    - 4.2.5 getSprite()
- 5. [EL USUARIO PULSA VER DETALLES EN UN POKEMON]
- 6. new VistaPokemon(id)
  - 6.1 cargarEstadisticasPokemon(id): JSONDetalles
    - 6.1.1 execSQL(sql4): ResultadoSQL4
      - 6.1.1.1 newResultadoSQL4()
    - 6.1.2 getNombre()
    - 6.1.3 getDescripcion()
    - 6.1.4 getTipo()
    - 6.1.5 getAtaque()
    - 6.1.6 getDefensa()
    - 6.1.7 getVelocidad()
    - 6.1.8 getMovimientos()

Estos serían las sentencias SQL necesarias para esta funcionalidad:

Para generar la lista completa:

```
sql1= ("SELECT nombre, daño, vida. sprite FROM listaPokemon")
```

Para buscar Pokemon:

```
sql2= ("SELECT nombre, daño, vida. sprite FROM listaPokemon WHERE name =" + nombrePokemon)
```

Para filtrar Pokemon:

```
sql3= ("SELECT nombre, daño, vida. sprite FROM listaPokemon WHERE type=" + tipoPokemon)
```

Para ver detalles de Pokemon:

```
sql4= ("SELECT nombre, daño, vida. sprite, descripcion, tipo, velocidad, movimientos FROM listaPokemon WHERE id =" + idPokemon)
```

```
JSONLista = { [ { id : int, nombre : string, imagenSprite : string, vida : int, daño : int} ] }
```

```
JSONDetalles = { [ { id : int, nombre : string, descripción : string, tipo : [string, string], ataque : int, defensa: int, velocidad: int, movimientos : [string, ...] } ] }
```

## EXPLICACIÓN:

### 1. Carga Inicial de la Lista (Pasos 1 a 2.6)

La funcionalidad comienza cuando el usuario selecciona la opción "Ver Lista Pokémon" en el Menú Principal (Ilustración 4) (Paso 1). Esta acción inicia el flujo creando la nueva Vista de Lista (VistaListaPokemon / Ilustración 31).

Una vez que la nueva Vista está lista, solicita al (Controlador) que cargue la lista, pasando un filtro vacío inicialmente (Paso 2.1) (que al principio será vacía, es decir se pondrán todos los Pokemon sin filtrado).

El GestorPokemon se comunica con el SGBD para obtener la lista de datos básicos (nombre, ID, imagen) a través del método `getListaPokemonDesdeBD` (Paso 3). El SGBD ejecuta la consulta implícita y devuelve el resultado (JSONLista) al Gestor (Paso 3.1).

Luego el GestorPokemon:

Almacena la lista completa en memoria (`almacenarListaCompleta`) (Paso 3.2). Esta lista se mantendrá viva para futuras búsquedas, para así no tener que estar todo el rato buscando en la base de datos. Muestra la lista (`mostrarLista`) en la Ilustración 31, renderizando los Pokémon básicos en la interfaz gráfica (Paso 3.3).

### 2. Búsqueda y filtrado (Pasos 3 y 4)

Esta secuencia se activa cada vez que el usuario utiliza la barra de búsqueda o aplica un filtro en la Ilustración 31.

La Vista captura el criterio introducido por el usuario y lo pasa al `GestorListaPokemon` mediante el mensaje `aplicarFiltro`.

El `GestorListaPokemon` ejecuta el método `filtrarListaCargada`. En este punto, el Gestor trabaja únicamente con la lista que ya tiene en la memoria, reduciendo la lista original a una `listaFiltrada` que coincide con el criterio (borrando Pokemons que no cumplen los filtros).

Finalmente, el Gestor notifica a la Vista (`actualizarLista`) para que se redibuje, mostrando únicamente los Pokémon de la `listaFiltrada`.

### 3. Visualización de los Detalles (Pasos 5 en adelante)

Esta secuencia se ejecuta cuando el usuario hace clic en un Pokémon de la lista para ver su información completa.

La Vista (Ilustración 31) notifica al `GestorListaPokemon` sobre la selección, pasándole el identificador único (`idPokemon`) del Pokemon pulsado.

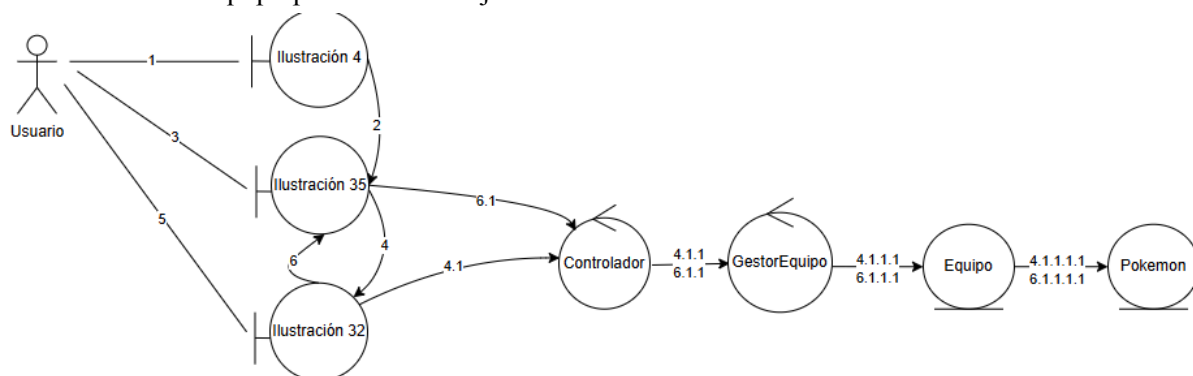
El `GestorListaPokemon` no tiene los detalles de cada Pokemon, por lo que lo hace el `GestorPokemon`.

El `GestorPokemon` accede directamente al SGBD solicitando todos los datos detallados (estadísticas, movimientos, habilidades, etc.) mediante `getEstadisticas`. El SGBD devuelve el objeto completo con todos los atributos (JSONDetalles) al `GestorPokemon`.

Finalmente, el GestorPokemon toma los datos recibidos y crea la nueva Vista de Especificaciones (Ilustración 33), pasándole el objeto JSONDetalles para que se renderice en pantalla.

## 5.5 CHATBOT

### 5.5.1 Seleccionar equipo para obtener mejor

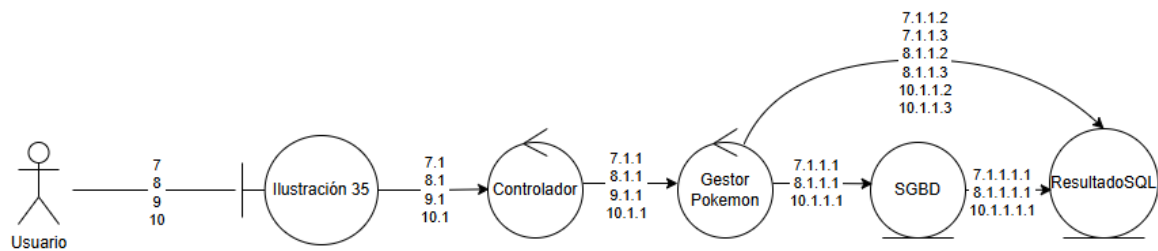


*Ilustración 84 - Diagrama Comunicación Seleccionar equipo*

1. El usuario pulsa el botón "ChatBot"
2. newFigura35()
3. El usuario pulsa el botón "Seleccionar Equipo"
4. newFigura32()
  - 4.1 obtenerEquiposNombreySprite(usuario): JSON1
    - 4.1.1. obtenerEquiposNombreySprite(usuario): JSON1
      - 4.1.1.1 obtenerPokemonNombreySprite(): JSON2
        - 4.1.1.1.1 obtenerNombreySprite (): JSON3
5. El usuario pulsa el botón "Seleccionar Equipo"
6. newFigura35(nombreEquipo)
  - 6.1 obtenerMejorPokemon(nombreEquipo): JSON3
    - 6.1.1 obtenerMejorPokemon(nombreEquipo): JSON3
      - 6.1.1.1 obtenerMejorPokemon(): JSON3

#### 6.1.1.1 obtenerMediaPokemon(): JSON4

#### 5.5.2 Seleccionar Pokémon para hacer diferentes consultas



*Ilustración 85 - Diagrama Comunicación Seleccionar pokémon*

7. El usuario pulsa el botón "Seleccionar Pokémon" y en el desplegable escribe el nombre de un Pokémon correcto:

7.1 obtenerPokemon(string): JSON3

7.1.1 obtenerPokemonEntero(string): JSON3

7.1.1.1 execSQL(SQL1): ResultadoSQL

7.1.1.1.1 newResultadoSQL()

7.1.1.2 next()

7.1.1.3 obtenerPokemonJSON(): JSON5

8. El usuario pulsa el botón "Ver cadena evolutiva"

8.1 obtenerCadenaEvolutiva(string): JSON6

8.1.1 obtenerCadenaEvolutiva(string): JSON6

8.1.1.1 execSQL(SQL2): ResultadoSQL

8.1.1.1.1 newResultadoSQL()

8.1.1.2 next()

#### 8.1.1.3 obtenerFotoEvolucionesJSON(): JSON7

9. El usuario pulsa el botón "Ver estadísticas"

9.1 obtenerEstadísticas(): JSON5

9.1.1 obtenerEstadísticas(): JSON5

10. El usuario pulsa el botón "Ver rivales"

10.1 obtenerRivales(string): JSON6

10.1.1 obtenerRivales(string): JSON6

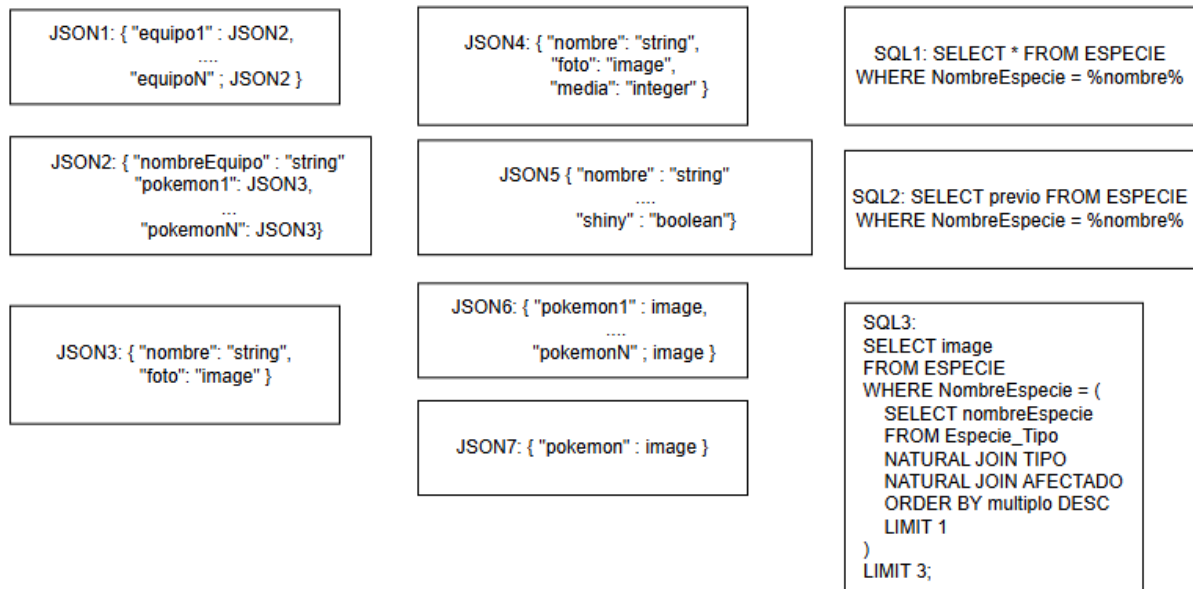
10.1.1.1 execSQL(SQL3): ResultadoSQL

10.1.1.1.1 newResultadoSQL()

10.1.1.2 next()

10.1.1.3 obtenerFotoJSON(): JSON7

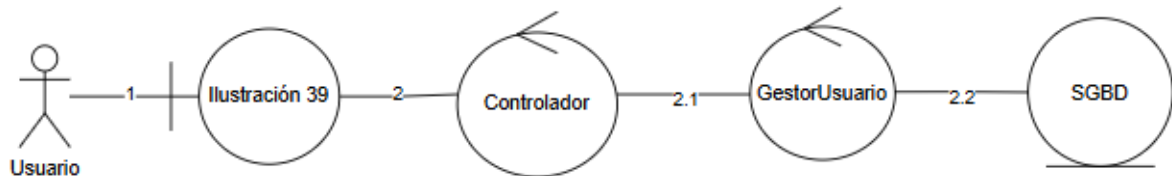
#### 5.5.3 JSON y consultas SQL





## 5.6 COMPARTIR A TRAVÉS DE TELEGRAM

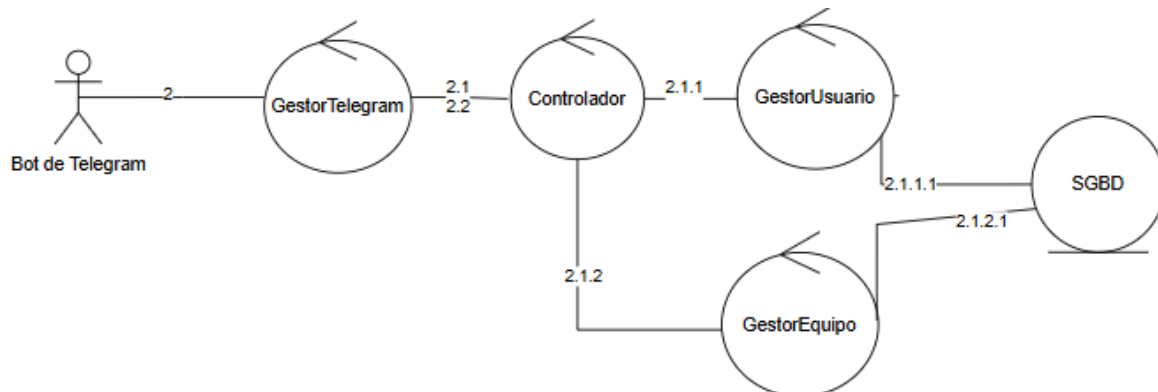
### Vincular telegram con usuario



*Ilustración 86 - Diagrama Comunicación Vincular telegram*

1. Un usuario ya identificado pincha en "Vincular cuenta", ingresa un nombre de telegram (que debería ser su username en Telegram) y pincha en el botón "continuar"
- 2.vincularUsuario(username, telegramUsername)
  - 2.1 vincularUsuario(username,telegramUsername)
  - 2.2vincularUsuario(username,telegramUsername)

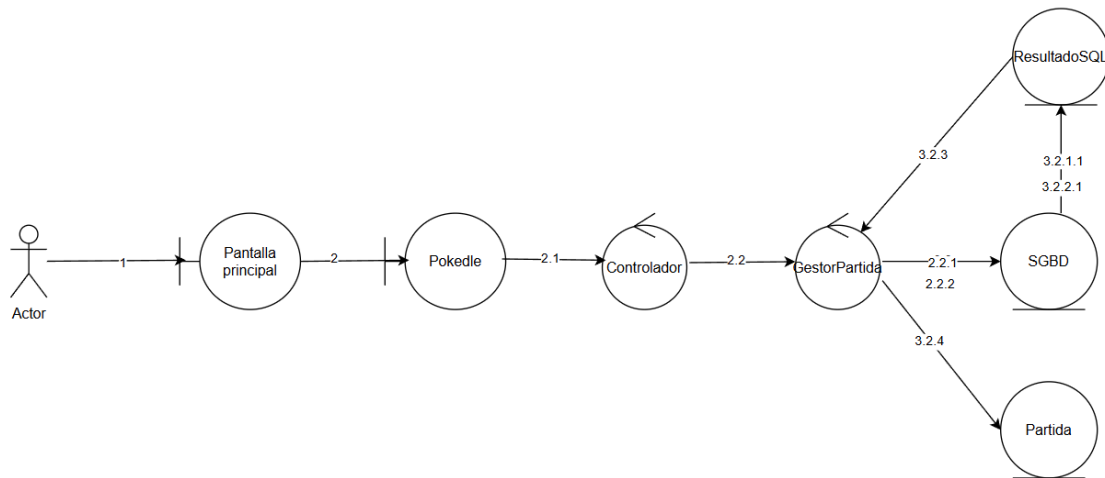
### Compartir equipo telegram



*Ilustración 87 - Diagrama Comunicación Compartir equipo*

- 1.El bot de telegram lee su commando característico
- 2-commando1(telegramUsername)
  - 2.1.1-getUserByTelegram(tgUser): string
    - 2.1.1.1-getUserByTelegram(tgUser): string
  - 2.1.2-getEquipo(username): JSON
    - 2.1.2.1-getEquipo(username): JSON

## 5.7 POKEDLE



*Ilustración 88 - Diagrama Comunicación Pokedle*

Pinchar en Pokedle

1: Usuario pulsa en Pokedle

2: new Pokedle()

2.1 iniciarPartida(): JSON

2.2. iniciarPartida(): JSON

2.2.1 execSQL(sql1):ResultadoSQL()

2.2.1.1 new ResultadoSQL()

2.2.2. next():bool

2.2.3 execSQL(sql2): ResultadoSQL

2.2.2.1 new ResultadoSQL()

2.2.4. next():bool

2.2.5 getJSON():JSON

2.2.6 new Partida(JSON)

sql1: count

sql2: select from



## EXPLICACIÓN:

El flujo comienza cuando el usuario pulsa “Jugar Pokedle”, lo que llama al método `IniciarPartida()`. Este método selecciona un Pokémon objetivo mediante `getPokemonAleatorioAPI()`, obteniendo sus datos directamente desde la API y almacenándolo como `pokemonObjetivo`.

Cada vez que el usuario introduce un nombre de Pokémon y pulsa ENTER, el sistema primero ejecuta `validarIntento(nombreUsuario)` para comprobar que no haya sido introducido antes. Si ya existe, se muestra un error; si no, se añade a la lista de intentos.

A continuación, se llama a `ComprobarAdivinanza(nombreUsuario)`, que obtiene los datos del Pokémon introducido por el usuario a través de `getPokemonAPI(nombreUsuario)` y lo convierte en un objeto `Pokemon`. Este objeto se compara con el Pokémon objetivo mediante distintos métodos (`compararTipos()`, `compararGeneracion()`, `compararAltura()`, `compararPeso()` y `compararColor()`), obteniendo como resultado qué atributos coinciden.

El intento y sus resultados se almacenan con `actualizarHistorialIntentos()` y se comprueba si el usuario ha ganado mediante `comprobarVictoria()`.

Finalmente, el resultado se muestra en la interfaz, coloreando las casillas según las coincidencias. El usuario puede seguir intentándolo, rendirse (mostrando el Pokémon correcto) o iniciar una nueva partida.

## 6. DIAGRAMA DE SECUENCIA

### FUNCIONALIDAD 1: Gestión de usuarios

#### REGISTRARSE

Registrarse

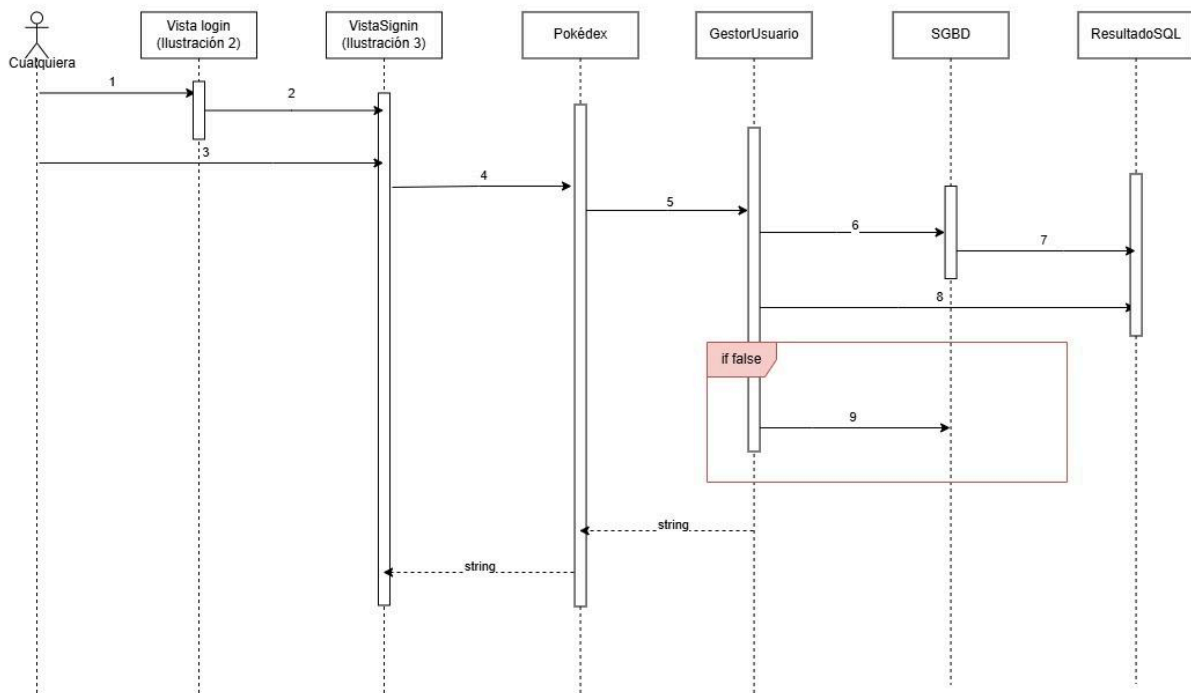


Ilustración 89 - Diagrama Secuencia Registrarse

1. En la página de inicio de sesión, pulsa "Registrarse"
2. new Ilustración3()
3. Pulsa "Registrarse"
4. registrarse(username, email, password, confirm\_password): string
5. crearCuenta(username, email, password, confirm\_password): string
6. execSQL(sql1): ResultadoSQL
7. new ResultadoSQL()
8. next(): bool

[SI FALSE]

9. execSQL(sql2)

sql1: "SELECT username FROM USUARIO WHERE username=%user%"

sql2: "INSERT INTO USUARIO(username, email, contraseña, esAdmin, aprobado) VALUES (%username%, %email% %contraseña%, 0, 0)"

## INICIAR SESIÓN

Iniciar sesión

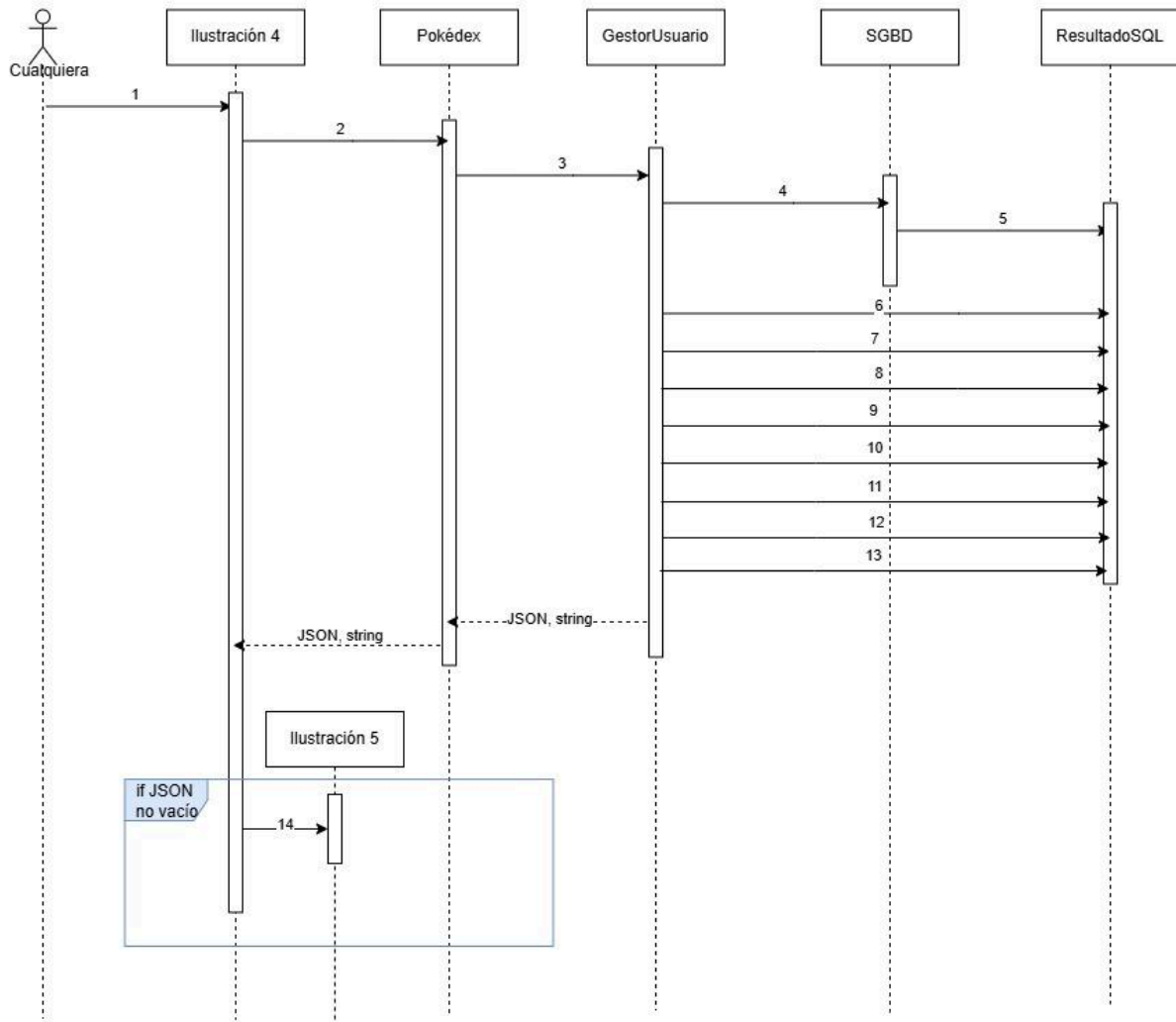


Ilustración 90 - Diagrama Secuencia Iniciar Sesión

1. Introduce los credenciales y pulsa "Iniciar sesión"
2. iniciarSesión(username, password): Tuple[JSON, string]
3. validarUsuario(user, password): Tuple[JSON, string]
4. execSQL(sql): ResultadoSQL
5. new ResultadoSQL()
6. next(): bool
7. getString("username"): string
8. getString("email"): string
9. getString("foto"): string
10. getString("contrasena"): string
11. getBool("esAdmin"): bool
12. getBool("aprobado"): bool

13. close()

[si devuelve un None]

Falla el inicio de sesión y muestra el mensaje de error por pantalla

[si JSON no está vacío]

14. new Ilustración5(username)

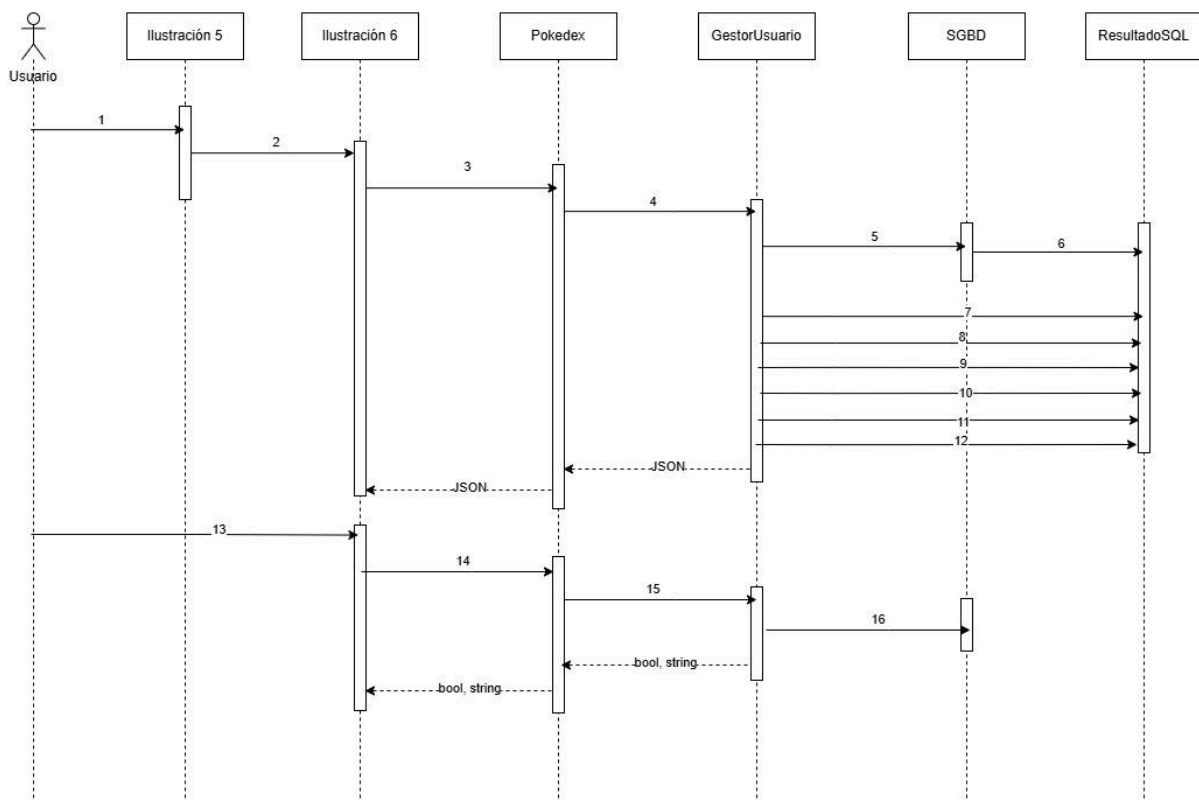
```
sql="SELECT * FROM USUARIO WHERE username=%user% AND contraseña=%pass% AND aprobado=1"
```

```
JSON = {username: string, email: string, contrasena: string, foto: string, esAdmin: bool, aprobado: bool}
```

string: es un mensaje que muestra el resultado del inicio de sesión (como, usuario o contraseña incorrectos, cuenta pendiente de aprobación o inicio de sesión exitoso)

## ACTUALIZAR DATOS

Actualizar datos



*Ilustración 91 - Diagrama Secuencia Actualizar Datos*

1. Pulsa el icono de perfil

2. new Ilustración6(username)

3. buscarUsuarioLogueado(username): JSON

4. buscarUsuarioLogueado(username): JSON

5. `execSQL(sql1): ResultadoSQL`
6. `new ResultadoSQL()`
7. `next()`
8. `getString("username"): string`
9. `getString("email"): string`
10. `getString("contrasena"): string`
11. `getString("foto"): string`
12. `close()`
13. Introduce nuevos cambios en los campos deseados y pulsa "Guardar cambios"
14. `actualizarDatos(username, email, foto, password, confirm_pass): Tuple[bool, string]`
15. `actualizarDatos(username, email, foto, password, confirm_pass): Tuple[bool, string]`
16. `execSQL(sql2)`

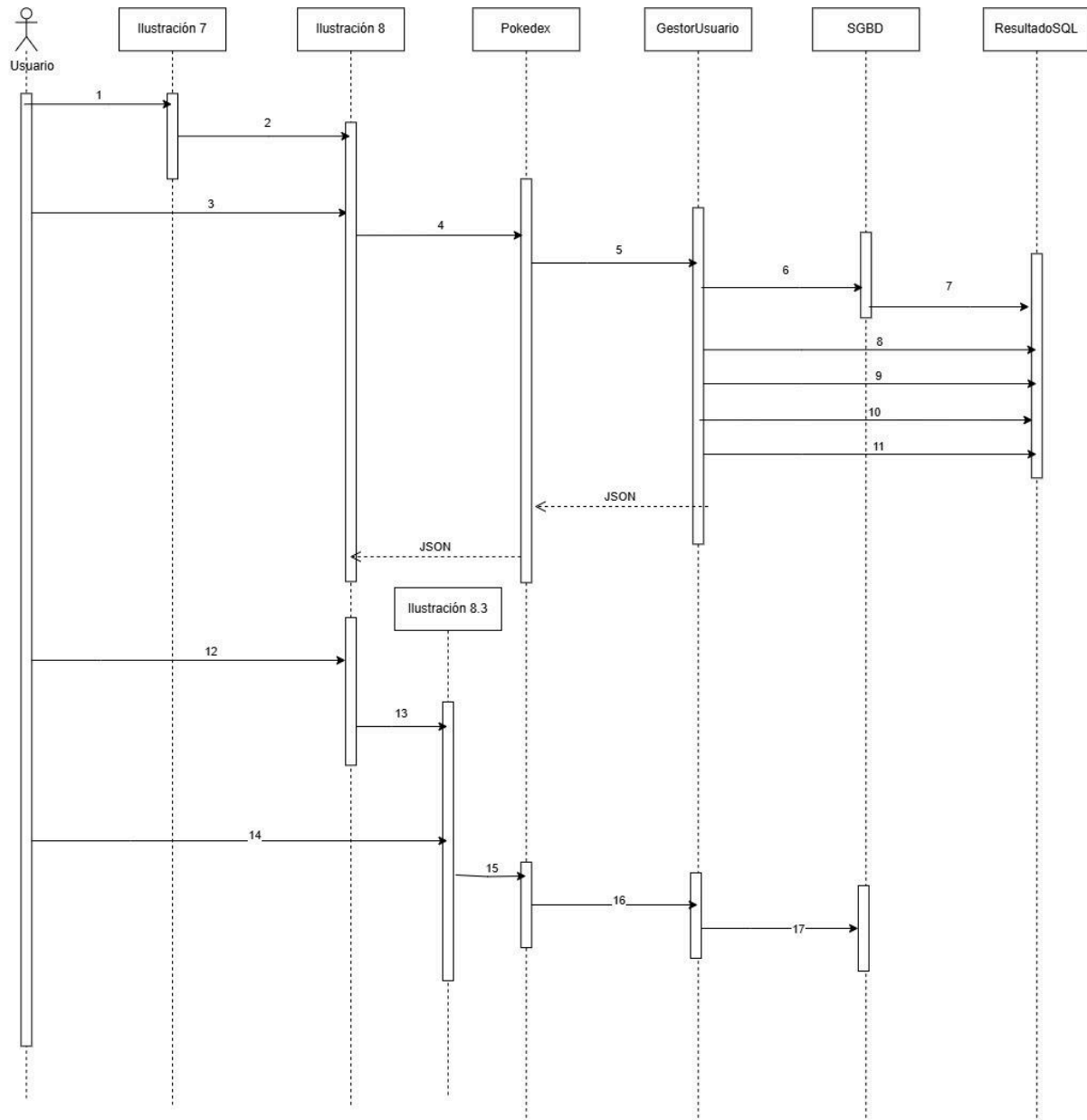
`sql1: SELECT username, email, contrasena, foto FROM USUARIO WHERE  
username=%username%`

`sql2: "UPDATE USUARIO SET email=%email%, contraseña=%contraseña%, foto=%foto%  
WHERE username=%username%"`

`JSON = {username: string, email: string, contrasena: string, foto: string}`

## GESTIONAR AMIGOS

### Listar usuarios seguidos y dejar de seguir



*Ilustración 92 - Diagrama Secuencia Listar seguidos*

1. En la página principal, pulsa en "Gestión de usuarios"
2. `new Ilustración8(username)`
3. Pulsar en "Ver seguidos"
4. `get_following(username): JSON`
5. `get_following(username): JSON`
6. `execSQL(sql1): ResultadoSQL`
7. `new ResultadoSQL()`
8. `next()`
9. `getString("username")`
10. `getString("foto")`

11. close()
- [Si pulsa algo]
12. Pulsa "Dejar de seguir" sobre un usuario
13. new Ilustración8.3(username)
14. Pulsa en "Aceptar"
15. dejarDeSeguir(seguidor, seguido): bool
16. dejarDeSeguir(seguidor, seguido): bool
17. execSQL(sql3)

sql1: "SELECT u.username, u.foto FROM USUARIO u INNER JOIN SEGUIDORES s ON u.username=s.seguido WHERE s.seguidor=%username%"

Si se quiere dejar de seguir a un usuario:

sql3: "DELETE FROM SEGUIDORES WHERE seguidor=%seguidor% AND seguido=%seguido%"

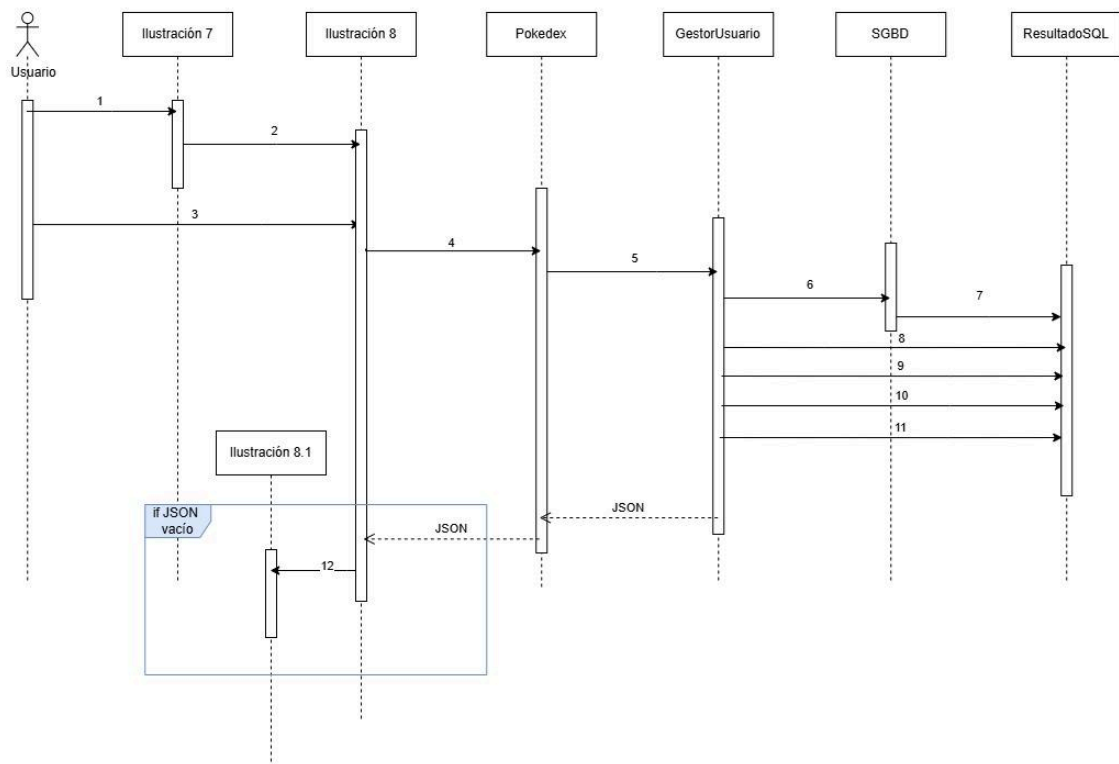
Lo mismo para ver la lista de seguidores del usuario (los pasos del punto 3), varía un poco el SELECT y el método es get\_followers(username).

sql2: "SELECT u.username, u.foto FROM USUARIO u INNER JOIN SEGUIDORES s ON u.username=s.seguidor WHERE s.seguido=%username%"

JSON: { [ {username: string, foto: string}, ... ] }

## Buscar usuario y seguirle

Buscar usuario



*Ilustración 93 - Diagrama Secuencia Buscar y seguir usuario*

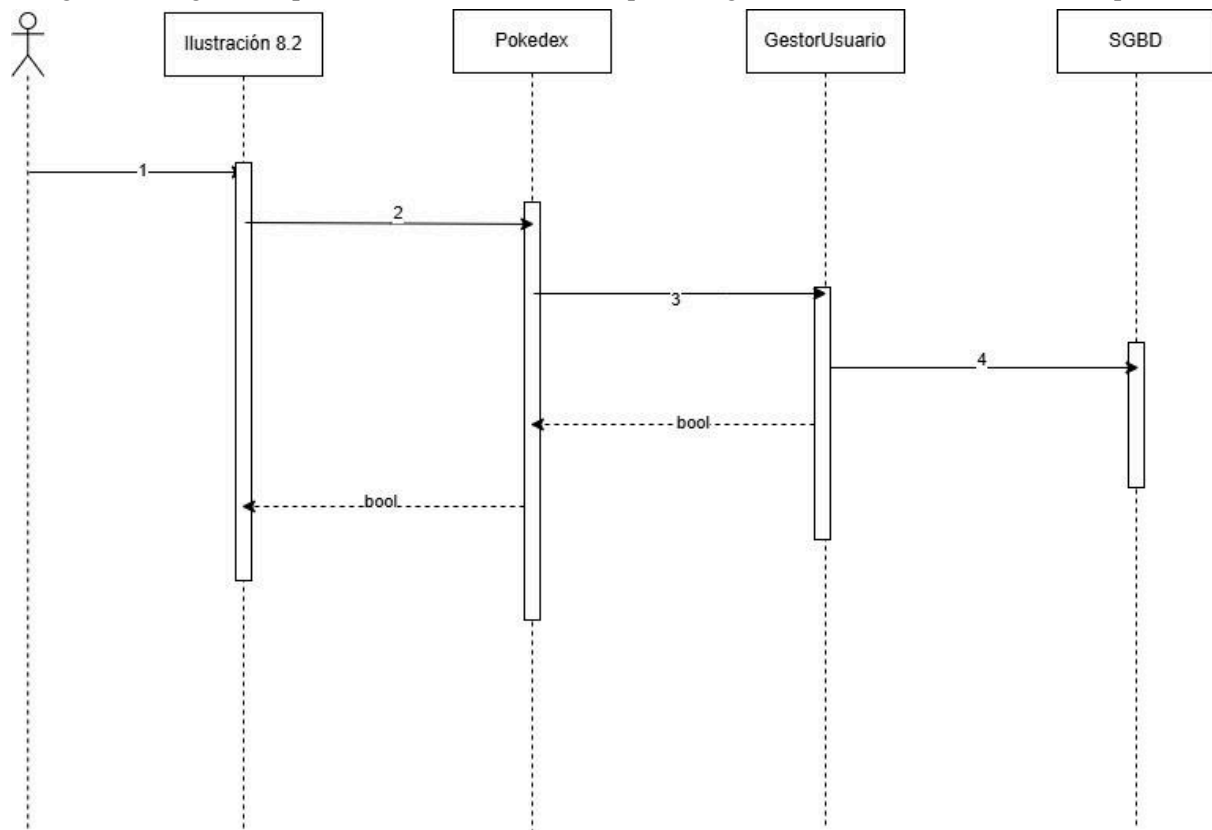
1. En la página principal, pulsa en "Gestión de usuarios"
2. new Ilustración8(username)
3. Introduce un nombre y pulsa en el icono de la lupa
4. buscarPorUsername(usuarioActual, usernameABuscar): JSON
5. buscarPorUsername(usuarioActual, usernameABuscar): JSON
6. execSQL(sql1): ResultadoSQL
7. new ResultadoSQL()
8. next(): bool
9. getString("username"): string
10. getString("foto"): string
11. close()

[Si JSON está vacío]

12. new Ilustración8.1()

sql1: "SELECT username, foto FROM USUARIO WHERE username LIKE %usernameABuscar%  
AND username != %usuarioActual% AND aprobado =1 AND esAdmin = 0 ORDER BY  
USERNAME"

El siguiente diagrama representa cuando un usuario quiere seguir a otro entrenador tras la búsqueda.



*Ilustración 94 - Diagrama Secuencia Seguir usuario*



[Si JSON no está vacío y pulsa algo]

1. Seleccionar usuario y pulsa "Seguir"
2. seguir(seguidor, seguido): bool
3. seguir(seguidor, seguido): bool
4. execSQL(sql2)

sql2: "INSERT INTO SEGUIDORES (seguidor, seguido) VALUES(%seguidor%, %seguido%)"

## GESTIONAR CUENTAS

### Obtener cuentas pendientes de aprobación y cuentas aprobadas

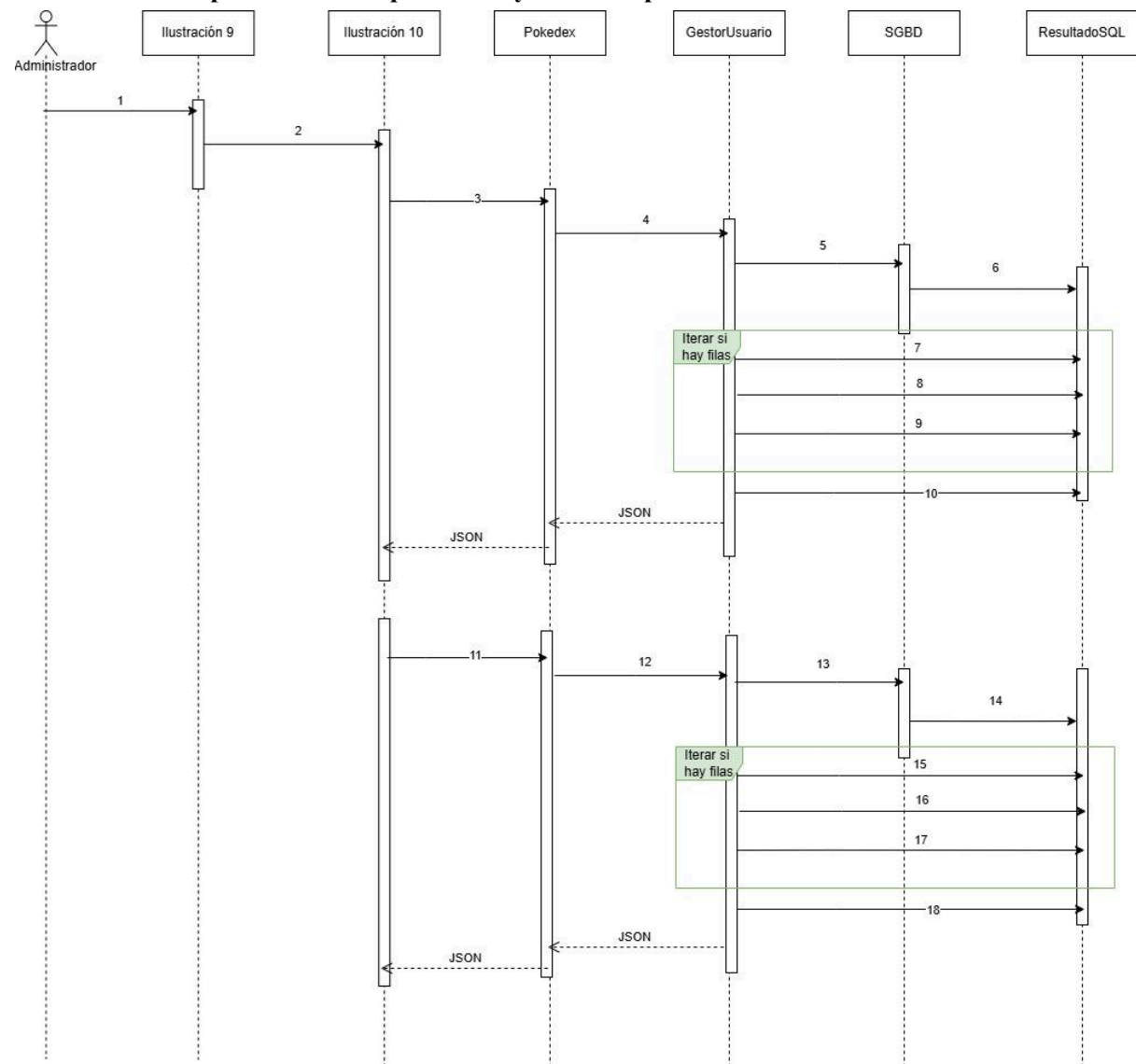


Ilustración 95 - Diagrama Secuencia Gestionar cuentas

1. En el menú principal, pulsa "Gestión de usuarios"
2. new Ilustración10()

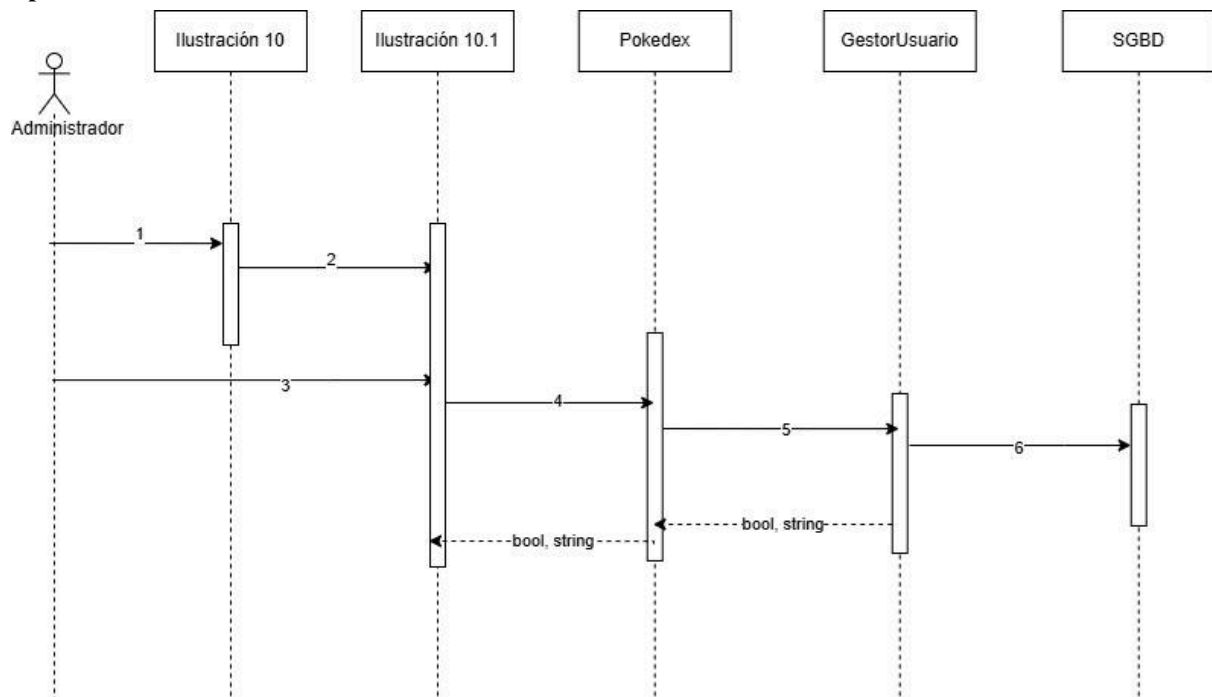
3. obtenerCuentasPendientes(): JSON
4. obtenerCuentasPendientes(): JSON
5. execSQL(sql1): ResultadoSQL
6. new ResultadoSQL()
7. next()
8. getString("username")
9. getString("email")
10. close()
11. obtenerCuentasAprobadas(): JSON
12. obtenerCuentasPendientes(): JSON
13. execSQL(sql2): ResultadoSQL
14. new ResultadoSQL()
15. next()
16. getString("username")
17. getString("email")
18. close()

sql1: "SELECT username, email FROM USUARIO WHERE aprobado=0"

sql2: "SELECT username, email FROM USUARIO WHERE aprobado=1"

JSON: { [ {username: string, email: string}, ...] }

### Aprobar cuenta



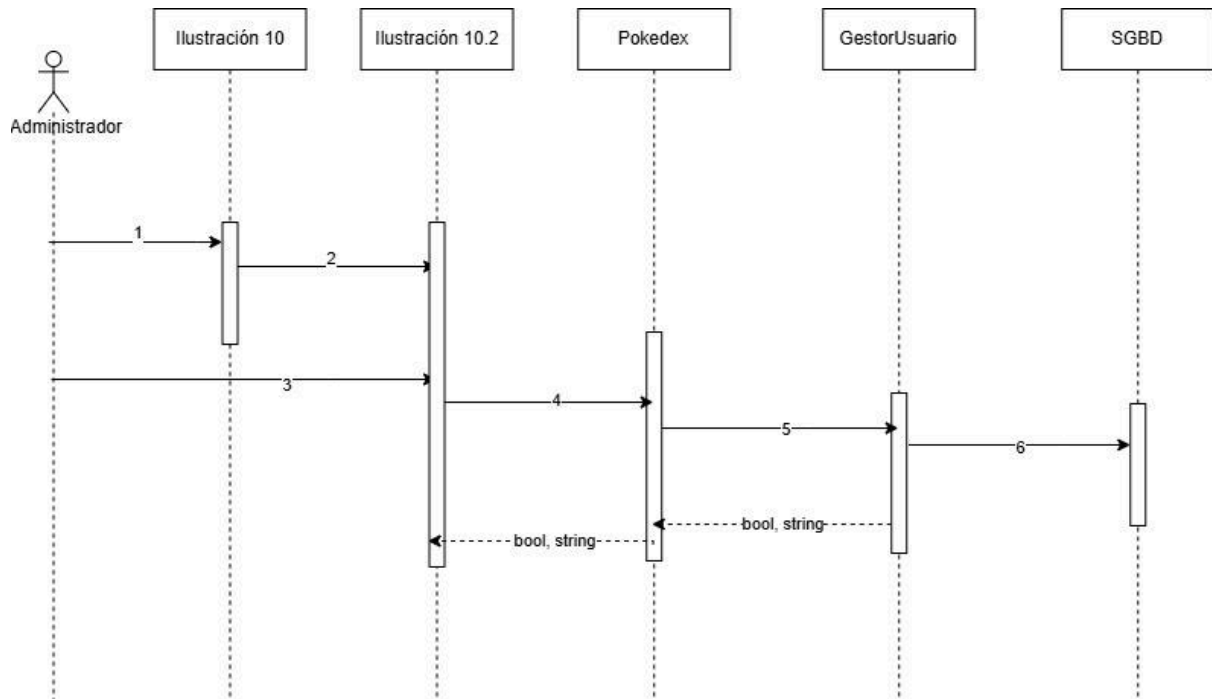
*Ilustración 96 - Diagrama Secuencia Aprobar cuenta*

[Si pulsa algo]

1. Seleccionar cuenta y pulsa "Aprobar"
2. new Ilustración10.1()
3. Pulsa "Aceptar"
4. aprobarCuenta(username): Tuple[bool, string]
5. aprobarCuenta(username): Tuple[bool, string]
6. execSQL(sql)

sql: "UPDATE USUARIO SET aprobado=1 WHERE username=%username%"

## Borrar cuenta



*Ilustración 97 - Diagrama Secuencia Borrar cuenta*

[Si pulsa algo]

1. Seleccionar cuenta y pulsa "Eliminar"
2. new Ilustración10.2()
3. Pulsa "Aceptar"
4. borrarCuenta(username): Tuple[bool. string]
5. borrarCuenta(username): Tuple[bool. string]
6. execSQL(sql)

sql: "DELETE FROM USUARIO WHERE username=%username%"

## FUNCIONALIDAD 2: Equipos

### 2.1 VER EQUIPO POKÉMON

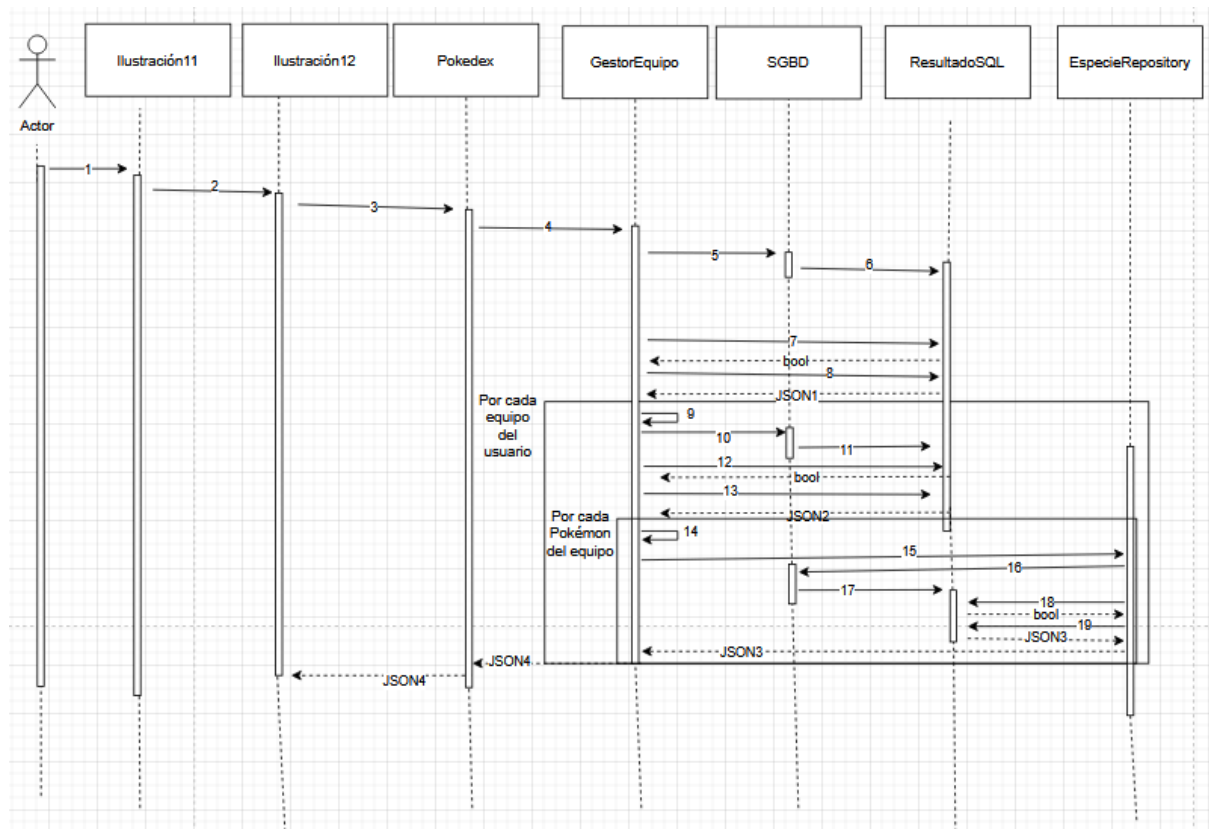


Ilustración 98 - Diagrama Secuencia Ver equipo

1. Hacer click en el botón “Ver equipos” desde el menú principal (Ilustración 11)
2. new Ilustración 12(username: string)
3. mis\_equipos():JSON4
4. get\_user\_equipos(username:String): JSON4
5. execSQL(SQL1):ResultadoSQL
6. new ResultadoSQL():void
7. next():bool
8. getJSON(): JSON1
9. get\_equipo\_pokemon(idEquipo:int):JSON2
10. execSQL(SQL2):ResultadoSQL
11. new ResultadoSQL():void
12. next():bool
13. getJSON(): JSON2
14. get\_especie\_details(nombreEspecie:string):JSON3
15. find\_by\_name(conn: conexion, nombreEspecie: string): JSON3
16. execSQL(SQL3): ResultadoSQL
17. new ResultadoSQL():void

18 next():bool

19. getJSON(): JSON3

JSON1: SELECT \* FROM equipo WHERE username = ?

JSON2: [{IDPokemon:int, Nombre: String, Ataque: int, AtaqueEsp:int, Def: int, DefEsp: int, Vel: int, vida: int, NombreEspecie: String},...{...}]

JSON3: {nombreEspecie:String, nombrePreEvolución:String, Ataque:int, AtaqueEsp:int, Def:int, DefEsp:int, Vida:int, Velocidad:int, foto:string, esLegendario:bool, shiny:bool}

JSON4: [{IDPokemon:int, Nombre: String, Ataque: int, AtaqueEsp:int, Def: int, DefEsp: int, Vel: int, vida: int, NombreEspecie: String, slot: int, foto: string},...{...}]

SQL1:

SELECT eq.nombreequipo, pok.nombre, eq.IDEquipo, esp.foto

FROM Equipo AS eq, Equipo\_Pokemon AS eqpok, Pokemon AS pok, Usuario AS usr, Especie AS esp

WHERE usr.username = %username% AND eq.idequipo = eqpok.idequipo AND pok.idpokemon = eqpok.idpokemon AND usr.username = eq.username AND esp.NombreEspecie = pok.NombreEspecie

SQL2:

SELECT p.\*, ep.slot FROM pokemon p

JOIN equipo\_pokemon ep ON p.idPokemon = ep.idPokemon

WHERE ep.idEquipo = ?

ORDER BY ep.slot

SQL3:SELECT \* FROM especie WHERE nombreEspecie = ?

## 2.2 ELIMINAR UN EQUIPO POKÉMON

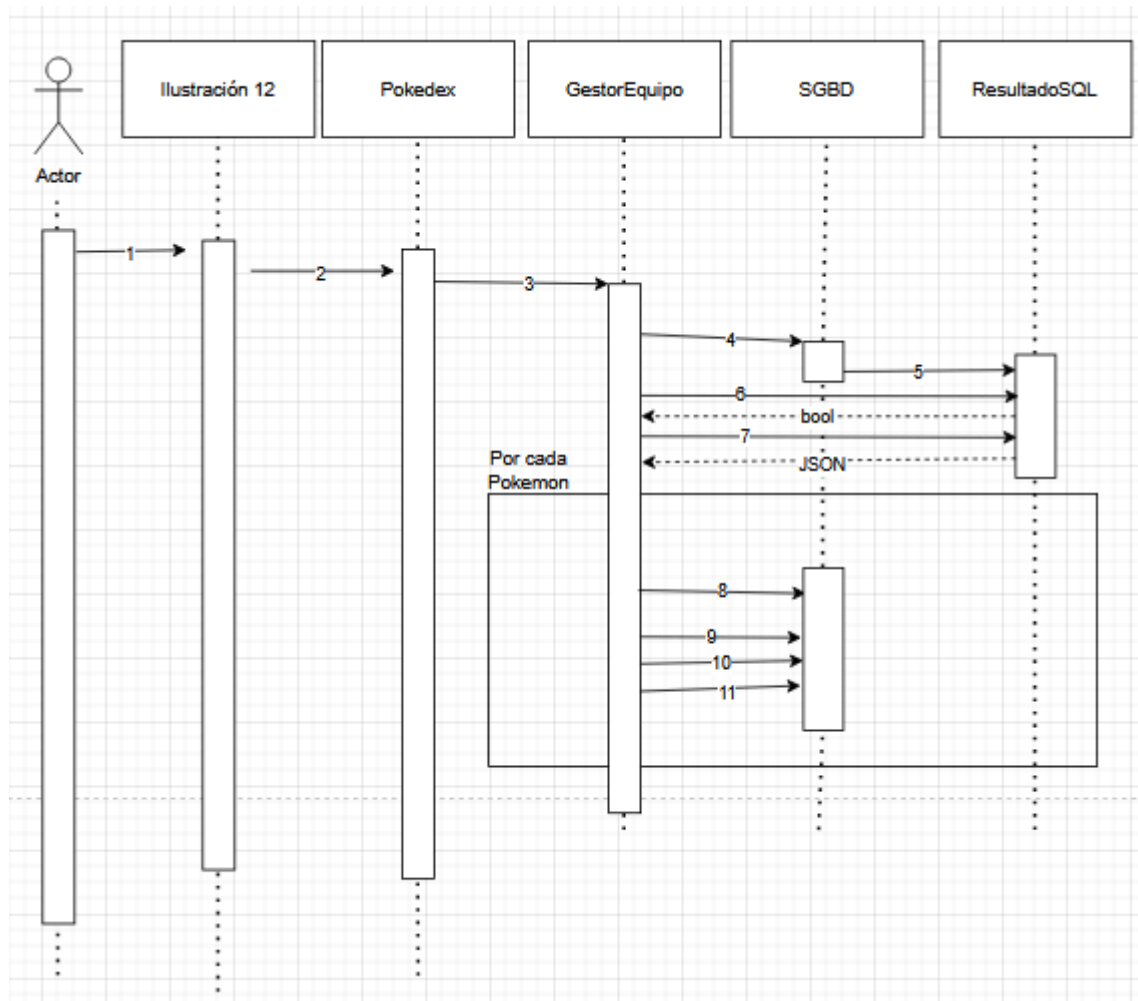


Ilustración 99 - Diagrama Secuencia Eliminar equipo

1. usuario pulsa "Eliminar equipo" en alguno de sus equipos
2. `eliminar_equipo(idequipo:int):void`
3. `delete_equipo(idequipo:int):void`
4. `execSQL(sql1):void`
5. `new ResultadoSQL():void`
6. `next():bool`
7. `getJSON(): JSON1`
8. `execSQL(sql1)`
9. `execSQL(sql2)`
10. `execSQL(sql3)`
11. `execSQL(sql4)`

JSON1:[{IDPokemon:int },...{...}]

SQL1:

SELECT p.idPokemon FROM pokemon p

JOIN equipo\_pokemon ep ON p.idPokemon = ep.idPokemon

WHERE ep.idEquipo = ?

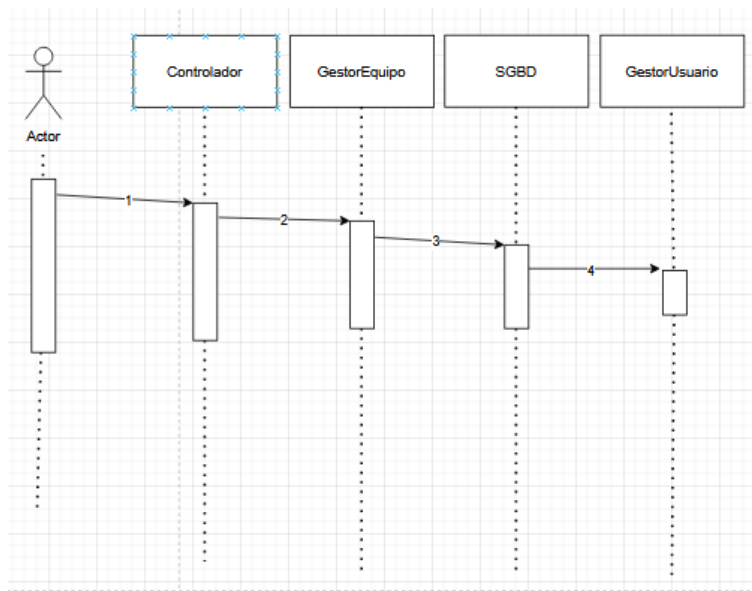
SQL2: DELETE FROM pokemon\_ataque AS pa WHERE pa.idPokemon = ?

SQL3: DELETE FROM equipo\_pokemon AS ep WHERE ep.idPokemon = ? AND ep.idEquipo = ?

SQL4: DELETE FROM pokemon AS p WHERE p.idPokemon = ?

SQL5: DELETE FROM equipo AS e WHERE e.idEquipo = ?

### 2.3 CREAR UN NUEVO EQUIPO POKÉMON



*Ilustración 100 - Diagrama Secuencia Crear equipo*

1. crear\_equipo(nombreEquipo:String):void

2. crear\_equipo(username:String, nombreEquipo:String):void

3. execSQL(sql1):void

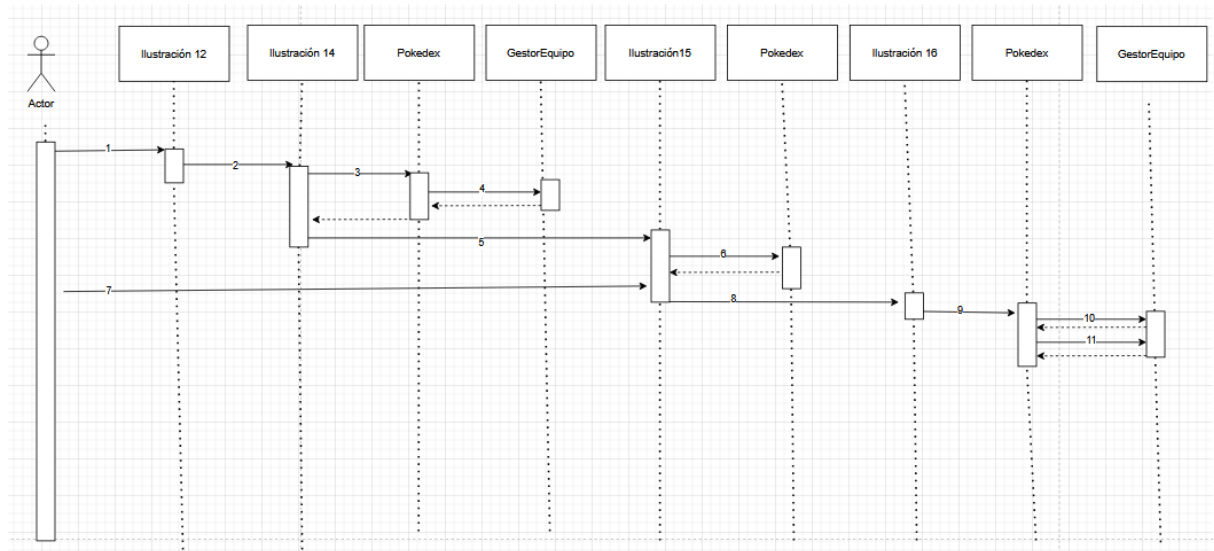
4. generarEvento(username:String, frase:string, frase2: string)

SQL1: INSERT INTO equipo (username, nombre, fechaCreacion) VALUES (?, ?, ?)



## 2.4 AÑADIR UN POKÉMON A UN EQUIPO

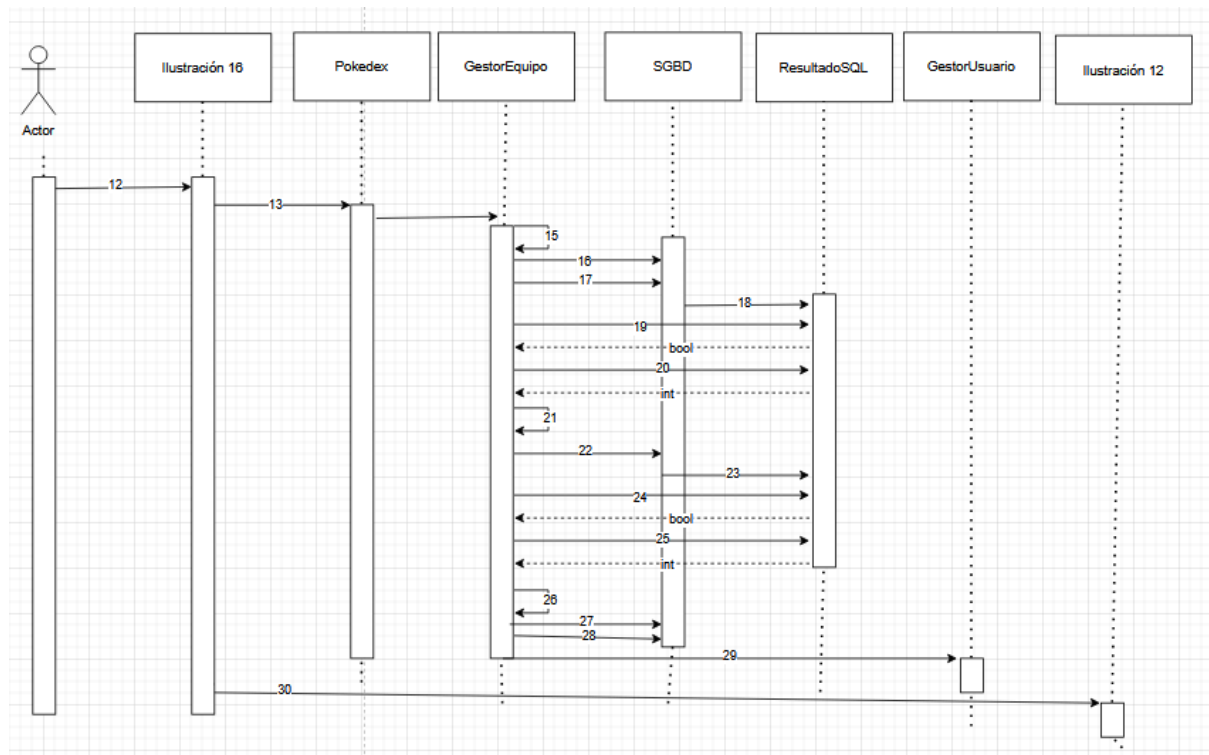
### PARTE 1



*Ilustración 101 - Diagrama Secuencia Añadir pokémon*

1. usuario pulsa "¡Captura un Pokémon!"
2. new Ilustracion14():JSON
3. index():JSON <-- Ver lista de todas las especies de Pokémon (correspondiente a la funcionalidad 4)
3. usuario pulsa "ver detalles" en un Pokémon
5. new Ilustración15():JSON
6. especie\_detail(nombreEspecie:string): JSON <-- Ver detalles / información de un Pokémon (correspondiente a funcionalidad 4)
7. usuario pulsa "Capturar" en la vista de detalles del Pokémon
8. new Ilustración16(nombreEspecie:string, username:string):JSON1
9. capture\_pokemon(nombreEspecie:string, username: string):JSON1
10. get\_user\_equipos(username:string):JSON <-- Ver los equipos del usuario (Correspondiente a funcionalidad 2.1)
11. get\_especie\_details(nombreEspecie:string):JSON <-- Ver los detalles de la especie de un Pokémon (en este caso el Pokémon a capturar)(Correspondiente a funcionalidad 2.1)

## PARTE 2



*Ilustración 101.1 - Diagrama Secuencia Añadir pokémon*

12. Usuario pulsa el botón "Añadir" de alguna de las ranuras disponibles de algún equipo suyo.
13. `create_pokemon_and_add(nombreEspecie:string, username:string, nickname:string, idequipo:int, slot:int):void`
14. `crear_pokemon(nickname:string, nombreEspecie:string): void`
15. `get_especie_details(nombreEspecie:string):JSON (Func.2.1)`
16. `execSQL(SQL1):void`
17. `execSQL(SQL2): ResultadoSQL`
18. `new ResultadoSQL()`
19. `next():bool`
20. `getInt('numPokemons'): int`  
 [SI EL NUMERO DE POKEMONS ES MAYOR O IGUAL QUE 6]
- 21.a. `eliminarPokemon(idPokemon:int, conn: conexion) <-- Parte de la funcionalidad 2.5`  
 [SI EL NUMERO DE POKEMONS ES MENOR QUE 6]
- 22.b. `execSQL(SQL3):ResultadoSQL`
- 23.b. `new ResultadoSQL()`
- 24.b. `next():bool`

25.b. getInt('sitioOcupado'): int <-- 1 si el sitio está ocupado, 0 si no está ocupado

[SI EL SITIO / RANURA / SLOT DEL EQUIPO ESTÁ OCUPADO]

26.b.a. eliminarPokemon(idPokemon:int, conn:conexion)

[SI EL SITIO ESTÁ LIBRE]

27.b.b. execSQL(SQL4):void

28.b.b. execSQL(SQL5):void

29.b.b generarEvento(username:string, frase:string,frase2:string) <-- Correspondiente a funcionalidad 3

30.b.b new Ilustración12(username:string)

JSON1: {nombreEspecie:String, foto:string, [{IDPokemon:int, Nombre: String, Ataque: int, AtaqueEsp:int, Def: int, DefEsp: int, Vel: int, vida: int, NombreEspecie: String, slot: int, foto: string},...{...}]}

SQL1: INSERT INTO pokemon(nombre, ataque, ataqueEsp, def, defEsp, vel, vida, nombreEspecie) VALUES (?, ?, ?, ?, ?, ?, ?, ?)

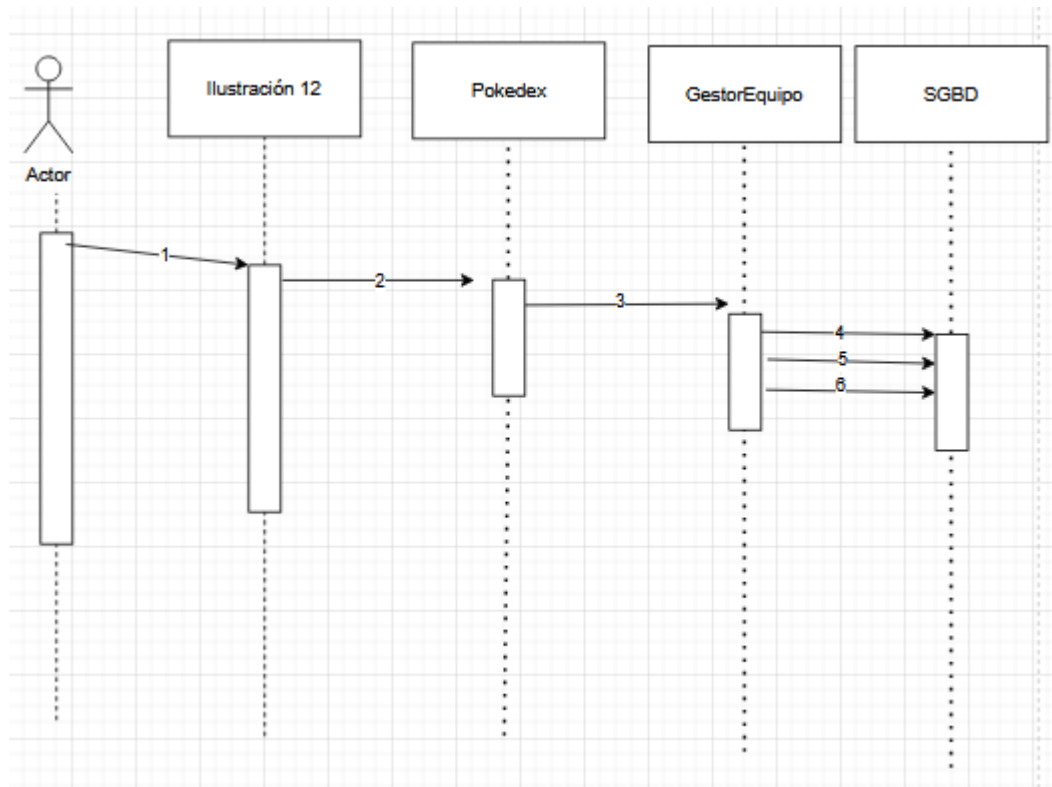
SQL2: SELECT COUNT(\*) FROM equipo\_pokemon WHERE idEquipo = ?

SQL3: SELECT 1 FROM equipo\_pokemon WHERE idEquipo = ? AND slot = ?

SQL4:INSERT INTO equipo\_pokemon (idEquipo, slot, idPokemon) VALUES (?, ?, ?)

SQL5:SELECT e.username, p.nombreEspecie FROM equipo e,pokemon p WHERE e.idEquipo = ? AND p.idPokemon = ?

## 2.5 EXPULSAR UN POKÉMON DE UN EQUIPO



*Ilustración 102 - Diagrama Secuencia Expulsar pokémon*

1. usuario pulsa "Expulsar" en cualquiera de las casillas en las que hay un Pokémon
2. expulsar\_pokemon(idPokemon:int)
3. delete\_pokemon(idPokemon:int):
4. execSQL(sql1):void
5. execSQL(sql2):void
6. execSQL(sql3):void

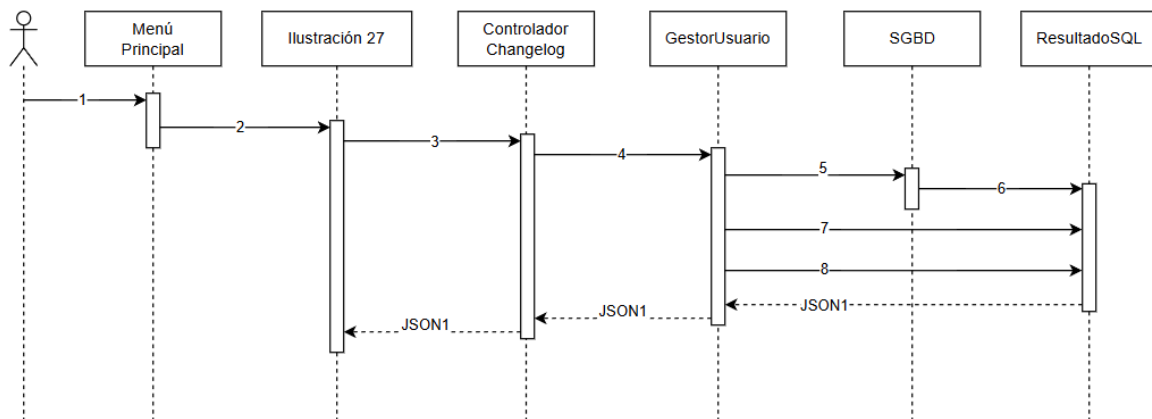
SQL1: DELETE FROM pokemon\_ataque AS pa WHERE pa.idPokemon = ?

SQL2: DELETE FROM equipo\_pokemon AS ep WHERE ep.idPokemon = ?

SQL3:DELETE FROM pokemon AS p WHERE p.idPokemon = ?

## FUNCIONALIDAD 3: ChangeLog

Obtener notificaciones:



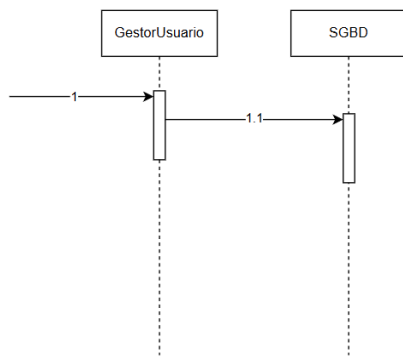
*Ilustración 103 - Diagrama Secuencia Obtener notificaciones*

1. El usuario selecciona la opción "Changelog"
2. `new vistaChangelog()`
3. `obtenerNotificaciones(username): JSON1`
4. `obtenerNotificaciones(username): JSON1`
5. `execSQL (sql): ResultadoSQL`
6. `new ResultadoSQL()`
7. `next() : bool`
8. `getJSON(): JSON1`

JSON1: { [ { username: string, fecha: date, tipo: string, texto: string } , ... ] }

sql1: "SELECT n.username, n.fecha, n.tipo, n.texto  
FROM NOTIFICACIONES n  
INNER JOIN SEGUIDORES s ON n.username = s.seguido  
WHERE s.seguidor = %username%  
ORDER BY n.fecha DESC  
LIMIT 20"

## Generar notificaciones

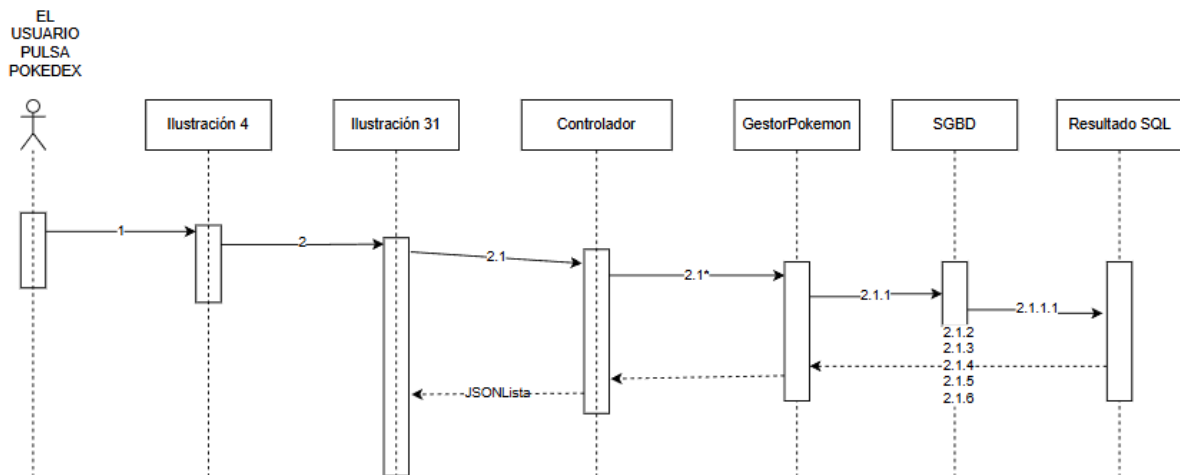


*Ilustración 104 - Diagrama Secuencia Generar notificación*

1. `generarEvento( usuario, tipo, texto)`
    - 1.1. `execSQL (sql2)`
- sql2: "INSERT INTO notificaciones (username, fecha, tipo, texto)  
VALUES (%username%, datetime('now'), %tipo%, %texto%) "

## FUNCIONALIDAD 4: Ver lista de pokemon

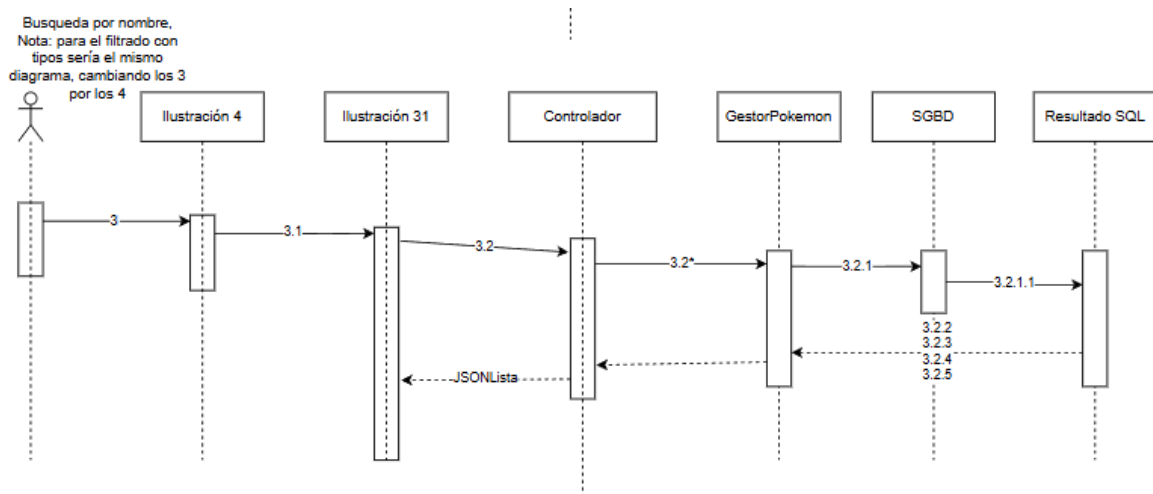
### 4.1 Acceder a la pokedex



*Ilustración 105- Diagrama de Comunicación Lista Pokémon*

1. [EL USUARIO PULSA POKEDEX]
2. new VistaListaPokemon()
  - 2.1. cargarListaPokemon(filtro= " "): JSONLista
    - 2.1.1 execSQL(sql1): ResultadoSQL1
      - 2.1.1.1 new ResultadoSQL1()
      - 2.1.1.2 next()
      - 2.1.1.3 getVida()
      - 2.1.1.4 getDaño()
      - 2.1.1.5 getSprite()
      - 2.1.1.6 getNombre()

### 4.2 Buscar y filtrar por tipo



*Ilustración 106 - Diagrama de Comunicación Búsqueda por nombre y tipo*

### 3. [EL USUARIO PULSA BUSCAR]

3.1 aplicarFiltroBusqueda(nombrePokemon)

3.2 cargarListaPokemon(filtro= nombrePokemon): JSONLista

3.2.1 execSQL(sql3): ResultadoSQL3

3.2.1.1 new ResultadoSQL3()

3.2.2 next()

3.2.3 getVida()

3.2.4 getDaño()

3.2.5 getSprite()

### 4. [EL USUARIO SELLECCIONA UN FILTRO]

4.1 aplicarFiltroBusqueda(tipoPokemon)

4.2 cargarListaPokemon(filtro= tipoPokemon): JSONLista

4.2.1 execSQL(sql2): ResultadoSQL2

4.2.1.1 new ResultadoSQL2()

4.2.2 next()

4.2.3 getVida()

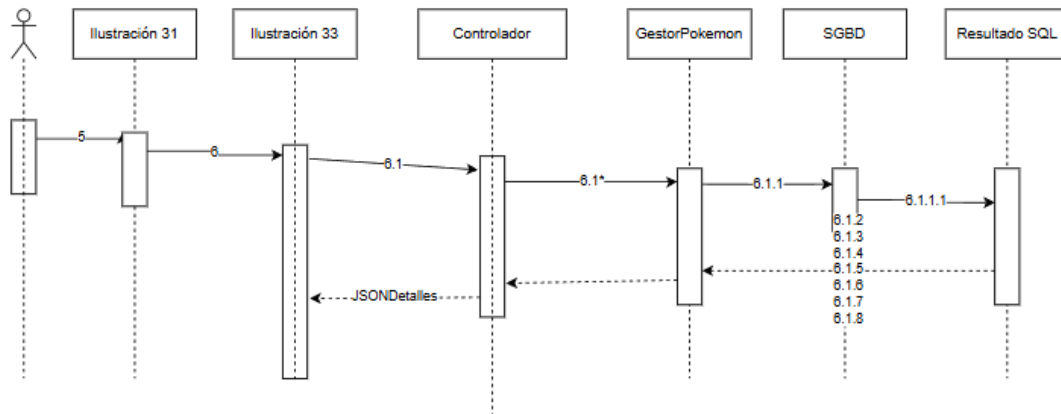
4.2.4 getDaño()

4.2.5 getSprite()

### 4.3 Ver detalles de un Pokemon



Ver info de  
Pokemon



*Ilustración 107 - Diagrama Comunicación*

5. [EL USUARIO PULSA VER DETALLES EN UN POKEMON]
6. new VistaPokemon(id)
  - 6.1 cargarEstadisticasPokemon(id): JSONDetalles
    - 6.1.1 execSQL(sql4): ResultadoSQL4
      - 6.1.1.1 newResultadoSQL4()
    - 6.1.2 getNombre()
    - 6.1.3 getDescripcion()
    - 6.1.4 getTipo()
    - 6.1.5 getAtaque()
    - 6.1.6 getDefensa()
    - 6.1.7 getVelocidad()
    - 6.1.8 getMovimientos()

JSON y SQL:

Estos serían las sentencias SQL necesarias para esta funcionalidad:

Para generar la lista completa:

sql1= ("SELECT nombre, daño, vida. sprite FROM listaPokemon")

Para buscar Pokemon:

sql2= ("SELECT nombre, daño, vida. sprite FROM listaPokemon WHERE name ='"+ nombrePokemon)

Para filtrar Pokemon:

sql3= ("SELECT nombre, daño, vida. sprite FROM listaPokemon WHERE type=" + tipoPokemon)

Para ver detalles de Pokemon:

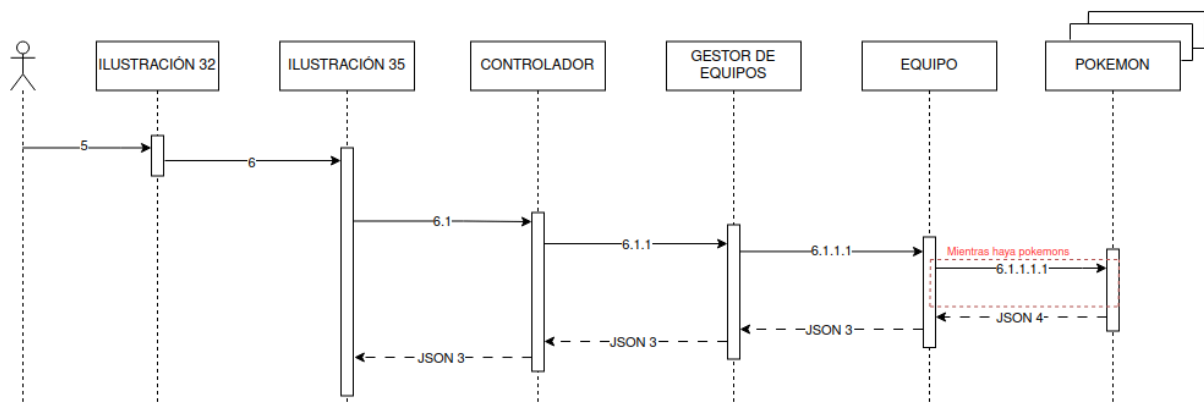
```
sql4= ("SELECT nombre, daño, vida. sprite, descripcion, tipo, velocidad, movimientos FROM
listaPokemon WHERE id =" + idPokemon)
```

```
JSONLista = { [ { id : int, nombre : string, imagenSprite : string, vida : int, daño : int} ] }
```

```
JSONDetalles = { [ { id : int, nombre : string, descripción : string, tipo : [string, string], ataque : int,
defensa: int, velocidad: int, movimientos : [string, ...] } ] }
```

## FUNCIONALIDAD 5: ChatBot

### 5.2 OBTENER MEJOR POKEMON DE UN EQUIPO:



*Ilustración 108- Diagrama Secuencia Obtener pokemon*

5. El usuario pulsa el botón "Seleccionar Equipo"

6. newFigura35(nombreEquipo)

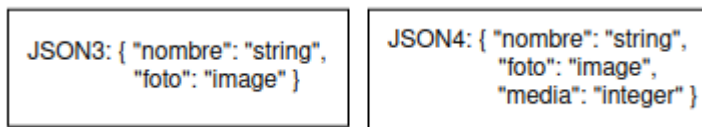
6.1 obtenerMejorPokemon(nombreEquipo): JSON3

6.1.1 obtenerMejorPokemon(nombreEquipo): JSON3

6.1.1.1 execSQL(SQL7): JSON3

6.1.1.1 newResultadoSQL(): JSON4

```
SELECT nombre, foto,
(Ataque + AtaqueEsp + Def + DefEsp + Vel + Vida) / 6 AS promedio_stats
FROM POKEMON;
```



### 5.3 BUSCAR POKEMON:

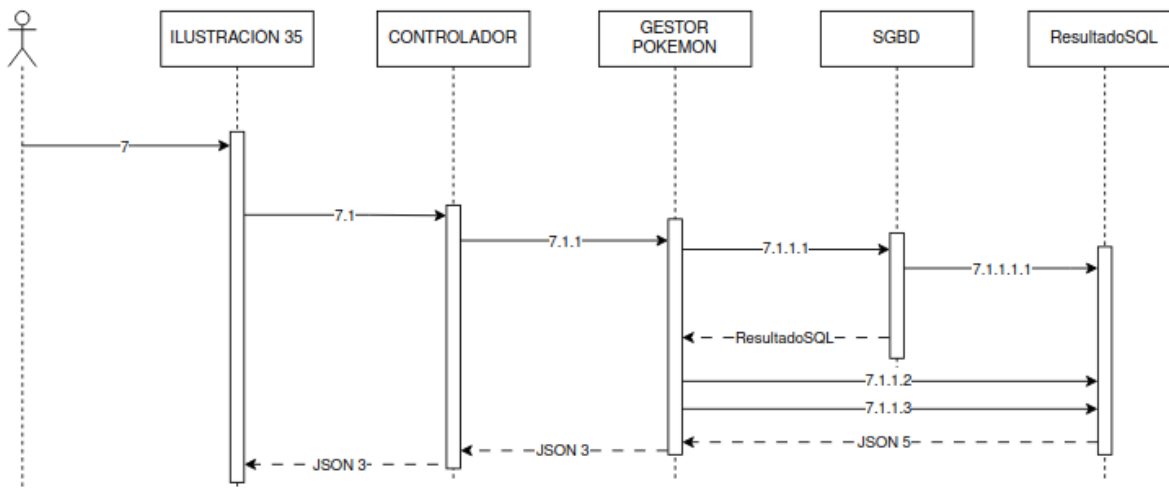


Ilustración 109 - Diagrama Secuencia Buscar pokemon

7. El usuario pulsa el botón "Seleccionar Pokémon" y en el desplegable escribe el nombre de un Pokémon correcto:

7.1 obtenerPokemon(string): JSON3

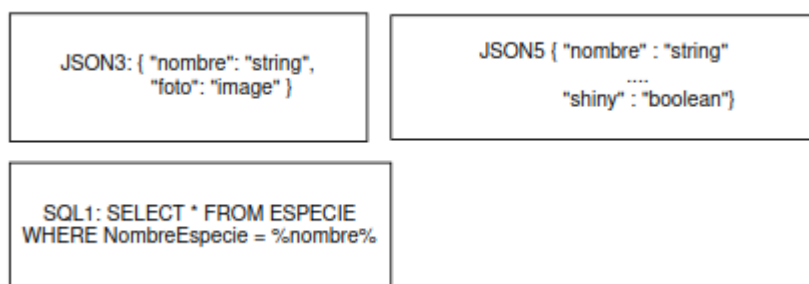
7.1.1 obtenerPokemonEntero(string): JSON3

7.1.1.1 execSQL(SQL1): ResultadoSQL

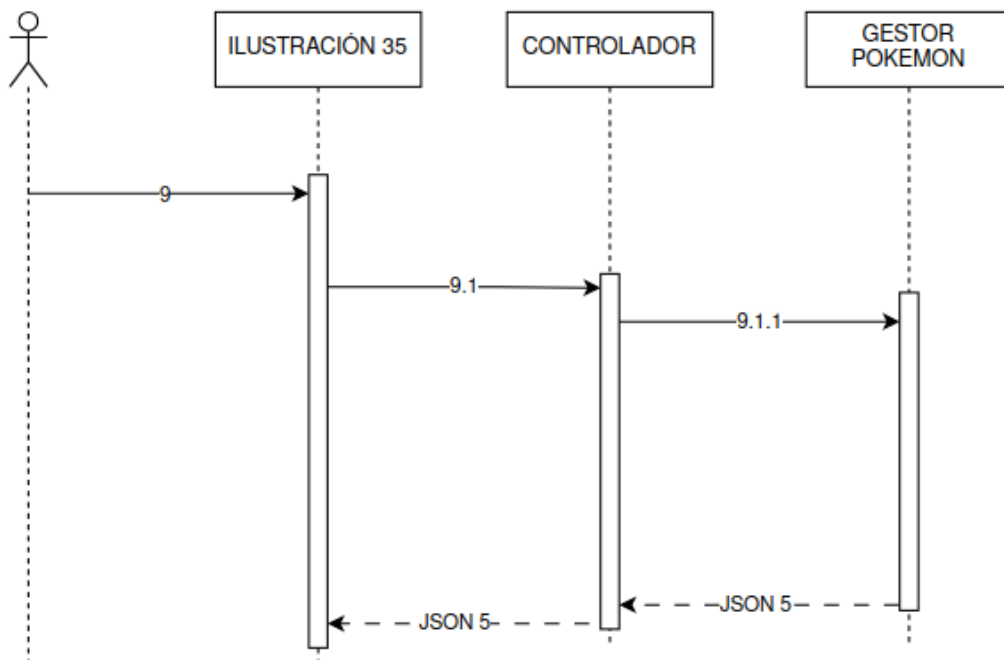
7.1.1.1.1 newResultadoSQL()

7.1.1.2 next()

7.1.1.3 obtenerPokemonJSON(): JSON5



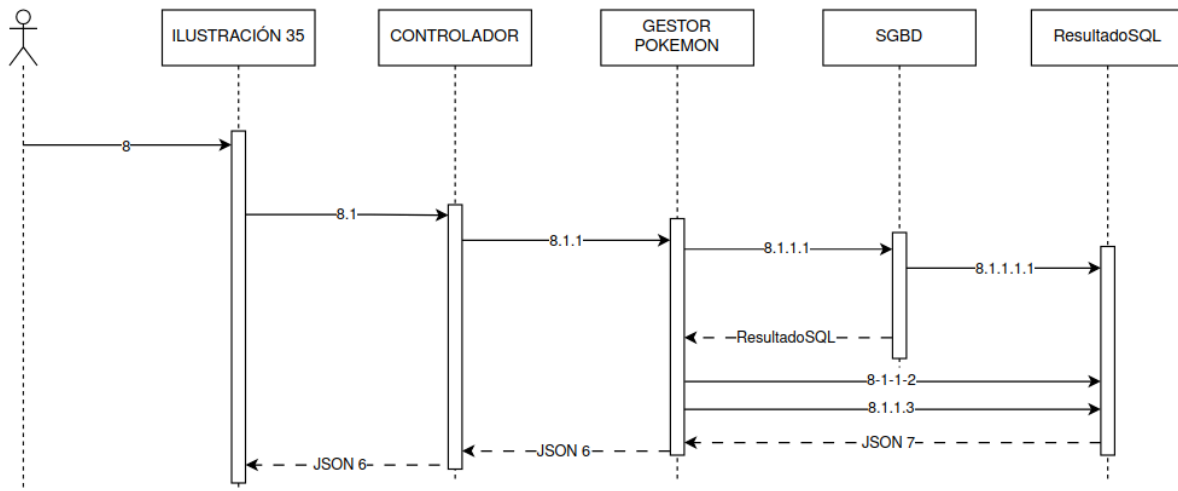
#### 5.4 VER ESTADÍSTICAS:



*Ilustración 110 - Diagrama Secuencia Ver estadísticas*

- 9. El usuario pulsa el botón "Ver estadísticas"
- 9.1 obtenerEstadísticas(): JSON5
- 9.1.1 obtenerEstadísticas(): JSON5

## 5.5 VER CADENA EVOLUTIVA:



*Ilustración 111 - Diagrama Secuencia Ver cadena evolutiva*

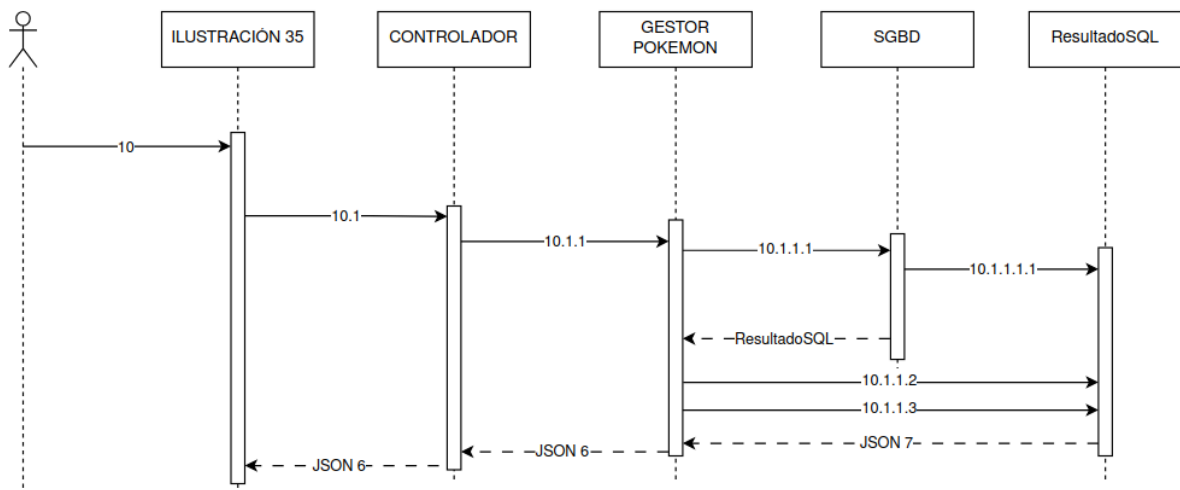
8. El usuario pulsa el botón "Ver cadena evolutiva"
  - 8.1 obtenerCadenaEvolutiva(string): JSON6
    - 8.1.1 obtenerCadenaEvolutiva(string): JSON6
      - 8.1.1.1 execSQL(SQL2): ResultadoSQL
        - 8.1.1.1.1 newResultadoSQL()
      - 8.1.1.2 next()
      - 8.1.1.3 obtenerFotoEvolucionesJSON(): JSON7

```
JSON6: { "pokemon1" : image,
        ....
        "pokemonN" ; image }
```

```
JSON7: { "pokemon" : image }
```

```
SQL2: SELECT previo FROM ESPECIE
      WHERE NombreEspecie = %nombre%
```

## 5.6 VER RIVALES:



*Ilustración 112 - Diagrama Secuencia Ver rivales*

10. El usuario pulsa el botón "Ver rivales"

10.1 obtenerRivales(string): JSON6

10.1.1 obtenerRivales(string): JSON6

10.1.1.1 execSQL(SQL3): ResultadoSQL

10.1.1.1.1 newResultadoSQL()

10.1.1.2 next()

10.1.1.3 obtenerFotoJSON(): JSON7

```

SQL3:
SELECT Image
FROM ESPECIE
WHERE NombreEspecie = (
  SELECT nombreEspecie
  FROM Especie_Tipo
  NATURAL JOIN TIPO
  NATURAL JOIN AFECTADO
  ORDER BY multiplo DESC
  LIMIT 1
)
LIMIT 3;
  
```

## 5.7 CARGAR EQUIPOS:

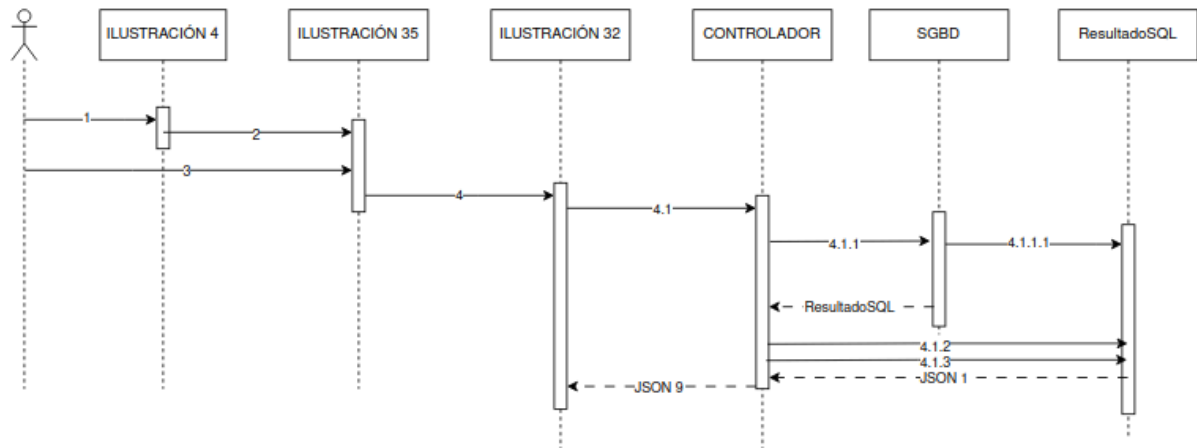
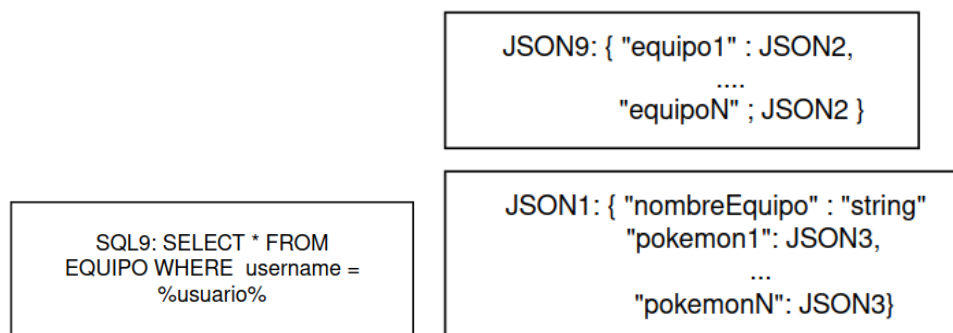


Ilustración 113 - Diagrama Secuencia Cargar equipos

1. El usuario pulsa el botón "ChatBot"
2. newFigura35()
3. El usuario pulsa el botón "Seleccionar Equipo"
4. newFigura32()
  - 4.1 obtenerEquiposNombreSprite(usuario): JSON9
    - 4.1.1 execSQL(SQL9): ResultadoSQL
      - 4.1.1.1 newResultadoSQL():
        - 4.1.1.2 next()
      - 4.1.3 obtenerPokemonJSON(): JSON5



## FUNCIONALIDAD 6: Bot de Telegram

Diagrama de secuencia de Command1() de TelegramAPI - Mostrar los equipos

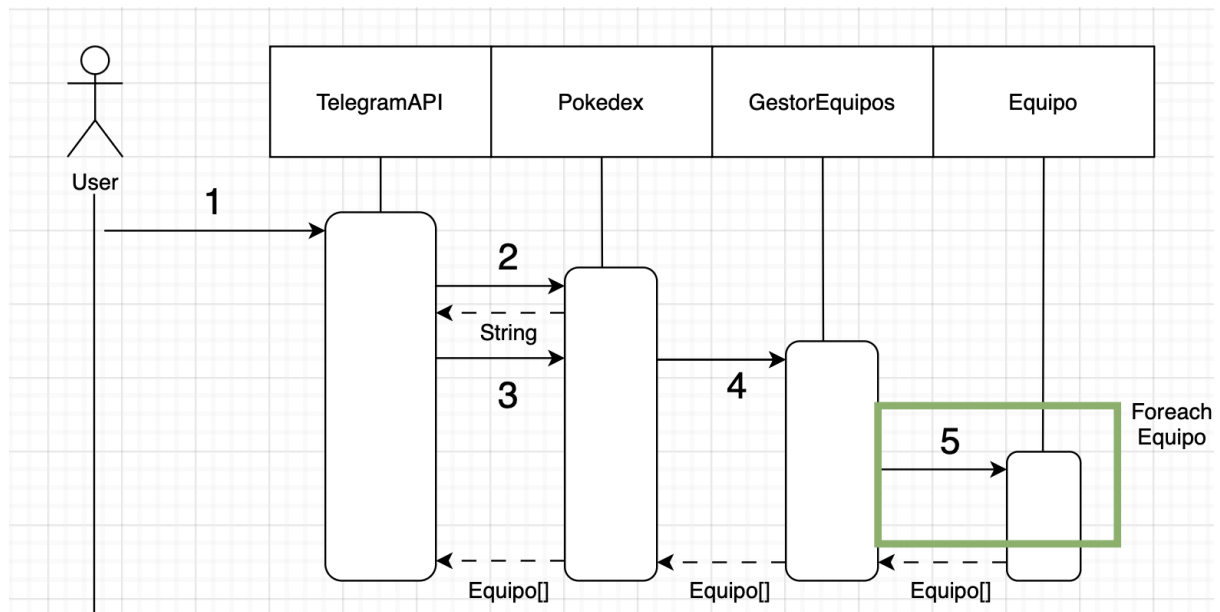
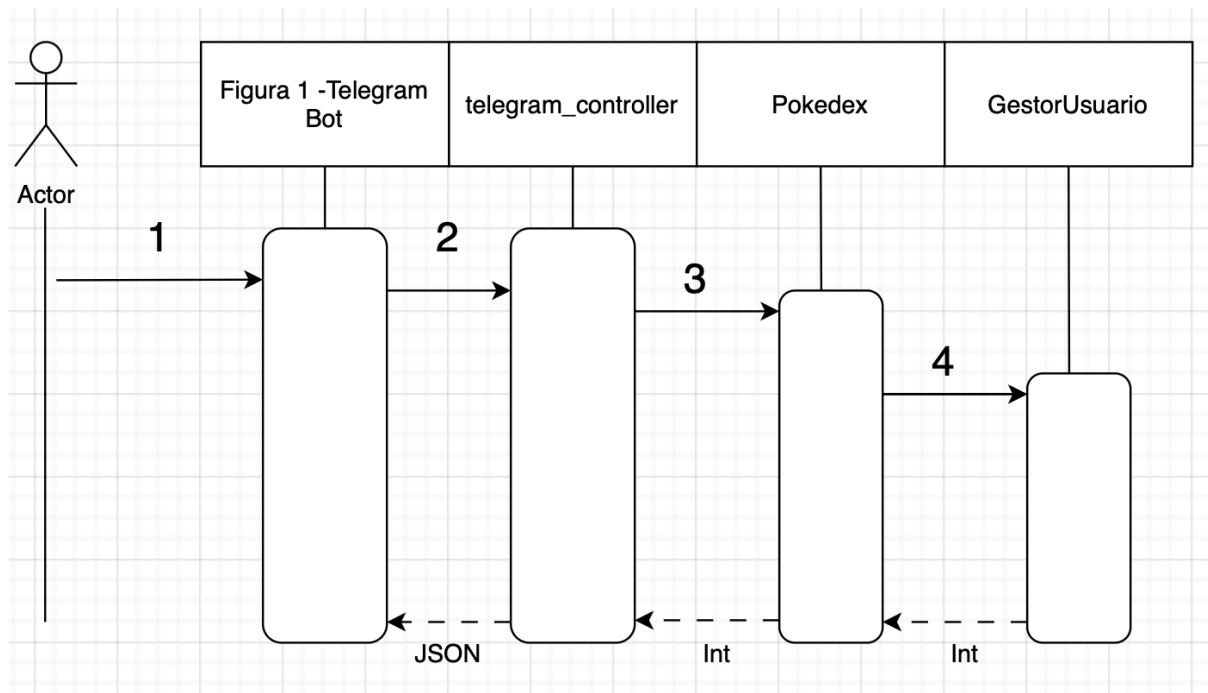


Ilustración 114 - Diagrama Secuencia Command1

1. El usuario ejecuta el comando 1
2. `getUserByTelegram(nombreTelegram) : String`
3. `getEquipoByUser(nombreApp) : Equipo[]`
4. `get_by_user(nombreApp) : Equipo[]`  
*[En ocasiones se ha prescindido de la BD y el gestor genera las consultas usando código en lugar de Strings pre-hechas. En este caso el Gestor pregunta por la tabla equipos cuyos campos "username" sean el mismo que "nombreApp"]*
5. `new Equipo()`  
*[Por cada equipo que tenga el usuario con "nombreAPP". El objeto Equipo resulta muy cómodo ya que tiene objetos de tipo Pokemon dentro así que se devuelve una lista de los equipos y finalmente TelegramAPI decide que quiere hacer con la información dependiendo de los argumentos del comando]*

Diagrama de secuencia del boton "Vincular mi Cuenta" de Fig.1 - Asociar cuenta APP->Telegram





*Ilustración 115 - Diagrama Secuencia Asociar cuenta*

1. Boton "Vincular mi Cuenta"
2. share() : JSON  
[Endpoint de flask. Se han creado periféricos al controlador principal (Pokedex) ya que es mala práctica que todas las URL vayan al mismo controlador. Devuelve al frontend un JSON con estructura {"status" : "okOrNot"} para que se notifique correctamente al user. El nombre de este método tiene un origen.]
3. vincularUsuario(username, telegramUsername) : Int
4. vincularUsuario(username, telegramUsername) : Int  
[En este caso el código SQL hace UPDATE del campo "cuentaTelegram" en la tabla "Usuario". Dependiendo de si la operación ha sido exitosa o no, devuelve un 1 o un 0 respectivamente.]

## FUNCIONALIDAD 7: Pokedle

### Caso de uso entrar a Pokedle desde pantalla principal

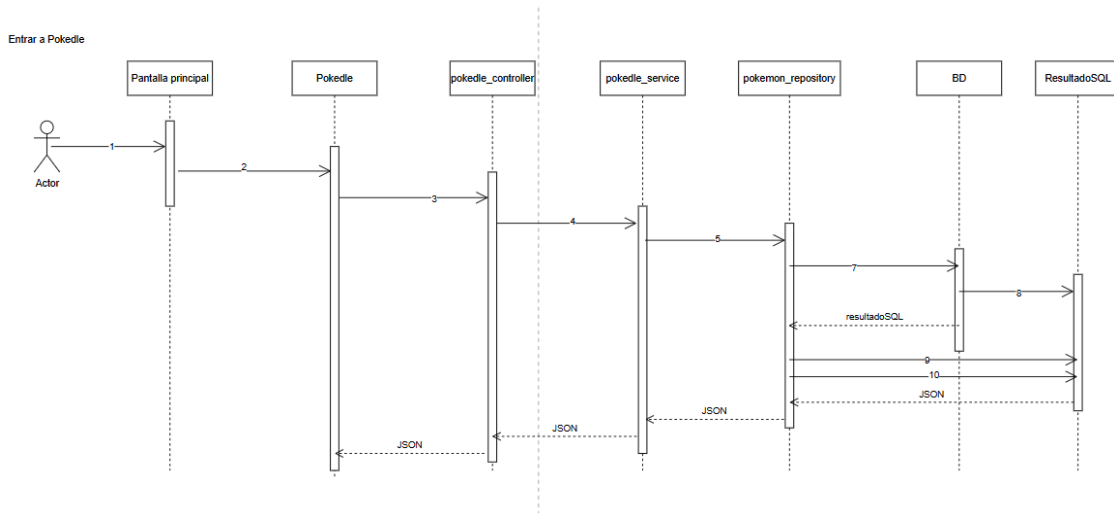


Ilustración 116 - Diagrama Secuencia Acceder pokedle

1. El usuario pulsa en Pokedle

2. new Pokedle()

3. start\_game(): JSON

4. start\_game(): JSON

5. getRandom(): JSON

7. execSQL(sql1): ResultadoSQL

8. new ResultadoSQL()

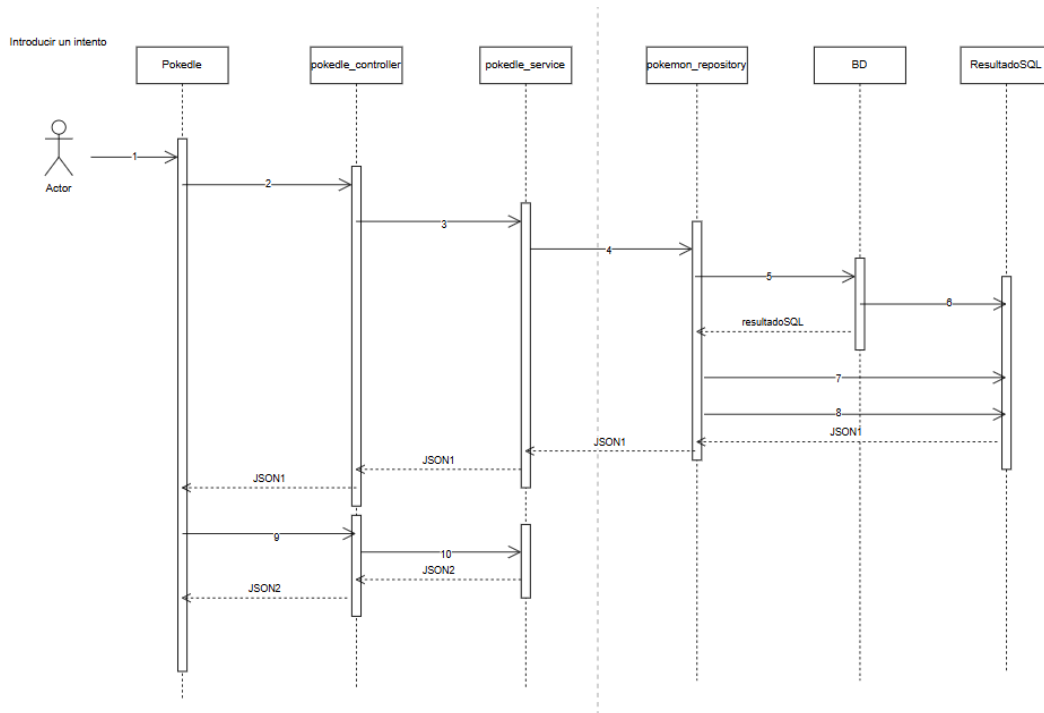
9. next()

10. obtenerInfoPokemon: JSON

sql1: SELECT pokemon ORDER BY RANDOM()

JSON: {nombreEspecie:String, nombrePreEvolución:String, Ataque:int, AtaqueEsp:int, Def:int, DefEsp:int, Vida:int}

## Caso de uso introducir un intento



*Ilustración 117 - Diagrama Secuencia Introducir un intento*

1. El usuario introduce el nombre de un pokemon y pulsa aceptar

2. sendGuess():JSON1

3. sendGuess():JSON1

4. findBySpecies(nombre)

5. execSQL(sql1):ResultadoSQL

si el pokemon no existe se manda un mensaje

6. new ResultadoSQL()

7. next()

8. getDatos():JSON1

9. compare\_pokemons(target, guess):JSON2

10. compare\_pokemons(target, guess):JSON2

sql1: SELECT pokemon WHERE nombreEspecie

JSON: {nombreEspecie:String, nombrePreEvolución:String, Ataque:int, AtaqueEsp:int, Def:int, DefEsp:int, Vida:int}

## 7. PLANES DE PRUEBA E IMPLEMENTACIÓN

### FUNCIONALIDAD 1: Gestión de usuarios - Ibai Munné

[si hay más de un error, ej: “nombre de usuario ya registrado” y “email no existe”, la aplicación mostrará solo el error que más arriba esté en esta tabla (preferencia mayor cuanto más arriba)]

| Código de Prueba | Descripción                                                                                                                                                                                  | Resultado esperado                                               | Resultado obtenido                               | Observaciones                                                                 |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------|
| 1. Registro      | <b>nombre de usuario ya registrado</b>                                                                                                                                                       | <b>error</b> → el nombre de usuario debe ser único               | Mensaje de error: “El usuario ya existe”         | Correcto                                                                      |
| 2. Registro      | nombre de usuario único, <b>email erróneo/no existe</b>                                                                                                                                      | <b>error</b> → no existe el correo electrónico                   | Mensaje de error: “Email inválido”               | Correcto                                                                      |
| 3. Registro      | nombre de usuario único, existe email, <b>contraseña no cumple con estándares marcados</b> (7 < length() < 17 && count(Mayuscula) > 0 && count(Numero) > 0 && count(CharacterEspecial) > 0 ) | <b>error</b> → contraseña demasiado débil                        | Mensaje de error sobre la contraseña débil       | Correcto. Detecta contraseñas sin mayúsculas, números o caracteres especiales |
| 4. Registro      | nombre de usuario único, existe email, contraseña segura, pero <b>(contraseña ≠ verificar contraseña)</b>                                                                                    | <b>error</b> → la contraseña y verificar contraseña no coinciden | Mensaje de error: “Las contraseñas no coinciden” | Correcto                                                                      |
| 5. Registro      | nombre de usuario único, existe email,                                                                                                                                                       | <b>ACCESO AL SISTEMA</b>                                         | Mensaje: “Usuario registrado                     | Correcto. Registra usuario pero requiere                                      |

|                      |                                                                                                                                                                                                                   |                                                                             |                                                                                          |                                                                                                  |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
|                      | contraseña segura, contraseña = verificar contraseña                                                                                                                                                              |                                                                             | exitosamente. Espera la aprobación...”                                                   | aprobación administrativa                                                                        |
| 6. Login             | <b>usuario no está registrado</b>                                                                                                                                                                                 | <b>error</b> → usuario no registrado o está mal escrito                     | Mensaje de error: “Usuario o contraseñas incorrectos”                                    | Correcto. Mensaje genérico por seguridad                                                         |
| 7. Login             | usuario registrado pero <b>contraseña incorrecta</b>                                                                                                                                                              | <b>error</b> → usuario o contraseña son incorrectos                         | Mensaje de error: “Usuario o contraseñas incorrectos”                                    | Correcto. Mismo mensaje genérico por seguridad                                                   |
| 8. Login             | usuario registrado y contraseña correcta, pero <b>cuenta no validada</b> (por un administrador)                                                                                                                   | <b>error</b> → falta la aprobación por un administrador para usar la cuenta | Mensaje de error: “Cuenta pendiente de aprobación”                                       | Correcto. Mensaje específico para cuenta no aprobada                                             |
| 9. Login             | usuario registrado y contraseña correcta y cuenta validada                                                                                                                                                        | <b>ACCESO AL SISTEMA</b>                                                    | <b>ACCESO AL SISTEMA</b>                                                                 | Correcto. Devuelve datos del usuario y mensaje de éxito                                          |
| 10. Gestionar amigos | busca un usuario y no existe ninguno con dicho nombre                                                                                                                                                             | <b>error</b> → no existe ningún usuario con este nombre                     | Lista vacía []                                                                           | La función devuelve lista vacía, no mensaje de error. El UI debería mostrar el mensaje apropiado |
| 11. Actualizar datos | nombre de usuario único, <b>email erróneo/no existe</b>                                                                                                                                                           | <b>error</b> → no existe el correo electrónico                              | False y “Email inválido”                                                                 | Correcto. Valida el formato de email                                                             |
| 12. Actualizar datos | nombre de usuario único, existe email, <b>contraseña no cumple con estándares marcados</b> ( $7 < \text{length}() < 17 \ \&\& \text{count}(\text{Mayuscula}) > 0 \ \&\& \text{count}(\text{Numero}) > 0 \ \&\&$ ) | <b>error</b> → contraseña demasiado débil                                   | False y “La contraseña debe tener entre 8 y 16 caracteres y mayúsculas/números/símbolos” | Correcto                                                                                         |

|                      |                                                                                                           |                                                                  |                                          |           |
|----------------------|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|------------------------------------------|-----------|
|                      | count(CaracterEspecial) > 0 )                                                                             |                                                                  |                                          |           |
| 13. Actualizar datos | nombre de usuario único, existe email, contraseña segura, pero <b>(contraseña ≠ verificar contraseña)</b> | <b>error</b> → la contraseña y verificar contraseña no coinciden | False y “Las contraseñas no coinciden”   | Correcto  |
| 14. Actualizar datos | nombre de usuario único, existe email, contraseña segura, contraseña = verificar contraseña               | <b>PERFIL ACTUALIZADO CORRECTAMENTE</b>                          | True y “Datos actualizados exitosamente” | Correcto. |

## FUNCIONALIDAD 2: Crear equipos Pokémon - Aitana Niño

| Código de Prueba | Descripción                                                                        | Resultado esperado                                                                                                                                                                                                          | Resultado obtenido                                                                                                                                                                                                          | Observaciones                                                                                                                                                                                                               |
|------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Ver equipos   | Acceder a la opción ver equipos desde el menú principal, teniendo mínimo un equipo | Se muestran los equipos con 6 ranuras para los Pokémon.                                                                                                                                                                     | Se muestran los equipos con 6 ranuras para los Pokémon.                                                                                                                                                                     | Se muestran los equipos con 6 ranuras para los Pokémon.                                                                                                                                                                     |
| 2. Ver equipos   | Acceder a la opción ver equipos desde el menú principal, sin tener ningún equipo.  | Se muestra un mensaje con “Aún no tienes equipos. ¡Crea tu primer equipo arriba!” y la opción para crear nuevo equipo.                                                                                                      | Se muestra un mensaje con “Aún no tienes equipos. ¡Crea tu primer equipo arriba!” y la opción para crear un nuevo equipo.                                                                                                   | Se muestra un mensaje con “Aún no tienes equipos. ¡Crea tu primer equipo arriba!” y la opción para crear un nuevo equipo.                                                                                                   |
| 3. Crear equipo  | Seleccionar la opción crear nuevo equipo, teniendo menos de 10 equipos.            | Se muestra el equipo, vacío, con el nombre del equipo, la opción de eliminarlo, la fecha y hora de creación, y un botón para capturar pokémon. En lugar de haber slots de Pokemons, hay un mensaje que anima a capturarlos. | Se muestra el equipo, vacío, con el nombre del equipo, la opción de eliminarlo, la fecha y hora de creación, y un botón para capturar pokémon. En lugar de haber slots de Pokemons, hay un mensaje que anima a capturarlos. | Se muestra el equipo, vacío, con el nombre del equipo, la opción de eliminarlo, la fecha y hora de creación, y un botón para capturar pokémon. En lugar de haber slots de Pokemons, hay un mensaje que anima a capturarlos. |
| 4. Crear equipo  | Seleccionar la opción crear nuevo equipo, 10 equipos.                              | Mensaje de error → “Límite de equipos alcanzado.”                                                                                                                                                                           | Se crea el 11vo equipo sin problemas. Además, se pueden seguir creando infinitos equipos.                                                                                                                                   | El método de crear equipo nunca comprueba la cantidad de equipos que tiene el usuario                                                                                                                                       |
| 5. Editar equipo | Añadir un Pokémon a una ranura vacía u ocupada, seleccionando la opción “añadir”.  | El Pokémon seleccionado aparece en la ranura correspondiente.                                                                                                                                                               | El Pokémon seleccionado aparece en la ranura correspondiente.                                                                                                                                                               | El Pokémon seleccionado aparece en la ranura correspondiente.                                                                                                                                                               |

|                    |                                                                                                                  |                                                               |                                                               |                                                               |
|--------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------|
| 6. Editar equipo   | Eliminar un Pokémon del equipo, seleccionando la opción “expulsar”.                                              | La ranura se muestra vacía.                                   | La ranura se muestra vacía.                                   | La ranura se muestra vacía.                                   |
| 7. Editar equipo   | Añadir un Pokémon de la misma especie en el equipo.                                                              | El Pokémon seleccionado aparece en la ranura correspondiente. | El Pokémon seleccionado aparece en la ranura correspondiente. | El Pokémon seleccionado aparece en la ranura correspondiente. |
| 8. Eliminar equipo | Eliminar un equipo, seleccionando “Aceptar” en el cuadro de texto.                                               | Se borra el equipo de su lista de equipos.                    | Se borra el equipo de su lista de equipos.                    | Se borra el equipo de su lista de equipos.                    |
| 9. Eliminar equipo | Eliminar un equipo, seleccionando “Cancelar” en el cuadro de texto.                                              | Se cancela la operación y no cambia la lista de equipos       | Se cancela la operación y no cambia la lista de equipos       | Se cancela la operación y no cambia la lista de equipos       |
| 10. Navegación     | Pulsar la flecha de la esquina superior derecha, estando en la lista de equipos.                                 | Se vuelve al menú principal.                                  | Se vuelve al menú principal.                                  | Se vuelve al menú principal.                                  |
| 11. Navegación     | Pulsar la flecha de la esquina superior izquierda, estando en pantalla de captura de Pokémons (ilustración 16) . | Se vuelve a la pantalla de detalles del Pokémon.              | Se vuelve a la pantalla de detalles del Pokémon.              | Se vuelve a la pantalla de detalles del Pokémon.              |



### FUNCIONALIDAD 3: ChangeLog - Ziyan Jiang

| Código de Prueba                          | Descripción                                                                                                           | Resultado esperado                                                                                     | Resultado obtenido | Observaciones                                                       |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------|---------------------------------------------------------------------|
| 1. Acceso y visualización básica          | Acceder a la opción “Ver ChangeLog”, sin iniciar sesión.                                                              | Redirige al login (código 302), porque no cumple la precondition.                                      | 302                | Correcto                                                            |
| 2. Acceso y visualización básica          | Acceder a la opción “Ver ChangeLog” en el menú principal, iniciando sesión.                                           | Accede correctamente (código 200).                                                                     | 200                | Correcto                                                            |
| 3. ChangeLog vacío                        | Acceder a la opción “Ver ChangeLog”. Sin notificaciones.                                                              | Se muestra una lista de eventos vacía o un mensaje indicando que no hay notificaciones recientes.      | Lista vacía.       | Correcto                                                            |
| 4. Límite de notificaciones               | Acceder a la opción “Ver ChangeLog”. Ahora con de 20 notificaciones                                                   | El sistema muestra las últimas 20 notificaciones.                                                      | 20 notificaciones. | Correcto                                                            |
| 5. Límite de notificaciones               | Acceder a la opción “Ver ChangeLog”. Ahora con más de 20 notificaciones                                               | El sistema sólo muestra las últimas 20 notificaciones.                                                 | 20 notificaciones  | Correcto                                                            |
| 6. Filtrar por usuario                    | Acceder a la opción “Ver ChangeLog” y seleccionar la opción de filtro por “Usuario” y seleccionar el usuario deseado. | La lista de eventos se actualiza para mostrar sólo los eventos realizados por el usuario seleccionado. | -                  | Se utiliza un script en la vista. No se puede automatizar.          |
| 7. Filtrar por el tipo de cambio: equipos | Acceder a la opción “Ver ChangeLog” y aplicar el filtro de “Creación de equipos”.                                     | La lista de eventos se actualiza para mostrar sólo los eventos de creación de equipos.                 | -                  | La categoría se envía bien, pero el filtro no se puede automatizar. |
| 8. Filtrar por el tipo de                 | Acceder a la opción “Ver                                                                                              | La lista de eventos se actualiza para mostrar                                                          | -                  | La categoría se envía bien,                                         |

|                                                    |                                                                                     |                                                                                      |   |                                                                                 |
|----------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---|---------------------------------------------------------------------------------|
| cambio:<br>captura                                 | ChangeLog” y<br>aplicar el filtro<br>por “Capturas”.                                | sólo los eventos de<br>capturas.                                                     |   | pero el filtro<br>no se puede<br>automatizar.                                   |
| 9. Filtrar por el<br>tipo de<br>cambio:<br>seguido | Acceder a la<br>opción “Ver<br>ChangeLog” y<br>aplicar el filtro<br>por “Seguidos”. | La lista de eventos se<br>actualiza para mostrar<br>sólo los eventos de<br>capturas. | - | La categoría<br>se envía bien,<br>pero el filtro<br>no se puede<br>automatizar. |
| 10. Filtrar y<br>resultado<br>vacío                | Acceder a la<br>opción “Ver<br>ChangeLog” y<br>aplicar un filtro<br>cualquiera.     | La lista de eventos<br>aparece vacía o un<br>mensaje: “Sin<br>resultado”.            | - | Los filtros no<br>se pueden<br>automatizar.                                     |

## FUNCIONALIDAD 4: Lista Completa de Pokémon y Búsquedas con Filtros - Adrián Vinagre

| Código de Prueba                    | Descripción                                                                                             | Resultado esperado                                                                                                                                                                                                                          | Resultado obtenido | Observaciones |
|-------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|---------------|
| 1. Ver lista de todos los Pokémon . | Desde el menú principal, hacer click en el botón de “Ver lista de todos los Pokémon”.                   | Debe aparecer la interfaz inicial de la funcionalidad, en la que se muestra una lista deslizable de todos los Pokémons. Además, aparece una barra de búsqueda y un botón para filtrar.                                                      |                    |               |
| 2. Ver información de Pokémon .     | Desde el menú inicial de “Ver lista de todos los Pokémon”, hacer click sobre cualquiera de los Pokémon. | Debe aparecer una pantalla en la que se muestre la información sobre la especie de Pokémon correspondiente al Pokémon sobre el que se ha hecho click.                                                                                       |                    |               |
| 3. Filtrar: interfaz.               | Desde el menú inicial de “Ver lista de todos los Pokémon”, hacer click sobre el botón filtrar.          | Debe aparecer una lista de todos los posibles filtros que se han querido añadir a la aplicación.                                                                                                                                            |                    |               |
| 4. Filtrar: selección de filtros.   | Desde la lista de filtros, al hacer click sobre cualquier filtro.                                       | Debe resaltarse el filtro sobre el que se ha hecho click, para hacer ver que está seleccionado. Al hacer click en otros filtros, éstos también deben resaltarse. Cuando se apliquen los filtros, la aplicación debe detectar que se aplican |                    |               |

|                                         |                                                                             |                                                                                                                                                                                                                                                                                                      |  |  |
|-----------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
|                                         |                                                                             | únicamente los filtros seleccionados.                                                                                                                                                                                                                                                                |  |  |
| 5. Filtrar: deseleccionar filtros.      | Desde la lista de filtros, al hacer click sobre cualquier filtro resaltado. | Debe dejar de resaltarse el filtro sobre el que se ha hecho click (si es que estaba resaltado), para hacer ver que ya no está seleccionado. Cuando se apliquen los filtros, la aplicación debe detectar que se aplican únicamente los filtros seleccionados, y ya no se aplican los deseleccionados. |  |  |
| 6. Filtrar: Aplicar filtros             | Desde la lista de filtros, al hacer click sobre el botón “aplicar”.         | Se debe cerrar automáticamente la lista de filtros, y se debe mostrar la nueva lista de Pokémons actualizada con los filtros.                                                                                                                                                                        |  |  |
| 7. Filtrar: Aplicar filtros, 0 filtros. | Tras la situación del caso de prueba nº 6, si no se aplican filtros.        | Se muestra la lista de Pokémons inicial, sin cambios, con todos los Pokémon.                                                                                                                                                                                                                         |  |  |
| 8. Filtrar: Aplicar filtros, 1 filtro.  | Tras la situación del caso de prueba nº 6, si solo se aplica un filtro.     | Se muestra la lista de Pokémons que cumplen solamente la condición del filtro aplicado.                                                                                                                                                                                                              |  |  |
| 9. Filtrar: Aplicar filtros, 2          | Tras la situación del caso de prueba nº 6, si se                            | Se muestra la lista de Pokémons que                                                                                                                                                                                                                                                                  |  |  |

|                                                     |                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                |  |  |
|-----------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| filtros o más.                                      | aplican 2 filtros o más.                                          | cumplen únicamente la condición de los filtros aplicados. Por ejemplo, si se aplican los filtros: tipo fuego, tipo roca, habilidad avalancha; Entonces se muestra la lista de Pokémons que son de tipo fuego y roca (los dos, no vale con ser uno solo), y que además pueden aprender la habilidad “avalancha”. Si un pokémon tipo fuego y roca no puede aprender “avalancha”, no se mostrará. |  |  |
| 10. Ver información de Pokémon tras filtrar.        | Después de aplicar filtros, hacer click sobre cualquier Pokémon.  | Debe aparecer una pantalla en la que se muestre la información sobre la especie de Pokémon correspondiente al Pokémon sobre el que se ha hecho click.                                                                                                                                                                                                                                          |  |  |
| 11. Filtrar: Interfaz, tras haber aplicado filtros. | Después de aplicar filtros, hacer click sobre el botón “filtrar”. | Debe aparecer la lista de todos los filtros posibles que se han querido añadir a la aplicación, y también deben aparecer marcados los filtros que han sido aplicados la última vez.                                                                                                                                                                                                            |  |  |
| 12. Cerrar lista de todos los Pokémon               | Pulsando la flecha que se muestra en la interfaz de la            | Se debe cerrar dicha interfaz y se debe poder regresar al menú                                                                                                                                                                                                                                                                                                                                 |  |  |

|   |                                  |                                                                                                                                                                                                                                                                                                                                                                                        |  |  |
|---|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|
| . | funcionalidad 4, en una esquina. | principal de la aplicación, reiniciando la lista de Pokémon y los filtros aplicados. Por ejemplo, si antes se mostraba la lista con los filtros: tipo planta y tipo veneno, al cerrar la lista, se deben deseleccionar todos esos tipos, y se deben desmarcar. De esta manera, si se vuelve a abrir la lista de todos los Pokémons, aparecerá la lista inicial, sin filtros aplicados. |  |  |
|---|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|

## FUNCIONALIDAD 5: ChatBot - Diego Pomares

| Código de Prueba               | Descripción                                                                                                                                                          | Resultado esperado                                                                                                                             | Resultado obtenido                                                       | Observaciones |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|---------------|
| 1. Mostrar mejor pokemon       | Se le pide al bot que genere la vista (mejor pokemon o consultar info de pokemon) cuando el usuario no tiene ningún equipo pokémon o ningún pokémon respectivamente. | El bot dice un mensaje de error indicando la ausencia de pokemon/equipo.                                                                       | Mensaje de error                                                         | Correcto      |
| 2. Mostrar estadísticas        | Se intenta acceder a las estadísticas y hay un atributo ausente en la base de datos.                                                                                 | Se pueden ver todas las estadísticas excepto ese atributo, cuyo valor se representa con una señal (NotFound, Unavailable, etc..).              | Se muestra la estadística con un valor ausente                           | Correcto      |
| 3. Mostrar mejor pokemon       | Al preguntar que pokemon es el mejor, puede que haya uno o más pokemons que “empaten” o que sean idénticos.                                                          | Una solución sería enseñar ambos como equivalentes o a la par. También se puede elegir un pokémon por ejemplo por su nombre (alfabéticamente). | Se devuelve el primer pokemon de la tabla                                | Correcto      |
| 4. Mostrar cadena evolutiva    | Un pokémon no tiene cadena evolutiva                                                                                                                                 | Lo ideal sería que no hubiera botón en primer lugar.                                                                                           | La cadena evolutiva se muestra con un solo pokemon                       | Correcto      |
| 5. Mostrar counters (+/-rival) | No es posible encontrar unos malos/buenos rivales (por lo que sea).                                                                                                  | Esto es prácticamente imposible, pero aparece un mensaje: “No se ha podido encontrar un buen/mal rival”.                                       | Siempre aparecen un par de pokemons. (suponiendo que la BD esta cargada) | Correcto      |

## **FUNCIONALIDAD 6: Compartir equipo a través de Telegram - Sofía Granja**

| Código de Prueba | Descripción                                                                                                               | Resultado esperado                                                                                                |
|------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 1                | Se le pide al bot que mande la vista a un usuario cuya ID no está relacionada con ningún usuario dentro de la aplicación. | El bot dice un mensaje de error: "Tu ID no está vinculada a ningún usuario en la app".                            |
| 2                | Se le pide al bot que genere la vista cuando el usuario no tiene ningún equipo pokémon.                                   | El bot dice un mensaje de error: "No tienes ningún equipo pokémon"                                                |
| 3                | Ocurre un error al intentar vincular la cuenta de telegram a la aplicación.                                               | Se notifica un error dentro de la aplicación y se vuelven a vaciar los formularios para que se vuelva a intentar. |
| 4                | Error de conexión Telegram->App.                                                                                          | El bot dice un mensaje de error: "Ha habido un error, intentelo más tarde".                                       |



## FUNCIONALIDAD 7: Pokedle - Mateo Ortiz

| Código de Prueba | Descripción                                                 | Resultado esperado                                                   | Resultado obtenido                                                   | Observaciones |
|------------------|-------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------|---------------|
| 1 Adivinar       | Introduces un pokemon ya probado                            | Mensaje de error y te deja seguir jugando                            | No lo repite y te deja seguir jugando                                | Correcto      |
| 2 Adivinar       | Introduces un pokemon incorrecto (no existe)                | Mensaje de error y te deja seguir jugando                            | Mensaje diciendo que no existe y te deja seguir jugando              | Correcto      |
| 3 Adivinar       | Adivinas sin haber escrito nada                             | Mensaje de error y te deja seguir jugando                            | no hace nada y sigues jugando                                        | Correcto      |
| 4 Adivinar       | Introduces el pokemon correcto                              | Mensaje de felicitación y te deja volver a jugar o salir             | Mensaje de felicitacion y opcion de volver a jugar o salir           | Correcto      |
| 5 Adivinar       | Introduces un pokemon que no es el objetivo por primera vez | Te da las pistas y te deja seguir jugando                            | te da las pistas y te deja seguir jugando                            | Correcto      |
| 6 Rendirse       | Te rindes (mediante el botón)                               | Te da el pokémon objetivo y te deja volver a jugar o salir           | Te da el pokémon objetivo y te deja volver a jugar o salir           | Correcto      |
| 7 Salir          | Sales de la pestaña y vuelves a entrar                      | Se pierde el progreso y se empieza de nuevo con un pokémon aleatorio | Se pierde el progreso y se empieza de nuevo con un pokémon aleatorio | Correcto      |

## 8. DESCRIPCIÓN DE LOS PROBLEMAS DE IMPLEMENTACIÓN

Durante la fase de implementación se han encontrado varios problemas derivados de la interacción entre los diagramas de diseño y la base de código. Los diagramas habían sido creados durante meses para servir de guía a la estructura y la lógica del proyecto.

Sin embargo, existía una base de código anterior a la creación de los diagramas, y generó inconsistencias entre la planificación y la implementación real. Dicho código fué muy útil para saber qué estructura seguir, pero también provocó confusión al momento de traducir los diagramas a funciones y servicios concretos, especialmente en la definición de clases, métodos y estructuras de datos, ya que algunas decisiones de diseño previas no coincidían con la nueva arquitectura planteada en los diagramas.

Otro problema importante fue la falta de uniformidad en los nombres de parámetros y estructuras de datos. Eso hizo más difícil la integración de los distintos módulos. Por ejemplo, diferencias en la nomenclatura de atributos entre Pokémon y especies, o la gestión de conexiones a la base de datos. Esta situación resaltó la importancia de sincronizar la documentación y los diagramas con la base de código, y la necesidad de establecer estándares claros de implementación para evitar malentendidos dentro del equipo.

Finalmente, se han logrado solventar la mayoría de problemas de implementación que mayor efecto negativo tenían sobre el código final, pero se tuvo que emplear tiempo y esfuerzo extra comparado con el que se hubiese necesitado para implementar todo con los diagramas ya planteados desde el principio, en una situación normal.

## 9. CONCLUSIONES

- 9.1 Logros principales

Se entregó una aplicación funcional con arquitectura MVC que incorpora:

- Autenticación segura con roles y flujo de aprobación.
- Catálogo de especies (Pokédex) con búsqueda y filtros por tipo.
- Integración con PokeAPI.
- Un ChatBot con comandos útiles
- Gestión de equipos de Pokémon y una UI que facilita seleccionar un equipo
- Tests unitarios específicos que cubren comportamientos.

- 9.2 Lecciones aprendidas

- Importancia de diseñar desde el inicio
- Elegir tecnologías acordes al entorno: SQLite es excelente para desarrollo y prototipado pero obliga a planificar la migración a una BD más robusta para producción.
- Manejar dependencias y entornos: recomendaciones para usar virtualenv/venv y documentación clara facilita el trabajo.
- Buena comunicación de equipo y control de cambios (ramas y PRs) acelera integración y reduce conflictos.

- 9.3 Limitaciones

La aplicación está preparada para entorno educativo o prototipo; para producción hay trabajo pendiente:

- Migrar DB a Postgres u otro SGBD con soporte de concurrencia.
- Mejorar la gestión de secretos, logging estructurado y monitorización.
- Optimizar el rendimiento (caché).

- 9.4 Recomendaciones para futuras iteraciones

Infraestructura y despliegue:

- Configurar CI (GitHub Actions) con lint, tests y despliegue automatizado.
- Desplegar en contenedor (Docker) y orquestación si se escala (K8s/Platform-as-a-Service).

Calidad y pruebas:

- Añadir tests de integración que ejecuten flujos completos (login → crear equipo → calcular mejor Pokémon).
- Introducir pytest y fixtures para facilitar pruebas y parametrización.

Observabilidad y seguridad:

- Añadir logging centralizado, métricas y alertas; forzar HTTPS y revisar headers de seguridad.

Datos y rendimiento:

- Añadir índices y una estrategia de caché para llamadas a PokeAPI.

UX:

- Refinar interfaz: paginación, autocompletado en búsquedas, notificaciones, y manejo de estados (loading, error).

## ● 9.5 Cierre

El proyecto cumple los objetivos principales: ofrece una Pokédex interactiva, con sus funcionalidades. El trabajo proporciona una base sólida para expandir la aplicación: mejorar pruebas, endurecer la seguridad y preparar el despliegue profesional. Con la continuación adecuada, la aplicación puede evolucionar de prototipo educativo a una solución lista para producción.